

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 21 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention-2split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_CUDNN)
130 #pragma nv_diag_suppress 128
131 #include <cuda/cudnn_frontend.h>
132 #pragma nv_diag_default 128
133 namespace fe = cudnn_frontend;
134 #endif
135
136 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
137 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
138 #endif
139
140 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
141 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
142 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
143
144 static constexpr int kWarpSize = 32;
145
146 // =====
147 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
148 // =====
149
150 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
151 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
152 // 复杂度  $O(\log N)$ ,  $N=32$  时只需 5 步
153 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
154 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
155 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
156 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
157 //
158 // 初始: 每个 lane 持有自己的值 v0..v7
159 // lane: 0 1 2 3 4 5 6 7
160 // val: v0 v1 v2 v3 v4 v5 v6 v7
161 //
162 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
163 //
164 //      ┌──────────┐
165 // 对: (0,4) (1,5) (2,6) (3,7)
166 //
167 // lane: 0 1 2 3 4 5 6 7
168 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
169 //
170 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
171 //
172 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
173 // 对: (0,2) (1,3) (4,6) (5,7)
174 //      ─── 前4个一组 ───      ─── 后4个一组 ───
175 //
176 // lane: 0 1 2 3 4 5 6 7 (每 lane 持有前一轮2个值的和再加本轮配对)
177 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
178 //      = v0+v2+v4+v6 ... (逐步归约)
179 //
180 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
181 //
182 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
183 // 对: (0,1) (2,3) (4,5) (6,7)
184 //
185 // lane: 0 1 2 3 4 5 6 7
186 // val:  $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
187 //
188 // mask=0: 循环终止, 归约完成。
189 //
190 // 关键性质:
191 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 

```

```

189 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
190 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
191 // - 0(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
192 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
193 // 当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
194 template <const int kWarpWidth = kWarpSize, typename T = float>
195 __device__ __forceinline__ T warp_reduce_sum(T val) {
196 #pragma unroll
197     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
198         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
199     }
200     return val;
201 }
202
203 // Warp Reduce Max — generic
204 template <const int kWarpWidth = kWarpSize, typename T = float>
205 __device__ __forceinline__ T warp_reduce_max(T val) {
206 #pragma unroll
207     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
208         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
209     }
210     return val;
211 }
212
213 // =====
214 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
215 // =====
216
217 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
218 // 两级归约流程:
219 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
220 // 2. warp leader (lane=0) 写入 shared memory
221 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
222 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
223 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
224 template <const int kNumThreads = 256>
225 __device__ float block_reduce_sum(float val) {
226     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
227     int warp = threadIdx.x / kWarpSize;
228     int lane = threadIdx.x % kWarpSize;
229     __shared__ float shared[kNumWarps];
230
231     float value = warp_reduce_sum<kWarpSize>(val);
232     if (lane == 0)
233         shared[warp] = value;
234     __syncthreads();
235     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
236     value = warp_reduce_sum<kNumWarps>(value);
237     // 关键: broadcast 结果到所有线程, 后续用 result
238     // 做除法等操作时所有线程都能拿到
239     value = __shfl_sync(0xffffffff, value, 0, 32);
240     return value;
241 }
242
243 // Block Reduce Max — FP32 (增强版, 带 broadcast)
244 template <const int kNumThreads = 256>
245 __device__ float block_reduce_max(float val) {
246     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
247     int warp = threadIdx.x / kWarpSize;
248     int lane = threadIdx.x % kWarpSize;
249     __shared__ float shared[kNumWarps];
250
251     float value = warp_reduce_max<kWarpSize>(val);

```

```

252     if (lane == 0)
253         shared[warp] = value;
254     __syncthreads();
255     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
256     value = warp_reduce_max<kNumWarps>(value);
257     value = __shfl_sync(0xffffffff, value, 0, 32);
258     return value;
259 }
260
261 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
262 // 多 block 各自做 warp->smem->warp0 reduce, 然后 atomicAdd 到全局 y
263 // 跨 block 求和的常见模式, 适合 N 较大时使用;
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (256, 1, 1)
266 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
267 template <const int kNumThreads = 256>
268 __global__ void block_reduce_all(float *a, float *y, int N) {
269     int tid = threadIdx.x;
270     int idx = blockIdx.x * kNumThreads + tid;
271     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
272     __shared__ float shared[kNumWarps];
273
274     float val = (idx < N) ? a[idx] : 0.0f;
275     int warp = tid / kWarpSize;
276     int lane = tid % kWarpSize;
277
278     val = warp_reduce_sum<kWarpSize>(val);
279     if (lane == 0)
280         shared[warp] = val;
281     __syncthreads();
282
283     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
284     if (warp == 0)
285         val = warp_reduce_sum<kNumWarps>(val);
286     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
287         atomicAdd(y, val);
288 }
289
290 // ---- Dot Product: y = sum(a[i] * b[i]) ----
291 // 核心模式: elementwise 乘法 -> block reduce -> atomicAdd 全局累加
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (256, 1, 1)
294 // source: LeetCUDA/kernels/dot-product/dot_product.cu
295 template <const int kNumThreads = 256>
296 __global__ void dot(float *a, float *b, float *y, int N) {
297     int tid = threadIdx.x;
298     int idx = blockIdx.x * kNumThreads + tid;
299     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
300     __shared__ float shared[kNumWarps];
301
302     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
303     int warp = tid / kWarpSize;
304     int lane = tid % kWarpSize;
305
306     prod = warp_reduce_sum<kWarpSize>(prod);
307     if (lane == 0)
308         shared[warp] = prod;
309     __syncthreads();
310
311     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
312     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
313         prod = warp_reduce_sum<kNumWarps>(prod);
314     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加

```

```

315     atomicAdd(y, prod);
316 }
317
318 // Dot Product + float4
319 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
320 // Block: (64, 1, 1), 256/4=64
321 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
322 // source: LeetCUDA/kernels/dot-product/dot_product.cu
323 template <const int kNumThreads = 256 / 4>
324 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
325     int tid = threadIdx.x;
326     int idx = (blockIdx.x * kNumThreads + tid) * 4;
327     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
328     __shared__ float shared[kNumWarps];
329
330     float4 reg_a = FLOAT4(a[idx]);
331     float4 reg_b = FLOAT4(b[idx]);
332     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
333                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
334                       : 0.0f;
335     int warp = tid / kWarpSize;
336     int lane = tid % kWarpSize;
337
338     prod = warp_reduce_sum<kWarpSize>(prod);
339     if (lane == 0)
340         shared[warp] = prod;
341     __syncthreads();
342
343     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
344     if (warp == 0)
345         prod = warp_reduce_sum<kNumWarps>(prod);
346     if (tid == 0)
347         atomicAdd(y, prod);
348 }
349
350
351 // =====
352 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
353 // =====
354 // 面试要点:
355 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
356 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
357 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
358
359 // ---- ReLU: y = max(0, x) ----
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (256, 1, 1)
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu(float *x, float *y, int N) {
364     int idx = blockIdx.x * blockDim.x + threadIdx.x;
365     if (idx < N)
366         y[idx] = fmaxf(0.0f, x[idx]);
367 }
368
369 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
370 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
371 // Grid: ((N + 255) / 256, 1, 1)
372 // Block: (64, 1, 1)
373 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
374 // source: LeetCUDA/kernels/relu/relu.cu
375 __global__ void relu_vec4(float *x, float *y, int N) {
376     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
377     if (idx < N) {

```

```

378     float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
379     float4 reg_y;
380     reg_y.x = fmaxf(0.0f, reg_x.x);
381     reg_y.y = fmaxf(0.0f, reg_x.y);
382     reg_y.z = fmaxf(0.0f, reg_x.z);
383     reg_y.w = fmaxf(0.0f, reg_x.w);
384     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
385 }
386 }
387
388 // ---- Elementwise Add: c = a + b ----
389 // Grid: ((N + 255) / 256, 1, 1)
390 // Block: (256, 1, 1)
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
393     int idx = blockIdx.x * blockDim.x + threadIdx.x;
394     if (idx < N)
395         c[idx] = a[idx] + b[idx];
396 }
397
398 // Elementwise Add + float4 向量化
399 // Grid: ((N + 255) / 256, 1, 1)
400 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
401 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
402 // source: LeetCUDA/kernels/elementwise/elementwise.cu
403 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
404     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
405     if ((idx + 3) < N) {
406         float4 reg_a = FLOAT4(a[idx]);
407         float4 reg_b = FLOAT4(b[idx]);
408         float4 reg_c;
409         reg_c.x = reg_a.x + reg_b.x;
410         reg_c.y = reg_a.y + reg_b.y;
411         reg_c.z = reg_a.z + reg_b.z;
412         reg_c.w = reg_a.w + reg_b.w;
413         FLOAT4(c[idx]) = reg_c;
414     } else if (idx < N) {
415         for (int i = 0; (idx + i) < N; i++) {
416             c[idx + i] = a[idx + i] + b[idx + i];
417         }
418     }
419 }
420
421 // ---- Histogram: y[a[i]]++ ----
422 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
423 // Grid: ((N + 255) / 256, 1, 1)
424 // Block: (256, 1, 1)
425 // source: LeetCUDA/kernels/histogram/histogram.cu
426 __global__ void histogram(int *a, int *y, int N) {
427     int idx = blockIdx.x * blockDim.x + threadIdx.x;
428     if (idx < N)
429         atomicAdd(&(y[a[idx]]), 1);
430 }
431
432 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
433 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
434 //
435 // 面试要点:
436 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
437 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
438 //   - 数学公式 (5 步推导):
439 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
440 //         L_max = max(LSE_1, LSE_2)

```



```

441 // Step 2 — 还原指数权重 (un-normalized) :
442 //      w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
443 // Step 3 — 归一化为混合比例:
444 //      alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
445 //      满足 alpha + beta = 1
446 // Step 4 — 逐元素加权合并:
447 //      O = alpha * O_1 + beta * O_2
448 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention-2惯例一致,
449 //   lse[head_idx][token_idx])
450 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
451 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
452 // -  $-\infty$  LSE  $\rightarrow$   $-\infty$ : 空 attention 段 (causal mask 等导致全部 score
453 //   为  $-\infty$ ) 的 LSE 可能为  $+\infty$ , 替换后  $\exp(-\infty - L_{\max}) = 0$ ,
454 //   该段权重退化为 0
455 // - 128-bit 向量化: uint4 = 4  $\times$  float, 每线程处理 4 个输出元素,
456 //   pack load  $\rightarrow$  FMA  $\rightarrow$  pack store 一条流水线
457 // - AI  $\approx$  3 FLOP / 20 bytes  $\approx$  0.15 FLOPS/Byte  $\rightarrow$  severely memory-bound
458 //
459 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
460 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
461 // threads_per_head = head_size / 4 = 2
462 // total_threads = num_tokens * num_heads * threads_per_head = 8
463 //
464 // global_idx:      0      1      2      3      4      5      6      7
465 // token_head_idx:  0      0      1      1      2      2      3      3
466 //   (第几个 (token,head) 对, 行优先)
467 // pack_idx:        0      1      0      1      0      1      0      1
468 //   (该 (token,head) 对内第几个 pack)
469 // token_idx:       0      0      0      0      1      1      1      1
470 //   (token_head_idx / num_heads, token 变化慢)
471 // head_idx:        0      0      1      1      0      0      1      1
472 //   (token_head_idx % num_heads, head 变化快)
473 // pack_offset:     0      4      0      4      0      4      0      4
474 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
475 // head_offset:     0      0      8      8      16     16     24     24
476 //   (token_idx * num_heads * head_size + head_idx * head_size,
477 //   该 (token,head) 对在 output 展平数组中的起始偏移)
478 //
479 // Grid: ((total_threads + 127) / 128, 1, 1)
480 // Block: (128, 1, 1)
481 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
482 __global__ void merge_attn_states(
483     float *output,           // [num_tokens, num_heads, head_size]
484     const float *prefix_output, // [num_tokens, num_heads, head_size]
485     const float *prefix_lse,   // [num_heads, num_tokens]
486     const float *suffix_output, // [num_tokens, num_heads, head_size]
487     const float *suffix_lse,   // [num_heads, num_tokens]
488     int num_tokens, int num_heads, int head_size) {
489
490     constexpr int kNumThreads = 128;
491     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
492     using pack_t = uint4;
493
494     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
495     const int threads_per_head = head_size / kPackSize;
496     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
497     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
498     const int total_threads = num_tokens * num_heads * threads_per_head;
499
500     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
501     if (global_idx >= total_threads)
502         return;
503

```

```

504 // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
505 // token_head_idx: 第几个 (token, head) 对, 行优先展平
506 const int token_head_idx = global_idx / threads_per_head;
507 // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
508 const int pack_idx = global_idx % threads_per_head;
509
510 // token_head_idx 分解为 (token_idx, head_idx)
511 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
512 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
513
514 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
515 const int pack_offset = pack_idx * kPackSize;
516 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
517 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
518
519 // 定位到当前 (token, head) 的输出段起始
520 const float *prefix_head = prefix_output + head_offset;
521 const float *suffix_head = suffix_output + head_offset;
522 float *output_head = output + head_offset;
523
524 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
525 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
526 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
527
528 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
529 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
530 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
531
532 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
533 const float max_lse = fmaxf(p_lse, s_lse);
534 p_lse -= max_lse;
535 s_lse -= max_lse;
536
537 // Step 2-3: 指数还原 → 混合比例 alpha, beta
538 const float p_se = expf(p_lse);
539 const float s_se = expf(s_lse);
540 const float out_se = p_se + s_se;
541 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
542 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
543
544 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
545 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
546 if (pack_offset < head_size) {
547     pack_t p_pack =
548         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
549     pack_t s_pack =
550         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
551     pack_t o_pack;
552
553     #pragma unroll
554     for (int i = 0; i < kPackSize; ++i) {
555         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
556         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
557         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
558         reinterpret_cast<float *>(&o_pack)[i] = o_v;
559     }
560
561     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
562 }
563 }
564
565 // =====
566 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm

```

```

567 // =====
568 // 面试要点:
569 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
570 //   online (1-pass, 增量更新)
571 //   - Online Softmax 是 FlashAttention-2的数学基础
572 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
573 //   + variance)
574 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
575
576 // =====
577 // Phase 3a: Softmax — 三级递进 (面试核心考点)
578 // =====
579 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
580 // 回答线索: naive(溢出) → safe → online
581
582 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
583 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
584 // 问题: x 值很大时 exp(x) 溢出为 inf
585 template <const int kNumThreads = 256>
586 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
587 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
588 // source: LeetCUDA/kernels/softmax/softmax.cu
589 __global__ void softmax_per_token(float *x, float *y, int N) {
590     const int tid = threadIdx.x;
591     const int idx = blockIdx.x * blockDim.x + tid;
592
593     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
594     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
595     if (idx < N)
596         y[idx] = exp_val / exp_sum;
597 }
598
599 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
600 // 面试重点: 为什么 Softmax 需要 Safe?
601 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
602 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
603 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
604 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
605 // Grid: (S, 1, 1)
606 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
607 // source: LeetCUDA/kernels/softmax/softmax.cu
608 template <const int kNumThreads = 256>
609 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
610     const int tid = threadIdx.x;
611     const int idx = blockIdx.x * blockDim.x + tid;
612
613     // Pass 1: block reduce max — 找最大值
614     float val = (idx < N) ? x[idx] : -FLT_MAX;
615     float max_val = block_reduce_max<kNumThreads>(val);
616
617     // Pass 2: exp(x - max) → block reduce sum
618     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
619     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
620
621     if (idx < N)
622         y[idx] = exp_val / exp_sum;
623 }
624
625 // ---- Level 3: Online Safe Softmax (FlashAttention-2的数学基础) ----
626 // 面试重点:
627 // 两种使用场景 (公式不同, 但结果等价) :
628 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
629 //       m_new = max(m_old, x_i)

```

```

630 //      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
631 //  2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
632 //      m = max(m1, m2) safe softmax用的max值
633 //      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
634 //      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
635 //  算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
636 //  Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
637 //
638 //  不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
639 //  warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
640 //  (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
641 //  (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
642 //
643 //  合并公式 (对称, 不依赖"新/旧"概念) :
644 //  m = max(m1, m2)
645 //  d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
646 //
647 //  该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
648 struct __align__(8) MD {
649     float m; // running max
650     float d; // running denominator (sum of exp(x - max))
651 };
652
653 template <const int kWarpWidth = kWarpSize>
654 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
655     #pragma unroll
656     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
657         MD md2;
658         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
659         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
660
661         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
662         MD s_m = (md1.m > md2.m) ? md2 : md1;
663
664         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
665         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
666         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
667         md1.m = b_m.m;
668     }
669     return md1;
670 }
671
672 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
673 // Grid: (S, 1, 1)
674 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
675 // source: LeetCUDA/kernels/softmax/softmax.cu
676 template <const int kNumThreads = 256>
677 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
678     int tid = threadIdx.x;
679     int idx = blockIdx.x * kNumThreads + threadIdx.x;
680     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
681     __shared__ MD shared[kNumWarps];
682
683     float val = (idx < N) ? x[idx] : -FLT_MAX;
684     int warp = tid / kWarpSize;
685     int lane = tid % kWarpSize;
686
687     // 初始化: 每个线程持有一个 (max, denom) 对
688     MD md;
689     md.m = idx < N ? val : -FLT_MAX;
690     md.d = idx < N ? 1.0f : 0.0f;
691
692     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d

```

```

693 md = warp_reduce_md<kWarpSize>(md);
694
695 if (lane == 0)
696     shared[warp] = md;
697 __syncthreads();
698
699 // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
700 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
701 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
702
703 // 用全局 max 和 denom 做最终 softmax
704 float d_inv = __fdivdef(1.0f, md.d);
705 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
706 if (idx < N) {
707     y[idx] = __expf(val - md.m) * d_inv;
708 }
709 }
710
711 // =====
712 // Phase 3b: RMS Normalization (1-pass reduce)
713 // =====
714 // 面试要点:
715 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
716 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
717 //   - Llama 系列使用 RMS Norm
718 //   - grid(N, K/K), block(K): 一行一个 block
719 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
720 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
721 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
722 template <const int kNumThreads = 128>
723 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
724     int tid = threadIdx.x;
725     int idx = blockIdx.x * blockDim.x + threadIdx.x;
726     const float epsilon = 1e-5f;
727
728     __shared__ float s_variance;
729     float value = (idx < N * K) ? x[idx] : 0.0f;
730     float variance = value * value;
731     variance = block_reduce_sum<kNumThreads>(variance);
732     if (tid == 0)
733         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
734     __syncthreads();
735     if (idx < N * K)
736         y[idx] = (value * s_variance) * g;
737 }
738
739 // RMS Norm + float4
740 // Grid: (N, 1, 1)
741 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
742 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
743 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
744 template <const int kNumThreads = 128 / 4>
745 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
746     int tid = threadIdx.x;
747     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
748     const float epsilon = 1e-5f;
749
750     __shared__ float s_variance;
751     float4 reg_x = FLOAT4(x[idx]);
752     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
753                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
754                                     : 0.0f;
755     variance = block_reduce_sum<kNumThreads>(variance);

```

```

756     if (tid == 0)
757         s_variance = rsqrtf(variance / (float)K + epsilon);
758     __syncthreads();
759     float4 reg_y;
760     reg_y.x = reg_x.x * s_variance * g;
761     reg_y.y = reg_x.y * s_variance * g;
762     reg_y.z = reg_x.z * s_variance * g;
763     reg_y.w = reg_x.w * s_variance * g;
764     if (idx < N * K)
765         FLOAT4(y[idx]) = reg_y;
766 }
767
768 // =====
769 // Phase 3c: Layer Normalization (2-pass reduce)
770 // =====
771 // 面试要点:
772 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
773 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
774 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
775 // Grid: (N, 1, 1), 一行一个 block
776 // Block: (128, 1, 1), kNumThreads=K=128
777 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
778 template <const int kNumThreads = 128>
779 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
780     int tid = threadIdx.x;
781     int idx = blockIdx.x * blockDim.x + threadIdx.x;
782     const float epsilon = 1e-5f;
783
784     __shared__ float s_mean;
785     __shared__ float s_variance;
786     float value = (idx < N * K) ? x[idx] : 0.0f;
787
788     // Pass 1: compute mean
789     float sum = block_reduce_sum<kNumThreads>(value);
790     if (tid == 0)
791         s_mean = sum / (float)K;
792     __syncthreads(); // 必须等待 s_mean 对所有线程可见
793
794     // Pass 2: compute variance = (x - mean)2
795     float variance = (value - s_mean) * (value - s_mean);
796     variance = block_reduce_sum<kNumThreads>(variance);
797     if (tid == 0)
798         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
799     __syncthreads(); // 必须等待 s_variance 对所有线程可见
800
801     if (idx < N * K)
802         y[idx] = ((value - s_mean) * s_variance) * g + b;
803 }
804
805 // Layer Norm + float4
806 // Grid: (N, 1, 1)
807 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
808 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
809 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
810 template <const int kNumThreads = 128 / 4>
811 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
812     int tid = threadIdx.x;
813     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
814     const float epsilon = 1e-5f;
815
816     __shared__ float s_mean;
817     __shared__ float s_variance;
818     float4 reg_x = FLOAT4(x[idx]);

```

```

819 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
820
821 float sum = block_reduce_sum<kNumThreads>(value);
822 if (tid == 0)
823     s_mean = sum / (float)K;
824 __syncthreads();
825
826 float4 reg_x_hat;
827 reg_x_hat.x = reg_x.x - s_mean;
828 reg_x_hat.y = reg_x.y - s_mean;
829 reg_x_hat.z = reg_x.z - s_mean;
830 reg_x_hat.w = reg_x.w - s_mean;
831 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
832                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
833 variance = block_reduce_sum<kNumThreads>(variance);
834 if (tid == 0)
835     s_variance = rsqrtf(variance / (float)K + epsilon);
836 __syncthreads();
837
838 float4 reg_y;
839 reg_y.x = reg_x_hat.x * s_variance * g + b;
840 reg_y.y = reg_x_hat.y * s_variance * g + b;
841 reg_y.z = reg_x_hat.z * s_variance * g + b;
842 reg_y.w = reg_x_hat.w * s_variance * g + b;
843 if (idx < N * K)
844     FLOAT4(y[idx]) = reg_y;
845 }
846
847 // =====
848 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
849 // =====
850 // 面试要点:
851 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
852 //     [x1']   [cos(θ)  -sin(θ)] [x1]
853 //     [x2'] = [sin(θ)   cos(θ)] [x2]
854 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
855 //   - token_pos = idx / N: token 在序列中的位置
856 //   - token_idx = idx % N: token 内的维度对索引
857 //   - 输入 [seq_len, hidden_size], 输出同形状
858
859 // Grid: ((seq_len * N + 255) / 256, 1, 1)
860 // Block: (256, 1, 1)
861 // source: LeetCUDA/kernels/rope/rope.cu
862 __global__ void rope(float *x, float *out, int seq_len, int N) {
863     int idx = blockIdx.x * blockDim.x + threadIdx.x;
864     float x1 = x[idx * 2];
865     float x2 = x[idx * 2 + 1];
866     int token_pos = idx / N; // 序列位置
867     int token_idx = idx % N; // 维度对索引
868
869     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
870     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
871     float sin_v = sinf(token_pos * exp_v);
872     float cos_v = cosf(token_pos * exp_v);
873
874     // 2D 旋转
875     float out1 = x1 * cos_v - x2 * sin_v;
876     float out2 = x1 * sin_v + x2 * cos_v;
877     out[idx * 2] = out1;
878     out[idx * 2 + 1] = out2;
879 }
880
881 // =====

```



```

882 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
883 // =====
884 // 面试要点 (Bank Conflict 专题) :
885 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
886 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
887 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
888 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
889 //
890 // 转置的四步演进:
891 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
892 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
893 //   BCF: smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
894 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
895 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
896
897 // 每个线程处理 1 个元素, block(16,16)
898 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
899 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
900 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
901 // Block: (16, 16, 1)
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
904     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
905     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
906     if (r < row && c < col) {
907         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
908         y[c * row + r] = x[r * col + c];
909     }
910 }
911
912 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
913 // Block: (16, 16, 1)
914 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
915 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
916 __global__ void mat_transpose_padded(
917     float *x, float *y, const int row, const int col) {
918     const int tx = threadIdx.x;
919     const int ty = threadIdx.y;
920
921     constexpr int TILE = 16;
922     constexpr int PAD = 1;
923     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
924     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
925
926     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
927     const int x_c = blockIdx.x * TILE + tx;
928     const int x_r = blockIdx.y * TILE + ty;
929
930     if (x_r * 4 < row && x_c < col) {
931         // Step 1: 4 rows × 1 col → smem (row-major);
932 #pragma unroll
933         for (int i = 0; i < 4; ++i) {
934             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
935         }
936         __syncthreads();
937
938         // Step 2: Transposed read from smem
939         float4 reg_trans;
940         reg_trans.x = tile[tx * 4 + 0][ty];
941         reg_trans.y = tile[tx * 4 + 1][ty];
942         reg_trans.z = tile[tx * 4 + 2][ty];
943         reg_trans.w = tile[tx * 4 + 3][ty];
944

```



```

945 // y 空间坐标: y_r = x 的列, y_c = x 的行
946 const int y_r = blockIdx.x * TILE + ty; // = x col
947 const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
948
949 // Coalesced write: y[y_r][y_c .. y_c+3]
950 FLOAT4(y[y_r * row + y_c]) = reg_trans;
951 }
952 }
953
954 // =====
955 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
956 // =====
957 // 面试要点:
958 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
959 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
960 // - 不同 K 值对应不同分块策略:
961 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
962 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
963 //   K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
964 //
965 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
966
967 // ---- SGEMV K32: 基础 warp-per-row ----
968 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
969 // grid(M/4), 每个 warp 负责一行
970 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
971 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
972 // Block: (32, 4, 1), 每行 1 warp
973 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
974 // source: LeetCode/kernels/sgemv/sgemv.cu
975 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
976     int tx = threadIdx.x; // 0~31
977     int ty = threadIdx.y; // 0~3
978     int lane = tx % kWarpSize; // 0~31
979     int m = blockIdx.x * blockDim.y + ty; // 全局行号
980     if (m < M) {
981         float sum = 0.0f;
982         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
983         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
984         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
985 #pragma unroll
986         for (int w = 0; w < NUM_ITERS; ++w) {
987             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
988             int k = w * kWarpSize + lane;
989             sum += a[m * K + k] * x[k];
990         }
991         sum = warp_reduce_sum<kWarpSize>(sum);
992         // 每个 warp 处理一行, lane 0 写回结果
993         if (lane == 0)
994             y[m] = sum;
995     }
996 }
997
998 // ---- SGEMV K128: float4 向量化 ----
999 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
1000 // Grid: ((M + 3) / 4, 1, 1)
1001 // Block: (32, 4, 1)
1002 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
1003 // source: LeetCode/kernels/sgemv/sgemv.cu
1004 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
1005     int tx = threadIdx.x; // 0~31
1006     int ty = threadIdx.y; // 0~3
1007     int lane = tx % kWarpSize; // 0~31

```

```

1008 int m = blockDim.y * blockIdx.x + ty;
1009
1010 if (m < M) {
1011     float sum = 0.0f;
1012     // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1013     const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1014 #pragma unroll
1015     for (int w = 0; w < NUM_ITERS; ++w) {
1016         int k = (w * kWarpSize + lane) * 4;
1017         float4 reg_x = FLOAT4(x[k]);
1018         float4 reg_a = FLOAT4(a[m * K + k]);
1019         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1020             reg_a.w * reg_x.w);
1021     }
1022     sum = warp_reduce_sum<kWarpSize>(sum);
1023     if (lane == 0)
1024         y[m] = sum;
1025 }
1026 }
1027
1028 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1029 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1030 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1031 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1032 // Block: (32, 4, 1)
1033 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1034 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1035 template <const int kRowPerWarp = 2>
1036 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1037     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1038     int tx = threadIdx.x;
1039     int ty = threadIdx.y;
1040     int lane = tx % kWarpSize;
1041     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1042     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1043     if (m < M) {
1044         float sum = A[m * K + k] * x[k];
1045         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1046         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1047         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1048         if (k == 0)
1049             y[m] = sum;
1050     }
1051 }
1052
1053 // =====
1054 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1055 // =====
1056 // 面试要点 (GEMM 优化五层金字塔):
1057 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1058 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1059 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1060 //   load/store 指令数 Level 4 — Tensor Core (MMA
1061 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1062 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1063 //
1064 // 计算密度递进:
1065 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1066 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1067 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1068
1069 // =====
1070 // Phase 7a: SGEMM (非 Tensor Core 路径)

```

```

1071 // =====
1072
1073 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
1074 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1075 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1076 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1077 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1078 // Block: (32, 32, 1), 1024 线程
1079 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1080 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1081     constexpr int BM = 32; // vec 版: 32x4 = 128
1082     constexpr int BN = 32; // vec 版: 32x4 = 128
1083     constexpr int BK = 32;
1084     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1085
1086     int bx = blockIdx.x;
1087     int by = blockIdx.y;
1088     int tx = threadIdx.x;
1089     int tid = threadIdx.y * blockDim.x + tx;
1090
1091     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1092     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1093     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1094     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1095     // 线程不连续访问 M 维度 → 用 /;
1096     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1097     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1098     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1099     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1100     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1101     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1102
1103     float sum = 0.f; // 遍历完整的K, slice K;
1104     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1105     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1106         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1107         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1108         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1109         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1110         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1111         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1112         __syncthreads(); // 确保整个 smem tile 加载完毕
1113
1114     #pragma unroll
1115     for (int k = 0; k < BK; ++k) {
1116         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1117         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1118         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1119     }
1120     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1121 }
1122 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1123 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1124 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1125 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1126 }
1127
1128 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1129 // 在 Level 1 基础上引入两层优化:
1130 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1131 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1132 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1133 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major

```

```

1134 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1135 // 1024 × 16 = 16384 = 128 × 128 ✓
1136 //
1137 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1138 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1139 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1140 //
1141 // 加载映射 (每线程 4 个元素, float4):
1142 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1143 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1144 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1145 //
1146 // △ Bank Conflict 提示 (面试加分点):
1147 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1148 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1149 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1150 //
1151 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1152 // Block: (32, 32, 1), 1024 线程
1153 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1154 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1155 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1156     constexpr int BM = 128;
1157     constexpr int BN = 128;
1158     constexpr int BK = 32;
1159     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1160     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1161
1162     int bx = blockIdx.x;
1163     int by = blockIdx.y;
1164     int tx = threadIdx.x;
1165     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1166
1167     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1168     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1169     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1170     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1171     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1172     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1173
1174     int load_gmem_a_m = by * BM + load_smem_a_m;
1175     int load_gmem_b_n = bx * BN + load_smem_b_n;
1176
1177     // 4×4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1178     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1179     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1180     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1181     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1182     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1183
1184     float sum[4][4] = {0.f};
1185     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1186         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1187         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1188         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1189         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1190         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1191         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1192         __syncthreads();
1193     }
1194     #pragma unroll
1195     for (int k = 0; k < BK; ++k) {
1196         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4×4=16 次 FMA

```

```

1197     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1198                       s_a[comp_smem_a_m_base + 1][k],
1199                       s_a[comp_smem_a_m_base + 2][k],
1200                       s_a[comp_smem_a_m_base + 3][k]};
1201     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1202                       s_b[k][comp_smem_b_n_base + 1],
1203                       s_b[k][comp_smem_b_n_base + 2],
1204                       s_b[k][comp_smem_b_n_base + 3]};
1205 #pragma unroll
1206     for (int i = 0; i < 4; ++i) {
1207 #pragma unroll
1208         for (int j = 0; j < 4; ++j) {
1209             sum[i][j] += a_vals[i] * b_vals[j];
1210         }
1211     }
1212 }
1213 __syncthreads();
1214 }
1215
1216 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1217 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1218 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1219 #pragma unroll
1220 for (int i = 0; i < 4; ++i) {
1221     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1222     float4 reg_c;
1223     reg_c.x = sum[i][0];
1224     reg_c.y = sum[i][1];
1225     reg_c.z = sum[i][2];
1226     reg_c.w = sum[i][3];
1227     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1228 }
1229 }
1230
1231 // =====
1232 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1233 // =====
1234 // 面试要点:
1235 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1236 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1237 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1238 //     寄存器)
1239 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1240 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1241 //
1242 // ★ TN 布局详解 (面试高频考点):
1243 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1244 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1245 //   N = Normal (列优先, BLAS 原生格式)
1246 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1247 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1248 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1249 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1250 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1251 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1252 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1253 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1254 //
1255 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1256 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1257 //   N = col-major (列优先, BLAS native = normal)
1258 //
1259 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]

```

```

1260 // - A: row-major [M, K], B: row-major [K, N]
1261 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1262 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1263 //
1264 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1265 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1266 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1267 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1268 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1269 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1270 //
1271 // WGMMA (Warp Group MMA, Hopper+):
1272 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1273 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1274 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1275 // threads) 做计算
1276 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1277 //
1278 // =====
1279 // Phase 7b-1: MMA PTX 宏定义
1280 // =====
1281 //
1282 // ---- gmem → smem: cp.async ----
1283 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1284 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1285 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1286 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1287 // - wait_all: 等价于 commit_group + wait_group 0。
1288 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1289 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1290 #define CP_ASYNC_WAIT_GROUP(n) \
1291     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1292 //
1293 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1294 #define CP_ASYNC_CG(dst, src, bytes) \
1295     asm volatile( \
1296         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1297         "l"(src), "n"(bytes))
1298 //
1299 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1300 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1301 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1302 // aligned: 要求 128-bit 对齐, trans: 转置加载
1303 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1304     asm volatile( \
1305         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1306         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1307         : "r"(addr))
1308 //
1309 #define LDMATRIX_X2(R0, R1, addr) \
1310     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1311         : "=r"(R0), "=r"(R1) \
1312         : "r"(addr))
1313 //
1314 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1315 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1316 #define LDMATRIX_X2_T(R0, R1, addr) \
1317     asm volatile( \
1318         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1319         : "=r"(R0), "=r"(R1) \
1320         : "r"(addr))
1321 //
1322 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----

```



```

1323 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1324 // row.col: A 是 row-major, B 是 col-major
1325 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1326 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1327 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1328 #define MMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1329     asm volatile( \
1330         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1331         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1332         : "=r"(RD0), "=r"(RD1) \
1333         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1334         "r"(RC1)) \
1335
1336 // mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
1337 // f32 accumulator variant: 4 output registers (RD0-RD3), A/B still f16.
1338 // C matrix 16x8=128 elements, 32 threads x 4 f32 = 4 uint32 per thread.
1339 #define MMA16816F32(RD0, RD1, RD2, RD3, RA0, RA1, RA2, RA3, RB0, RB1, RC0, \
1340                     RC1, RC2, RC3) \
1341     asm volatile( \
1342         "mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 {%0, %1, %2, " \
1343         "%3}, {%4, %5, %6, %7}, {%8, %9}, {%10, %11, %12, %13};\n" \
1344         : "=r"(RD0), "=r"(RD1), "=r"(RD2), "=r"(RD3) \
1345         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1346         "r"(RC1), "r"(RC2), "r"(RC3)) \
1347
1348 // ---- MMA 辅助函数 ----
1349 // div_cceil: 整数除法向上取整
1350 #define HOST_DEVICE_INLINE __device__ __host__ inline
1351 HOST_DEVICE_INLINE int div_cceil(int a, int b) {
1352     return (a % b != 0) ? (a / b + 1) : (a / b);
1353 }
1354
1355 // =====
1356 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1357 // =====
1358 // 面试重点 — Tile Hierarchy:
1359 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1360 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1361 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1362 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1363 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1364 //
1365 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1366 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1367 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1368 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1369 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1370 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1371 //
1372 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1373 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1374 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1375 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1376 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1377 //
1378 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1379 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1380 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1381 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1382 //
1383 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1384 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1385 //

```

```

1386 //      N / bx → 0      1      2      3      ...
1387 //      M ↓
1388 //      by = 0      

|        |        |        |        |
|--------|--------|--------|--------|
| SM0    | SM1    | SM2    | SM3    |
| A 0,B0 | A 0,B1 | A 0,B2 | A 0,B3 |

      ...
1389 //
1390 //
1391 //
1392 // 同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1393 // 推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1394 // L2 更容易被其他 B tile 挤占。
1395 //
1396 // Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1397 // 下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:
1398 //
1399 //      N / local x → 0      1
1400 //      M ↓
1401 //      by = 0      

|     |     |
|-----|-----|
| SM0 | SM1 |
| SM2 | SM3 |

      → A 0; B^T 0, B^T 1
1402 //
1403 //      by = 1      

|     |     |
|-----|-----|
| SM2 | SM3 |
|-----|-----|

      → A 1; B^T 0, B^T 1
1404 //
1405 //
1406 // 对于这个 2x2 窗口: 同一行横向复用 A (SM0/SM1、SM2/SM3), 同一列纵向复用 B
1407 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2, 但缩小它会缩短再次访问相同 bx 的距离。
1408 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会, 不保证 CTA 的 z 顺序、
1409 // 缓存驻留或 L2 hit。
1410 //
1411 // Launch (kBlockSwizzle=false, 当前默认路径):
1412 //   grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1413 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid):
1414 //   tiles_n = ceil(N / BN), S = ceil(N / 2048)
1415 //   grid = (ceil(tiles_n / S), ceil(M / BM), S)
1416 //   bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1417 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1418 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1419 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1420 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1421 // Block: (256, 1, 1), 8 warps
1422 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1423 // =====
1424 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1425           const int kMmaN = 8,           // MMA atom N dim
1426           const int kMmaK = 16,          // MMA atom K dim, also BK tile = kMmaK
1427           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1428           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1429           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1430           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1431           const int kStages = 3,         // cp.async pipeline depth
1432           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1433 __global__ void __launch_bounds__(256)
1434   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1435   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1436   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1437   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1438   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1439   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1440   const int by = blockIdx.y;
1441   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1442   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1443   constexpr int BK = kMmaK; // 16
1444
1445   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1446   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1447   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1448   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外

```



```

1449 // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1450 extern __shared__ half smem[];
1451 half *s_a = smem;
1452 half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1453 constexpr int s_a_stage_offset = BM * BK; // 128*16
1454 constexpr int s_b_stage_offset = BN * BK; // 128*16  $\Delta$  BN(128)*BK(16) = B^T row-major
1455 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1456 const int warp_id = tid / kWarpSize; // 0~7
1457 const int lane_id = tid % kWarpSize; // 0~31
1458 // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1459 // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1460 //
1461 //           N direction (warp_n)
1462 //           0      1      2      3
1463 // M direction (warp_m) 0  MMA0  MMA2  MMA4  MMA6
1464 //                       1  MMA1  MMA3  MMA5  MMA7
1465 // MMA的排布方式不是唯一的, 现在这样排是因为结合MMA/VAL Tile之后, 刚好能覆盖整个
1466 // C Tile[128,128]。只要调整MMA/VAL Tile, 这里的排布方式也可以跟着调整。要注意的
1467 // 是, MMA0-7逻辑上是可以认为是并行执行的, 各自的计算结果累计加到对应的C Tile位置上。
1468 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1469 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1470
1471 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1472 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1473 int load_smem_a_m = tid / 2; // 0~127
1474 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1475 int load_smem_b_n = tid / 2; // 0~127  $\rightarrow$  B^T 的 N 方向 (row-major 的行)
1476 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8  $\rightarrow$  B^T 的 K 方向 (row-major 的列)
1477 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1478 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1479 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1480     return;
1481
1482 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1483 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1484 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1485 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1486 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1487
1488 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1489 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1490 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1491
1492 // 预加载前 (kStages-1) 个 stage
1493 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K*N])
1494 #pragma unroll
1495 for (int k = 0; k < (kStages - 1); ++k) {
1496     int load_gmem_a_k = k * BK + load_smem_a_k;
1497     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1498     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1499         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1500     );
1501     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1502
1503     int load_gmem_b_k = k * BK + load_smem_b_k;
1504     // B^T: [n][k] row-major (即 B[k][n] col-major)  $\Delta$ 
1505     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1506     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1507         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1508     );
1509     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1510
1511     CP_ASYNC_COMMIT_GROUP();
1512 }

```

```

1512 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1513 __syncthreads();
1514
1515 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1516 const int NUM_K_TILES = div_ceil(K, BK);
1517 // 此处不用pragma unroll, 因为 K 不是编译期常量, 因此NUM_K_TILES 也不是编译期常量
1518 for (int k = 0; k < NUM_K_TILES; ++k) {
1519     int smem_sel = k % kStages; // 计算 tile k 的 stage
1520     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1521
1522     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1523     // loaded了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1524     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1525     // (row-major, 内维连续的是 K)
1526     if (k + kStages - 1 < NUM_K_TILES) {
1527         int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1528         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1529         int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1530         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1531
1532         uint32_t load_smem_a_ptr =
1533             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1534                 load_smem_a_m * BK + load_smem_a_k) *
1535                 sizeof(half));
1536         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1537
1538         uint32_t load_smem_b_ptr =
1539             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1540                 load_smem_b_n * BK + load_smem_b_k) *
1541                 sizeof(half));
1542         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1543         CP_ASYNC.COMMIT_GROUP();
1544     }
1545
1546     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1547     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1548     // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1549     uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1550     uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1551
1552     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1553 #pragma unroll
1554     for (int i = 0; i < kValTileM; ++i) {
1555         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1556         // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1557         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1558         // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1559         // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1560         int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1561         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1562         uint32_t lane_smem_a_ptr =
1563             (smem_a_base_ptr +
1564                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1565                 sizeof(half));
1566         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1567     }
1568
1569     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1570     // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1571     // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1572 #pragma unroll
1573     for (int j = 0; j < kValTileN; ++j) {

```

```

1575 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1576 // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1577 // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1578 int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1579 // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1580 // ldmatrix {...}.x2 {...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1581 int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1582 int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1583 uint32_t lane_smem_b_ptr =
1584     (smem_b_base_ptr +
1585      (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1586       sizeof(half));
1587 LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1588 }
1589
1590 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1591 #pragma unroll
1592 for (int i = 0; i < kValTileM; ++i) {
1593 #pragma unroll
1594     for (int j = 0; j < kValTileN; ++j) {
1595         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1596                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1597                 RB[j][0], RB[j][1], // B fragment
1598                 RC[i][j][0], RC[i][j][1]);
1599     }
1600 }
1601
1602 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1603 if (k + kStages - 1 < NUM_K_TILES) {
1604     CP_ASYNC_WAIT_GROUP(kStages - 2);
1605 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1606     CP_ASYNC_WAIT_GROUP(0);
1607 }
1608 __syncthreads();
1609 }
1610
1611 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1612 {
1613     for (int i = 0; i < kValTileM; ++i) {
1614         // RC[kValTileM][kValTileN][2] 中 RC[...][...][2] 中保存了2个 uint32
1615         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...][...][0]
1616         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1617         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1618         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1619         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1620         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1621         // =====
1622         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1623         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1624         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1625         //
1626         //      cols: c0-1   c2-3   c4-5   c6-7
1627         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1628         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1629         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1630         //  ...
1631         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1632         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1633         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1634         //  r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1635         //  ...
1636         //  r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1637         //

```

```

1638 // Within a 4-lane group (e.g. T0-T3 for row 0):
1639 //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1640 //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1641 // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1642 // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1643 // =====
1644 #pragma unroll
1645 for (int j = 0; j < kValTileN; ++j) {
1646     RC0[j][0] = RC[i][j][0];
1647     RC1[j][0] = RC[i][j][1];
1648     RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1649     RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1650     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1651     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1652     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1653     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1654 }
1655 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1656 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):
1657 //
1658 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1659 // |-----|-----|-----|-----|
1660 // | 0~3    | 0        | row 0      | row 8      |
1661 // | 4~7    | 1        | row 1      | row 9      |
1662 // | 8~11   | 2        | row 2      | row 10     |
1663 // | 12~15  | 3        | row 3      | row 11     |
1664 // | 16~19  | 4        | row 4      | row 12     |
1665 // | 20~23  | 5        | row 5      | row 13     |
1666 // | 24~27  | 6        | row 6      | row 14     |
1667 // | 28~31  | 7        | row 7      | row 15     |
1668 //
1669 if (lane_id % 4 == 0) {
1670     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1671     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1672     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1673     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1674 #pragma unroll
1675     for (int j = 0; j < kValTileN; ++j) {
1676         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1677         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1678         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1679         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1680         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1681         // 128-bit store: 一次写入 8 个 half
1682         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1683             *reinterpret_cast<float4*>(&RC0[j][0]);
1684         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1685             *reinterpret_cast<float4*>(&RC1[j][0]);
1686     }
1687 }
1688 }
1689 }
1690 }
1691
1692 // =====
1693 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1694 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1695 // =====
1696 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除):
1697 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1698 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1699 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1700 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:

```

```

1701 //      rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1702 //      rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1703 //      rows 8~11: repeat rows 0~3 pattern
1704 //      rows 12~15: repeat rows 4~7 pattern
1705 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1706 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1707 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1708 //      对 MMA m16n8k16 的 BK=16 而言刚好满足。
1709 //
1710 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1711 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1712
1713 // ---- Swizzle 辅助函数 ----
1714
1715 // i: row index; j: col index.
1716 // Returns the chunk-level (8 fp16 = 16 B per chunk) column swizzle, i.e.
1717 // 0 / 8 / 16 / ... that the caller adds to the chunk-aligned base column.
1718 // The within-chunk offset (`j & 7`) is preserved by the caller.
1719 //
1720 // kColStride matches the equivalent CuTe ``Swizzle<B, M, S>`` swizzle
1721 // hardware mode used by TMA bulk-tensor descriptors:
1722 // * kColStride = 16 fp16 = 32 B/row -> Swizzle<1, 4, 3> = SWIZZLE_32B
1723 // * kColStride = 32 fp16 = 64 B/row -> Swizzle<2, 4, 3> = SWIZZLE_64B
1724 // * kColStride = 64 fp16 = 128 B/row -> Swizzle<3, 4, 3> = SWIZZLE_128B
1725 // The XOR mask is derived from the byte-address bit positions used by the
1726 // underlying CuTe swizzle pattern: bits {4..(4+B-1)} XOR bits
1727 // {7..(7+B-1)} of the absolute byte address. Translating to (i, j) with
1728 // row stride ``kColStride * 2`` bytes:
1729 // * 32B (B=1) : (j>>3) ^ (i>>2) [chunk in {0, 1}]
1730 // * 64B (B=2) : chunk(2b) ^ {(i>>1)&1, (i>>2)&1} [{0..3}]
1731 // * 128B (B=3) : chunk(3b) ^ {i&1, (i>>1)&1, (i>>2)&1} [{0..7}]
1732 // source: LeetCUDA/ffpa-attn/cuffpa/swizzle.cuh
1733 //
1734 // v1/v2 实现 (由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控):
1735 // swizzle_v1_impl : 原手写 XOR 分支 (permuted<kColStride,8>), 按列宽展开。
1736 // swizzle_v2_impl : 用本地 SwizzleBMS<B,M,S> 镜像 cute::Swizzle<B,M,S>::apply
1737 // 位级公式统一表达 (16/32/64); kColStride=8 回退 v1.
1738 // swizzle : 公开派发器, 宏门控, 接口与原 swizzle<kColStride> 一致。
1739 // v2 不是新算法, 而是把 v1 的手写展开重新表达为 cute 的统一位置换公式,
1740 // 便于脱离 cute 框架理解 swizzle 本质. v1/v2 bit-exact 等价。
1741 template <const int kColStride = 16, const int kStep = 8>
1742 static __host__ __device__ __forceinline__ int permuted(int i, int j) {
1743     static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
1744         kColStride == 8,
1745         "kColStride must be one of {8, 16, 32, 64} (matches "
1746         "SWIZZLE_32B/64B/128B + the "
1747         "kStep=4 narrow legacy case).");
1748     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1749     static_assert(kColStride % kStep == 0,
1750         "kColStride must be multiple of kStep.");
1751     if constexpr (kStep == 4) {
1752         static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
1753         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1754     } else if constexpr (kColStride == 16) {
1755         return (((j >> 3) ^ (i >> 2)) & 1) << 3; // SWIZZLE_32B
1756     } else if constexpr (kColStride == 32) {
1757         const int chunk = (j >> 3) & 3;
1758         const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
1759         return (chunk ^ xor_mask) << 3; // SWIZZLE_64B
1760     } else { // kColStride == 64
1761         const int chunk = (j >> 3) & 7;
1762         const int xor_mask =
1763             (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);

```

```

1764     return (chunk ^ xor_mask) << 3; // SWIZZLE_128B
1765 }
1766 }
1767
1768 // Manually SMEM swizzling for bank conflict free.
1769 // -----
1770 // [INFO] Assert smem store layout col_stride <= 16, prefer 16. |
1771 // [INFO] For logical_col_stride > 16, we have to permute the |
1772 // [INFO] smem store layout using col major ZigZag method: |
1773 // [INFO] e.g, --> Q smem logical layout [Br][64]. |
1774 // [INFO] --> col major ZigZag permuted --> |
1775 // [INFO] --> Q smem store layout [4][Br][16]. |
1776 // -----
1777 // -----
1778 // -----swizzle layout-----
1779 // -----logical col 0~64, step 8-----
1780 // -----smem col 0~16, step 8-----
1781 // -----
1782 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1783 // |row 0 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1784 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1785 // |row 1 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1786 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1787 // |row 2 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1788 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1789 // |row 3 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1790 // -----
1791 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1792 // |row 4 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1793 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1794 // |row 5 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1795 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1796 // |row 6 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1797 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1798 // |row 7 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1799 // -----
1800 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1801 // |row 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1802 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1803 // |row 9 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1804 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1805 // |row 10| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1806 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1807 // |row 11| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1808 // -----
1809 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1810 // |row 12| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1811 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1812 // |row 13| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1813 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1814 // |row 14| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1815 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1816 // |row 15| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1817 // -----
1818 template <const int kColStride = 16>
1819 static __host__ __device__ __forceinline__ int swizzle_v1_impl(int i, int j) {
1820     return permuted<kColStride, 8>(i, j);
1821 }
1822
1823 // Swizzle v2: cute::Swizzle<B,M,S> 位级镜像（脱离 cute 框架独立理解）
1824 //
1825 // v2 双重目的：
1826 // (1) 探索优化 v1 的可能 -- v1 按 kColStride 手写 XOR 分支展开，已被 nvcc

```



```

1827 //      strength-reduce 到极简位运算; v2 用 cute 的统一位置换公式重新表达,
1828 //      预期中性, 但统一表达式可能更利于合并到地址算术 (bonus).
1829 //      (2) 把 cute Swizzle 的核心位逻辑从 cute 框架提取出来独立理解 -- 让 notes-v2
1830 //      不依赖 cute 头即可展示 Swizzle<B,M,S> 的位级语义, 便于读者脱离 cute
1831 //      模板体系直接掌握 swizzle 原理.
1832 //
1833 // 注释叙述参考: 上方 hgemm_mma_stages_tn_cute 处的 Swizzle<B,M,S> 原理解释
1834 // (二维逻辑空间 + 列置换), 此处适配 v2 的 (B,M,S)=(1/2/3, 4, 3) 语境.
1835 //
1836 // Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 byte offset
1837 // 重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理
1838 // offset:
1839 // 1. 连续的 2^M 个元素组成二维空间中的一个基本元素 (元素宽度);
1840 // 2. 连续的 2^S 个基本元素组成一行 (列数);
1841 // 3. 二维空间包含 2^B 行 (行数, 这些行的 chunk 互相置换);
1842 // 4. 对二维坐标做列置换: icol' = irow ^ icol;
1843 // 5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset.
1844 //
1845 // 位级实现 (与 cutlass/include/cute/swizzle.hpp 一致):
1846 // bit 布局:  0bxxxxxxxxxxxxxxxxYYxxxxxxxxZZxxxx
1847 //              ^--^ MBase: 保留不参与置换的低位数
1848 //              ^-^   ^-^   BBits: XOR 掩码位数
1849 //              ^-----^   SShift: YYY 相对 ZZZ 的位移
1850 // apply(off) = off ^ shiftr(off & yyy_msk, S)
1851 // yyy_msk    = ((1<<B)-1) << (M + max(0,S))    // YYY 掩码位置
1852 // zzz_msk    = ((1<<B)-1) << (M - min(0,S))    // ZZZ 掩码位置 (文档用)
1853 //
1854 // 模板参数物理/语义含义:
1855 // B (BBits) : 二维逻辑空间的行数位 = 2^B 行, 即 swizzle 周期覆盖 2^B 个 chunk
1856 //             (这些 chunk 互相置换); 同时是 TMA 硬件 swizzle 模式
1857 //             SWIZZLE_{32,64,128}B 的位数 (B=1/2/3).
1858 // M (MBase) : 基本元素宽度位 = 2^M 个连续元素组成一个基本元素. 对 fp16 +
1859 //             8-element chunk, 基本元素 = 16B = 2^4, 故 M=4; 地址 bit[0..M-1]
1860 //             在 apply 中保持不变 (保留).
1861 // S (SShift) : 二维空间每行列数位 = 2^S 个基本元素组成一行; 同时是 YYY 掩码
1862 //             相对 ZZZ 掩码的位移. S>0 表示 YYY 在更高位, 向右移 S 位到 ZZZ
1863 //             位置后 XOR. 本 notes-v2 三模式 S 恒为 3 (YYY 在 bit[7..7+B-1],
1864 //             即行索引 i 的低 B 位; ZZZ 在 bit[4..4+B-1], 即列 chunk 索引).
1865 //
1866 // 完整 swizzle 周期覆盖 2^(M+S+B) 个元素 (fp16 下 2^(M+S+B+1) 字节):
1867 // (B,M,S)=(1,4,3): 256 elem / 512B (SWIZZLE_32B)
1868 // (B,M,S)=(2,4,3): 512 elem / 1024B (SWIZZLE_64B)
1869 // (B,M,S)=(3,4,3): 1024 elem / 2048B (SWIZZLE_128B)
1870 template <int B, int M, int S>
1871 struct SwizzleBMS {
1872     static_assert(M >= 0, "MBase must be non-negative.");
1873     static_assert(B > 0, "BBits must be positive.");
1874     static_assert((S > 0 ? S : -S) >= B,
1875         "abs(SShift) must be >= BBits (cute::Swizzle constraint).");
1876
1877     static constexpr int bit_msk = (1 << B) - 1;
1878     static constexpr int yyy_msk = bit_msk << (M + (S > 0 ? S : 0));
1879     static constexpr int zzz_msk = bit_msk << (M - (S < 0 ? S : 0));
1880
1881     // apply: 对 byte offset 做 XOR 位置换, ZZZ ^= (YYY >> S).
1882     static __host__ __device__ __forceinline__ int apply(int offset) {
1883         if constexpr (S >= 0) {
1884             return offset ^ ((offset & yyy_msk) >> S);
1885         } else {
1886             return offset ^ ((offset & yyy_msk) << (-S));
1887         }
1888     }
1889 };

```



```

1890
1891 // swizzle_v2_impl: 用 SwizzleBMS<B,M,S>::apply 计算 chunk 级列 swizzle.
1892 //
1893 // 流程: (i,j) -> byte offset off=(i*kColStride+j)*sizeof(half) -> SwizzleBMS
1894 //      位级置换 -> 取 [M, M+B) 位作为物理 chunk 索引 -> 乘 kStep 还原为列偏移.
1895 //
1896 // kColStride -> (B,M,S) 映射表 (对应 TMA 硬件 swizzle 模式):
1897 //   kColStride=16 fp16=32B/row -> (B,M,S)=(1,4,3) = SWIZZLE_32B
1898 //   kColStride=32 fp16=64B/row -> (B,M,S)=(2,4,3) = SWIZZLE_64B
1899 //   kColStride=64 fp16=128B/row -> (B,M,S)=(3,4,3) = SWIZZLE_128B
1900 //
1901 // kColStride=8 (kStep=4 legacy): 无对应 cute 标准 TMA swizzle 模式, 回退到
1902 //   swizzle_v1_impl. 该路径当前无任何 kernel 使用, 回退保证接口契约零变更.
1903 //
1904 // 等价性: 对 (16,32,64), v2 输出与 v1 bit-exact 相同 (见 test_swizzle_equiv).
1905 //   证明: off 的 bit[4..4+B-1] 恰是 (j>>3) 低 B 位 (因 i*kColStride 在这些位
1906 //   为 0); bit[7..7+B-1] 恰是 i 低 B 位 (因 j<kColStride 不贡献高位). apply
1907 //   把 YYY(bit[7..]) XOR 到 ZZZ(bit[4..]), 取 [M,M+B)=bit[4..4+B-1] 即得
1908 //   (j>>3)^i 的低 B 位 = v1 的 chunk^xor_mask.
1909 template <const int kColStride = 16>
1910 static __host__ __device__ __forceinline__ int swizzle_v2_impl(int i, int j) {
1911     if constexpr (kColStride == 8) {
1912         return swizzle_v1_impl<kColStride>(i, j);
1913     } else {
1914         static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64,
1915             "v2 supports kColStride in {16, 32, 64} (cute TMA swizzle); "
1916             "8 falls back to v1.");
1917         constexpr int B = (kColStride == 16) ? 1 : (kColStride == 32) ? 2 : 3;
1918         constexpr int M = 4;
1919         constexpr int S = 3;
1920         constexpr int kStep = 8;
1921         const int off = (i * kColStride + j) * (int)sizeof(half);
1922         const int sw = SwizzleBMS<B, M, S>::apply(off);
1923         return ((sw >> M) & ((1 << B) - 1)) * kStep;
1924     }
1925 }
1926
1927 // swizzle: 公开派发器, 由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控 v1/v2.
1928 //   未定义宏 (默认) -> swizzle_v1_impl (原手写 XOR 分支, 历史默认路径).
1929 //   定义宏 -> swizzle_v2_impl (cute Swizzle<B,M,S> 位级镜像).
1930 // 接口与原 swizzle<kColStride>(i,j) 完全一致, 所有 kernel 调用点无需改动.
1931 // v1/v2 等价性由 host 端 test_swizzle_equiv (--swizzle-eq-check) 验证.
1932 template <const int kColStride = 16>
1933 static __device__ __forceinline__ int swizzle(int i, int j) {
1934     #if defined(NOTES_V2_ENABLE_SWIZZLE_V2)
1935         return swizzle_v2_impl<kColStride>(i, j);
1936     #else
1937         return swizzle_v1_impl<kColStride>(i, j);
1938     #endif
1939 }
1940
1941 // =====
1942 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1943 //      + Register Double Buffering (generic kValTileK, default BK=64)
1944 // =====
1945 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1946 //   - kValTileK: 每个 BK tile 包含 kValTileK 个 kMmaK slice (BK = kMmaK * kValTileK)
1947 //   - 统一 BK tile: 所有 k_step slice 连续存放在同一个 BK 宽的 smem tile 中
1948 //   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换 (与 kValTileK 无关)
1949 //   - ldmatrix 与 MMA 计算重叠: 加载 k_step+1 的同时用另一组寄存器做 k_step 计算
1950 //   - 支持任意 kValTileK >= 2; 默认 kValTileK=4 (BK=64), kStages=2 推荐 64KB smem
1951 //
1952 // ★ smem 布局:

```

```

1953 // s_a: [stage0][stage1]...[stage(kStages-1)], 每个 stage = BM × BK = 128 × BK
1954 // s_b: 紧接 s_a 之后
1955 // 每个 stage 内 k_step slice 列偏移 = k_step * kMmaK
1956 //
1957 // ★ 寄存器双缓冲时间线 (每 k 迭代内) :
1958 // Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1959 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1960 // Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1961 // Step 4: adaptive wait + __syncthreads
1962 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1963 //
1964 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1965 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1966 // Block: (256, 1, 1), 8 warps
1967 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1968           const int kMmaN = 8,           // MMA atom N dim
1969           const int kMmaK = 16,          // MMA atom K dim
1970           const int kMmaTileM = 2,        // warps along M, 2 → warp tile M = 32
1971           const int kMmaTileN = 4,        // warps along N, 4 → warp tile N = 32
1972           const int kValTileM = 4,        // value-repeat along M, BM = 16*2*4 = 128
1973           const int kValTileN = 4,        // value-repeat along N, BN = 8*4*4 = 128
1974           const int kValTileK = 4,        // MMA_K slices per BK tile, BK = kMmaK * kValTileK
1975           const int kStages = 2,          // cp.async pipeline depth (2 for kValTileK>=4 to fit smem)
1976           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
1977 __global__ void __launch_bounds__(256)
1978   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1979   static_assert(kValTileK >= 2, "Register double buffering requires kValTileK >= 2");
1980   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1981   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1982   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1983   const int bx = ((int)kBlockSwizzle) * blockDim.z * gridDim.x + blockIdx.x;
1984   const int by = blockIdx.y;
1985   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1986   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1987   constexpr int BK = kMmaK * kValTileK;             // 16 * kValTileK
1988
1989   // kStages stages, each with BM×BK for A, BN×BK for B
1990   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK 宽 tile 中
1991   extern __shared__ half smem[];
1992   half *s_a = smem;
1993   half *s_b = smem + kStages * BM * BK;
1994   constexpr int s_a_stage_offset = BM * BK;
1995   constexpr int s_b_stage_offset = BN * BK;
1996
1997   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1998   const int warp_id = tid / kWarpSize;
1999   const int lane_id = tid % kWarpSize;
2000   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
2001   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
2002
2003   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
2004   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
2005   // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 在后续的 kValTileK 循环中
2006   // 会加上 k_step*kMmaK, 最终覆盖全部 BK 列
2007   int load_smem_a_m = tid / 2;           // 0~127
2008   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
2009   int load_smem_b_n = tid / 2;           // 0~127 → B^T 的 N 方向 (row-major 的行)
2010   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
2011   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
2012   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
2013   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
2014     return;
2015

```

```

2016 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
2017 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
2018 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
2019 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
2020 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
2021
2022 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
2023 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
2024
2025 // Prefetch (kStages-1) stages: load all k_step slices for each early stage
2026 #pragma unroll
2027 for (int k = 0; k < (kStages - 1); ++k) {
2028 #pragma unroll
2029 for (int k_step = 0; k_step < kValTileK; ++k_step) {
2030     int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
2031     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
2032     int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
2033     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
2034
2035     uint32_t load_smem_a_ptr =
2036         (smem_a_base_ptr +
2037          (k * s_a_stage_offset + load_smem_a_m * BK +
2038           k_step * kMmaK +
2039           swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
2040           sizeof(half));
2041     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
2042
2043     uint32_t load_smem_b_ptr =
2044         (smem_b_base_ptr +
2045          (k * s_b_stage_offset + load_smem_b_n * BK +
2046           k_step * kMmaK +
2047           swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
2048           sizeof(half));
2049     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
2050 }
2051 CP_ASYNC_COMMIT_GROUP();
2052 }
2053
2054 CP_ASYNC_WAIT_GROUP(kStages - 2);
2055 __syncthreads();
2056
2057 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
2058 uint32_t RA[2][kValTileM][4];
2059 uint32_t RB[2][kValTileN][2];
2060 int reg_st_idx = 0; // write target for ldmatrix
2061 int reg_ld_idx = 1; // read source for MMA
2062
2063 // Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) -> reg[0]
2064 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
2065 #pragma unroll
2066 for (int i = 0; i < kValTileM; ++i) {
2067     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2068     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2069     int lane_smem_a_k = (lane_id / 16) * 8;
2070     uint32_t lane_smem_a_ptr =
2071         (smem_a_base_ptr +
2072          (0 * s_a_stage_offset + lane_smem_a_m * BK +
2073           swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2074           sizeof(half));
2075     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2076                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2077                lane_smem_a_ptr);
2078 }

```

```

2079 #pragma unroll
2080 for (int j = 0; j < kValTileN; ++j) {
2081     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2082     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2083     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2084     uint32_t lane_smem_b_ptr =
2085         (smem_b_base_ptr +
2086          (0 * s_b_stage_offset + lane_smem_b_n * BK +
2087           swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2088            sizeof(half));
2089     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2090                lane_smem_b_ptr);
2091 }
2092
2093 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
2094 // BK = kMmaK * kValTileK, 每个 k 迭代覆盖 BK 个 K 元素
2095 const int NUM_K_TILES = div_ceil(K, BK);
2096 for (int k = 0; k < NUM_K_TILES; ++k) {
2097     int smem_sel = k % kStages;
2098     int smem_sel_next = (k + kStages - 1) % kStages;
2099
2100     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
2101     if (k + kStages - 1 < NUM_K_TILES) {
2102 #pragma unroll
2103         for (int k_step = 0; k_step < kValTileK; ++k_step) {
2104             int load_gmem_a_k =
2105                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
2106             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
2107             int load_gmem_b_k =
2108                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
2109             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
2110
2111             uint32_t load_smem_a_ptr =
2112                 (smem_a_base_ptr +
2113                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
2114                   k_step * kMmaK +
2115                  swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
2116                   sizeof(half));
2117             CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
2118
2119             uint32_t load_smem_b_ptr =
2120                 (smem_b_base_ptr +
2121                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
2122                   k_step * kMmaK +
2123                  swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
2124                   sizeof(half));
2125             CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
2126         }
2127         CP_ASYNC.COMMIT_GROUP();
2128     }
2129
2130     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
2131     // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
2132     // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
2133
2134 #pragma unroll
2135     for (int k_step = 0; k_step < kValTileK; ++k_step) {
2136         reg_st_idx ^= 1; // 0 → 1; 1 → 0
2137         reg_ld_idx ^= 1; // 1 → 0; 0 → 1
2138
2139         if (k_step + 1 < kValTileK) {
2140             int smem_k_offset = (k_step + 1) * kMmaK;
2141 #pragma unroll

```

```

2142     for (int i = 0; i < kValTileM; ++i) {
2143         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2144         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2145         int lane_smem_a_k = (lane_id / 16) * 8;
2146         uint32_t lane_smem_a_ptr =
2147             (smem_a_base_ptr +
2148              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2149               smem_k_offset +
2150               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2151              sizeof(half));
2152         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2153                     RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2154                     lane_smem_a_ptr);
2155     }
2156     #pragma unroll
2157     for (int j = 0; j < kValTileN; ++j) {
2158         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2159         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2160         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2161         uint32_t lane_smem_b_ptr =
2162             (smem_b_base_ptr +
2163              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2164               smem_k_offset +
2165               swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2166              sizeof(half));
2167         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2168                     lane_smem_b_ptr);
2169     }
2170 }
2171
2172 #pragma unroll
2173 for (int i = 0; i < kValTileM; ++i) {
2174     #pragma unroll
2175     for (int j = 0; j < kValTileN; ++j) {
2176         HMMMA16816(RC[i][j][0], RC[i][j][1],
2177                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2178                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2179                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2180                    RC[i][j][0], RC[i][j][1]);
2181     }
2182 }
2183 }
2184
2185 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2186 if (k + kStages - 1 < NUM_K_TILES) {
2187     CP_ASYNC_WAIT_GROUP(kStages - 2);
2188 } else {
2189     CP_ASYNC_WAIT_GROUP(0);
2190 }
2191 __syncthreads();
2192
2193 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2194 if (k + 1 < NUM_K_TILES) {
2195     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2196     #pragma unroll
2197     for (int i = 0; i < kValTileM; ++i) {
2198         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2199         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2200         int lane_smem_a_k = (lane_id / 16) * 8;
2201         uint32_t lane_smem_a_ptr =
2202             (smem_a_base_ptr +
2203              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2204               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *

```

```

2205         sizeof(half));
2206     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2207                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2208                lane_smem_a_ptr);
2209 }
2210 #pragma unroll
2211 for (int j = 0; j < kValTileN; ++j) {
2212     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2213     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2214     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2215     uint32_t lane_smem_b_ptr =
2216         (smem_b_base_ptr +
2217          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2218           swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2219           sizeof(half));
2220     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2221                lane_smem_b_ptr);
2222 }
2223 }
2224 }
2225
2226 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2227 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2228 for (int i = 0; i < kValTileM; ++i) {
2229 #pragma unroll
2230     for (int j = 0; j < kValTileN; ++j) {
2231         RA[0][j][0] = RC[i][j][0];
2232         RA[1][j][0] = RC[i][j][1];
2233         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2234         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2235         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2236         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2237         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2238         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2239     }
2240     if (lane_id % 4 == 0) {
2241         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2242         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2243 #pragma unroll
2244         for (int j = 0; j < kValTileN; ++j) {
2245             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2246             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2247             int store_gmem_c_addr_0 =
2248                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2249             int store_gmem_c_addr_1 =
2250                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2251             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2252                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2253             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2254                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2255         }
2256     }
2257 }
2258 }
2259
2260 // =====
2261 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2262 // =====
2263 // 面试要点 (CuTe vs 手写 MMA PTX) :
2264 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2265 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2266 //   - 与手写 MMA PTX (Phase 7b) 的对比:
2267 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle

```



```

2268 // - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2269 //      cute::copy / cute::gemm 自动展开为高效指令序列
2270 // - 核心抽象 ("CuTe 五要素") :
2271 //      1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2272 //      2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2273 //      3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2274 //      4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2275 //      5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2276 //
2277 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2278 //      MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2279 //      → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2280 //      → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2281 //      → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2282 //
2283 // 调参口诀 (面试速记) :
2284 //      BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2285 //      Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2286 //      重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2287 //      1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
2288 //      2. 连续的 2^S 个基本元素组成一行; (列数)
2289 //      3. 二维空间包含 2^B 行; (行数)
2290 //      4. 对二维坐标做列置换: icol' = irow ^ icol;
2291 //      5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2292 //      因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2293 //      CuTe 的位级实现等价于:
2294 //      offset' = offset ^ ((offset & YYY) >> S),
2295 //      YYY = ((1 << B) - 1) << (M + S)。
2296 //      对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2297 //      二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2298 //      一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2299 //      kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2300 //      EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2301 //      与逻辑 tile 组织; MMA atom越多, 需要的线程数就越多
2302 //
2303 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2304 //      - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2305 //      - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2306 //      - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2307 //
2308 // ★ 本实现的 Tile 配置:
2309 //      - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2310 //      - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2311 //      - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2312 //      BN=256 不是由 "8x2x16" 直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2313 //      - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2314 //      的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2315 //      - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2316 //      - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2317 //
2318 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2319 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2320 // =====
2321
2322 #if defined(NOTES_V2_ENABLE_CUTE)
2323
2324 #include <cute/tensor.hpp>
2325
2326 // =====
2327 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2328 // =====
2329 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
2330 //      - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [KxN])

```



```

2331 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2332 //
2333 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
2334 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2335 // 2. 主循环 over K tiles:
2336 // a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2337 // b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2338 // c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2339 // d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2340 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2341 //
2342 // 面试对比 — 手写 MMA vs CuTe 代码量:
2343 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2344 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2345 template <typename T, // element type (half)
2346          int BM, int BN, int BK, // block tile: BM=128, BN=256, BK=32
2347          int kStage, // cp.async pipeline depth (2)
2348          typename TiledMMA, // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2349          typename G2SCopyA, // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2350          typename G2SCopyB, // G→S B: same as G2SCopyA
2351          typename SmemLayoutA, // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2352          typename SmemLayoutB, // smem B: same atom → (BN,BK,kStage)
2353          typename SmemLayoutC, // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2354          typename S2RCopyAtomA, // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2355          typename S2RCopyAtomB, // S→R B: same as S2RCopyAtomA
2356          typename R2SCopyAtomC, // R→S C: UniversalCopy<int> (32-bit store)
2357          typename S2GCopyAtomC, // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2358          typename S2GCopyC, // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2359          const int BlockSwizzle = 0> // 1 enables 3D grid swizzle for L2 locality
2360 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2361          int n, int k) {
2362     using namespace cute;
2363     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2364     static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");
2365
2366     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2367     extern __shared__ T shm_data[];
2368     T *Ashm = shm_data;
2369     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2370
2371     int idx = threadIdx.x;
2372     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2373     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2374     int iy = blockIdx.y; // M 方向 block 索引
2375     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2376     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2377     if (iy * BM >= m || ix * BN >= n) return;
2378
2379     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2380     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2381     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2382                             make_stride(k, Int<1>{}));
2383     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2384                             make_stride(k, Int<1>{}));
2385     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2386                             make_stride(n, Int<1>{}));
2387
2388     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2389     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2390     // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2391     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2392     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2393     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));

```

```

2394
2395 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2396 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2397 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2398
2399 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2400 TiledMMA tiled_mma;
2401 auto thr_mma = tiled_mma.get_slice(idx);
2402 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2403 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2404 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2405 clear(tCrD); // 累加器清零
2406
2407 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2408 G2SCopyA g2s_tiled_copy_a;
2409 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2410 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2411 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2412
2413 G2SCopyB g2s_tiled_copy_b;
2414 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2415 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2416 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2417
2418 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2419 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2420 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2421 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2422 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2423 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2424
2425 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2426 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2427 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2428 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2429
2430 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2431 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2432 #pragma unroll
2433 for (int istage = 0; istage < kStage - 1; ++istage) {
2434     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2435               tAsA_copy(_, _, _, istage));
2436     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2437               tBsB_copy(_, _, _, istage));
2438     cp_async_fence();
2439     ++itile_to_read;
2440     ++ismem_write;
2441 }
2442 cp_async_wait<kStage - 2>();
2443 __syncthreads();
2444
2445 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2446 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2447 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2448 int num_k_tiles = k / BK;
2449 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2450 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2451 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2452
2453 #pragma unroll 1
2454 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2455     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2456     //

```

```

2457 // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2458 //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2459 //   - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2460 //     所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2461 //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2462 //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2463 //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2464 //     无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2465 //     for (i=0; i<kValTileM; ++i)
2466 //       for (j=0; j<kValTileN; ++j)
2467 //         HMMA16816(RC[i][j][0], RC[i][j][1], ...);
2468 //
2469 // CUTLASS 推导链 (mma_atom.hpp) :
2470 //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnk
2471 //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2472 //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2473 //   做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2474 //     K 被分为 BK/kMmaPK 个 perm-K slice
2475 //     随后按 AtomShape_Mnk<2> (MMA_K_atom) 做 zipped_divide
2476 //     每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2477 //   → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2478 // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2479 int num_k_steps = size<2>(tCrA);
2480
2481 #pragma unroll
2482 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2483   int k_step_next = (k_step + 1) % num_k_steps;
2484
2485   if (k_step == num_k_steps - 1) {
2486     // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2487     cp_async_wait<kStage - 2>();
2488     __syncthreads();
2489     ismem_read = (ismem_read + 1) % kStage;
2490   }
2491
2492   // S→R: 加载下一组 A/B fragment 到寄存器。
2493   //
2494   // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2495   //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2496   // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**:
2497   //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2498   //   让 S→R 延迟与 MMA 计算重叠。
2499   // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2500   cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2501             tCrA_view(_, _, k_step_next));
2502   cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2503             tCrB_view(_, _, k_step_next));
2504
2505   if (k_step == 0) {
2506     // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2507     if (itile_to_read < num_k_tiles) {
2508       cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2509                 tAsA_copy(_, _, _, ismem_write));
2509       cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2510                 tBsB_copy(_, _, _, ismem_write));
2511       ++itile_to_read;
2512       ismem_write = (ismem_write + 1) % kStage;
2513     }
2514     cp_async_fence();
2515   }
2516 }
2517
2518 // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2519 // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。

```

```

2520     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2521 }
2522 }
2523
2524 // Epilogue: D 寄存器 → shared memory → global memory。
2525 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2526 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2527 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2528 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2529 //   解释这同一段地址。当前 wrapper 的固定配置下:
2530 //   一个 A stage: 128 x 32 = 4096 个 half;
2531 //   C scratchpad: 32 x 32 x 4 = 4096 个 half。
2532 //   两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2533 //   是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2534 //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2535 //   检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2536 //
2537 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2538 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2539 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2540 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2541
2542 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2543 // R2SCopyAtomC = CopyAtom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2544 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2545 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2546 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2547 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2548 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2549 //
2550 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2551 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2552 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2553 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2554 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2555 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2556 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2557 // 角色, 而不是另一次 memory transfer。
2558 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2559 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2560 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2561 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2562 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2563 //   → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2564 //   → make_tiled_copy_C 的 tiler = (32, 32)
2565 //   → SmemLayoutC 的逻辑 MxN = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2566 //   → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2567 //   → 结果 shape 的 M/N remainder = _1, _1
2568 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2569 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2570 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2571 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2572
2573 // S2G TiledCopy: shared memory → global memory (128-bit store)
2574 // S2GCopyAtomC(CopyAtom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2575 S2GCopyC s2g_tiled_copy_c;
2576 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2577 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2578 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2579 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2580 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2581 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2582 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)

```

```

2583 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2584
2585 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2586 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2587 //   -> Tensor(CPY, (CPY_M, CPY_N))
2588 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2589 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2590 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说是“合并 mode”,
2591 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2592 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2593 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2594 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2595 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw
2596 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2597 //
2598 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2599 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2600 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2601 //   - 两者使用相同的 [1,3) 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2602 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2603 //
2604 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2605 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2606 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2607 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,0,j),
2608 // 再从同一个 slot 写回 global memory。
2609 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2610 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2611 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2612 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2613 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2614
2615 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2616 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2617 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2618 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2619 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2620 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2621 //
2622 //   这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2623 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2624 //   通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2625 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2626 //   唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2627 //
2628 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2629 //   各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2630 //   写入和读取命中同一段 shared memory。
2631 #pragma unroll
2632 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2633     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2634 #pragma unroll
2635     for (int j = 0; j < step; ++j) {
2636         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2637         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2638         // 成输出元素类型 T 的临时 payload。
2639         //
2640         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2641         //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2642         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2643         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2644         //
2645         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。

```



```

2646 // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2647 // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2648 // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2649 // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2650 // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2651 // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2652 // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2653 //
2654 // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2655 // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2656 // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由 R2R copy 承担, 而是
2657 // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2658 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2659 // 保持同源实现的通用 staging 写法。
2660 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2661 cute::copy(tCrC_r2sx(_, i + j), t);
2662 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2663 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2664 //
2665 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2666 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2667 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2668 //   → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2669 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2670 //   走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2671 //   → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2672 //
2673 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2674 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2675 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2676 //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2677 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2678 //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2679 //   计算了当前线程每个 val 在 smem 中的物理 offset
2680 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2681 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2682 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2683 }
2684 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2685 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2686 __syncthreads();
2687
2688 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2689 #pragma unroll
2690 for (int j = 0; j < step; ++j) {
2691 // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2692 // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2693 // (如 st.global.v4.f32 或 st.global.b128)。
2694 //
2695 // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2696 //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2697 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2698 //   → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1
2699 //   → 直接 copy_atom.call(src, dst)
2700 //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2701 //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2702 //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2703 //
2704 // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2705 //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2706 //   用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2707 //   在 sC (shared memory) 中的物理 offset。
2708 //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})

```

```

2709 //          = (32x1, 4x8) = (32, 32) — 恰好与 R2S tiler 相同,
2710 // 因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2711 // M/N remainder。
2712 // - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2713 // → slice。gD=(128,256) 相对于 tiler (32,32) 有 4x8 个 tile repetition,
2714 // 因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2715 // repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2716 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2717 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2718 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2719 }
2720 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2721 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2722 __syncthreads();
2723 }
2724
2725 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2726 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2727 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2728 // CuTe 版没有把这些 lane 映射写在 kernel 源码中: retile_S 表达 accumulator
2729 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2730 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2731 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2732 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2733 }
2734
2735 // =====
2736 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2737 // =====
2738 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2739 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2740 // kernel 接收这些类型的实例 (通常为空 struct), 在编译期完成全部映射推导。
2741 //
2742 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2743 //
2744 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2745 // 答: 手写需要:
2746 // 1) 手动计算每线程的 smem 地址偏移
2747 // 2) 手动计算 ldmatrix 的 lane 映射
2748 // 3) 手动插入 swizzle XOR 到每个地址计算点
2749 // 4) 手动做 epilogue 的 warp shuffle 编排
2750 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2751 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2752 // =====
2753 template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
2754 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2755     using namespace cute;
2756
2757     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2758     //
2759     // kernel 用法对照:
2760     // BM=128: local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2761     //          local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2762     // BN=256: local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2763     //          local_tile(D, ...) → gD 的列方向
2764     // BK=32: num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2765     // kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2766     // kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2767     auto BM = Int<128>{};
2768     auto BN = Int<256>{};
2769     auto BK = Int<32>{};
2770     auto KStage = Int<Stages>{};
2771     auto kSmemLayoutCBatch = Int<4>{};

```



```

2772 // — SmemLayoutA / SmemLayoutB —
2773 // kernel 用法:
2774 // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2775 // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2776 // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2777 //
2778 // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2779 //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2780 //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2781 //   - B=3: 共有 2^3=8 行。
2782 // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2783 // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2784 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2785 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2786 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2787 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2788 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2789 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2790 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2791 using SmemLayoutAtom = decltype(composition(
2792     Swizzle<3, 3, 3>{},
2793     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2794         make_stride(Int<BK>{}, Int<1>{}))));
2795 using SmemLayoutA = decltype(tile_to_shape(
2796     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<kStage>{})));
2797 using SmemLayoutB = decltype(tile_to_shape(
2798     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<kStage>{})));
2799
2800 // — MMA (TiledMMA) —
2801 // kernel 用法:
2802 // TiledMMA tiled_mma;
2803 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2804 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2805 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2806 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2807 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2808 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2809 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2810 //
2811 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2812 //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2813 //   → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2814 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2815 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2816 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2817 using mma_traits = MMA_Traits<mma_op>;
2818 using mma_atom = MMA_Atom<mma_traits>;
2819 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2820
2821 static constexpr int kMmaEURepeatM = 2;
2822 static constexpr int kMmaEURepeatN = 2;
2823 static constexpr int kMmaEURepeatK = 1;
2824 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2825 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2826 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2827 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2828
2829 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2830     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2831 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2832 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2833

```

```

2835 // — G2SCopyA / G2SCopyB —
2836 // kernel 用法:
2837 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2838 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2839 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2840 // G→S: 128-bit cp.async。ThrLayout{32,4} × ValLayout{1,8}:
2841 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2842 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2843 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2844 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2845 using G2SCopyA = decltype(make_tiled_copy(
2846     g2s_copy_atom{},
2847     make_layout(make_shape(Int<32>{}, Int<4>{}),
2848         make_stride(Int<4>{}, Int<1>{})),
2849     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2850 using G2SCopyB = G2SCopyA;
2851
2852 // — S2RCopyAtomA / S2RCopyAtomB —
2853 // kernel 用法:
2854 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2855 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2856 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2857 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2858 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2859 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout。
2860 using s2r_copy_op = SM75_U32x4_LDSM_N;
2861 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2862 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2863 using S2RCopyAtomA = s2r_copy_atom;
2864 using S2RCopyAtomB = s2r_copy_atom;
2865
2866 // — SmemLayoutC —
2867 // kernel 用法:
2868 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2869 // 复用当前 A stage 的空间作为 C scratchpad
2870 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2871 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2872 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2873 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等于主循环
2874 // 的 kStage K-tile pipeline。
2875 using SmemLayoutAtomC = decltype(composition(
2876     Swizzle<3, 3, 3>{},
2877     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2878         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2879 using SmemLayoutC = decltype(tile_to_shape(
2880     SmemLayoutAtomC{},
2881     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2882
2883 // — R2SCopyAtomC —
2884 // kernel 用法:
2885 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2886 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2887 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2888 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2889 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2890 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2891
2892 // — S2GCopyC —
2893 // kernel 用法:
2894 // S2GCopyC s2g_tiled_copy_c;
2895 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2896 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
2897 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),

```

```

2898 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2899 using S2GCopyAtomC = CopyAtom<UniversalCopy<cute::uint128_t>, T>;
2900 using S2GCopyC = decltype(make_tiled_copy(
2901     S2GCopyAtomC{}),
2902     make_layout(make_shape(Int<32>{}, Int<4>{}),
2903         make_stride(Int<4>{}, Int<1>{}))),
2904     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2905
2906 // — Grid/Block 配置 —
2907 // kernel 用法:
2908 // int idx = threadIdx.x;          // 0..127
2909 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2910 // int iy = blockIdx.y;           // M 方向 CTA 坐标
2911 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2912 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2913 // M % BM == 0, N % BN == 0, K % BK == 0。
2914 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2915 constexpr int kSwizzleStride = 2048;
2916 int BX = (N + BN - 1) / BN;
2917 int BY = (M + BM - 1) / BM;
2918 int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2919 BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2920 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2921 dim3 grid(BX, BY, BZ);
2922
2923 // — Dynamic Shared Memory 大小 —
2924 // kernel 用法:
2925 // extern __shared__ T shm_data[];
2926 // T *Ashm = shm_data;
2927 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2928 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2929 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2930 // 合计 A+B = 24576 个 T; C epilogue 复用一个 A stage (128*32 = 4096 个 T)。
2931 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2932 // 能放进一个 A stage (128*32 = 4096)。
2933 static constexpr int shm_size_AB =
2934     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2935 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2936 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2937     "C shared memory must fit within one A pipe");
2938 static constexpr int kShmSize =
2939     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2940
2941 cudaFuncSetAttribute(
2942     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2943         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2944         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2945         S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
2946     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2947
2948 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2949     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2950     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2951     BlockSwizzle>
2952     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2953 }
2954
2955 #endif /* NOTES_V2_ENABLE_CUTE */
2956
2957 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2958 // TMA/mbarrier helpers.
2959 //
2960 // Two implementations gated by NOTES_V2_FORCE_INLINE_ASYNC_PROXY:

```

```

2961 // - Defined (default): raw `asm volatile` PTX. ptxas keeps setmaxnreg
2962 // because raw asm carries no source annotation that ptxas would treat as
2963 // an "extern call" boundary (warning C7506 otherwise drops the hint).
2964 // - Undef'd: cuda::ptx:: / cuda::device:: C++ wrappers. Cleaner API but
2965 // ptxas drops setmaxnreg with C7506 even though the wrappers inline to
2966 // identical PTX; useful for comparing behavior / debugging.
2967 //
2968 // cuda::barrier (init/arrive/wait) is shared by both paths: it inlines to raw
2969 // mbarrier PTX and does not trigger C7506, so it needs no gating.
2970 //
2971 // Mirrors the raw-PTX style of tmp/LeetGPU/CUDA/22_GEMM/sm90_wgmma_tma_ws_pingpong.cu.
2972 #if defined(NOTES_V2_FORCE_INLINE_ASYNC_PROXY)
2973
2974 static __device__ __forceinline__ uint32_t
2975 cast_smem_ptr_to_uint(void const *ptr) {
2976     return static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2977 }
2978
2979 static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2980     asm volatile("fence.proxy.async.shared::cta;\n" ::: "memory");
2981 }
2982
2983 // cp.async.bulk.tensor.2d: 2D TMA copy from global to shared::cluster.
2984 // [dst] = shared memory destination (smem addr, 32-bit)
2985 // [desc, {c0, c1}] = CUTensorMap + (minor_coord, major_coord) coords
2986 // [mbar] = mbarrier in shared memory (smem addr, 32-bit)
2987 // mbarrier::complete_tx::bytes: barrier flips phase when TMA bytes land.
2988 // L2::cache_hint omitted (EVICT_NORMAL); matches the cuda::ptx default.
2989 static __device__ __forceinline__ void tma_load_2d(
2990     void *dst, const CUTensorMap *tensor_map, int minor_coord, int major_coord,
2991     cuda::barrier<cuda::thread_scope_block> &barrier) {
2992     uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(tensor_map);
2993     uint32_t smem_int_ptr = cast_smem_ptr_to_uint(dst);
2994     uint32_t smem_int_mbar =
2995         cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
2996     asm volatile(
2997         "cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes"
2998         " [%0], [%1, {%3, %4}], [%2];"
2999         :
3000         : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
3001           "r"(minor_coord), "r"(major_coord)
3002         : "memory");
3003 }
3004
3005 // mbarrier.arrive.expect_tx: register one arrival on the mbarrier and declare
3006 // that `bytes` of async copy traffic is expected before the phase flips.
3007 // Equivalent to cuda::ptx::mbarrier_arrive_expect_tx(release, cta, shared).
3008 static __device__ __forceinline__ void tma_arrive_expect_tx(
3009     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
3010     uint32_t smem_int_mbar =
3011         cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
3012     asm volatile(
3013         "mbarrier.arrive.expect_tx.shared::cta.b64 __, [%0], %1;\n"
3014         :
3015         : "r"(smem_int_mbar), "r"(bytes)
3016         : "memory");
3017 }
3018
3019 #else // !NOTES_V2_FORCE_INLINE_ASYNC_PROXY: use cuda::ptx / cuda::device wrappers
3020
3021 static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
3022     #if CUDART_VERSION >= 13020
3023         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);

```

```

3024 #else
3025     cuda::device::experimental::fence_proxy_async_shared_cta();
3026 #endif
3027 }
3028
3029 static __device__ __forceinline__ void tma_load_2d(
3030     void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
3031     cuda::barrier<cuda::thread_scope_block> &barrier) {
3032     #if CUDART_VERSION >= 13020
3033         const int32_t coords[] {minor_coord, major_coord};
3034         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
3035         cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
3036                                         cuda::ptx::space_global, dst, tensor_map,
3037                                         coords, barrier_handle);
3038     #else
3039         cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3040             dst, tensor_map, minor_coord, major_coord, barrier);
3041     #endif
3042 }
3043
3044 static __device__ __forceinline__ void tma_arrive_expect_tx(
3045     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
3046     #if CUDART_VERSION >= 13020
3047         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
3048         [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
3049             cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
3050             barrier_handle, bytes);
3051     #else
3052         [[maybe_unused]] auto token =
3053             cuda::device::barrier_arrive_tx(barrier, 1, bytes);
3054     #endif
3055 }
3056
3057 #endif // NOTES_V2_FORCE_INLINE_ASYNC_PROXY
3058
3059 // Warpgroup-level register rebalancing for warp specialization.
3060 // setmaxnreg.dec releases registers (producer TMA path needs few),
3061 // setmaxnreg.inc requests registers (consumer MMA path needs many).
3062 // Both require all warps in a warpgroup to execute the same instruction,
3063 // and require __launch_bounds__(N, 1) so the compiler permits up to 256
3064 // regs/warp (otherwise the hint may have no effect). Supported on sm_90a/sm_120a.
3065 // __forceinline__ is mandatory: without it ptxas drops setmaxnreg with
3066 // warning C7506 "ignored to maintain compatibility into 'extern' call",
3067 // because a non-inlined call boundary forces a fixed register convention.
3068 //
3069 // On sm_120a (Blackwell, CUDA 13.2) ptxas drops setmaxnreg with C7506 even
3070 // when the PTX is fully inlined (no call.uni), because ptxas treats
3071 // cp.async.bulk.tensor (TMA) usage as an implicit extern-call boundary.
3072 // sm_90a (Hopper) is unaffected. Gate the *call sites* with
3073 // NOTES_V2_ENABLE_SETMAXNREGS so sm_120a builds stay warning-free and avoid
3074 // the register-allocation side effects of __launch_bounds__(N,1) until ptxas
3075 // is fixed. The function templates themselves are always defined.
3076 #if defined(NOTES_V2_ENABLE_SETMAXNREGS)
3077     #define NOTES_V2_REG_DEALLOC(N) warpgroup_reg_dealloc<N>()
3078     #define NOTES_V2_REG_ALLOC(N) warpgroup_reg_alloc<N>()
3079 #else
3080     #define NOTES_V2_REG_DEALLOC(N) ((void)0)
3081     #define NOTES_V2_REG_ALLOC(N) ((void)0)
3082 #endif
3083
3084 template <uint32_t kNumRegs>
3085 __device__ __forceinline__ void warpgroup_reg_dealloc()
3086 {

```

```

3087     asm volatile("setmaxnreg.dec.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
3088 }
3089
3090 template <uint32_t kNumRegs>
3091 __device__ __forceinline__ void warpgroup_reg_alloc()
3092 {
3093     asm volatile("setmaxnreg.inc.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
3094 }
3095 #endif
3096
3097 #if defined(NOTES_V2_ENABLE_WGMMMA)
3098 // =====
3099 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
3100 // =====
3101 // 面试要点 (WGMMMA vs MMA 对比) :
3102 //   - MMA: warp 级 (32 threads) , 同步执行
3103 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
3104 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
3105 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
3106 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
3107 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
3108
3109 // ---- WGMMMA 辅助函数 ----
3110 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
3111 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
3112 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
3113 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
3114 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
3115 //     都是 "wait until only N or fewer groups are pending"。
3116 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
3117 #define WGMMMA_COMMIT_GROUP() \
3118     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory") \
3119 #define WGMMMA_WAIT_GROUP(n) \
3120     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
3121
3122 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
3123 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
3124 //   encoded = (x & 0x3FFFF) >> 4
3125 // 其中 0x3FFFF = 218 - 1 = 262143, 只保留低 18 bit。
3126 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (24) , 低 4 bit 恒为 0,
3127 // 可省略以扩大可表示范围。
3128 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
3129 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
3130
3131 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
3132 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
3133 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
3134 //
3135 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
3136 // -----
3137 // | 位域          | 范围      | 大小 | 含义 |
3138 // |-----|-----|-----|-----|
3139 // | start_address  | [0, 14)   | 14   | smem 基址编码 |
3140 // | (unused)       | [14, 16)  | 2    | - |
3141 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1) |
3142 // | (unused)       | [30, 32)  | 2    | - |
3143 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距 |
3144 // | (unused)       | [46, 49)  | 3    | - |
3145 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移 |
3146 // | (unused)       | [52, 62)  | 10   | - |
3147 // | layout_type    | [62, 64)  | 2    | swizzle 模式 |
3148 // -----
3149 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle

```



```

3150 //
3151 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
3152 // - 128B swizzle atom 在 128-bit 单位下为 8×8
3153 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
3154 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
3155 // - 这是 stride_byte_offset=1024 的来源
3156 //
3157 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
3158 // 此处填 16 仅为占位, 编码后为 1。
3159 //
3160 // - base_offset=0 (bits [49,52] 未显式置位) , 表示 smem ptr 已 1024B 对齐。
3161 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
3162 //
3163 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
3164 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
3165 __device__ inline uint64_t make_smem_desc(half *ptr) {
3166     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
3167     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
3168     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
3169
3170     // 从全零开始: 所有位域初始化为 0。
3171     uint64_t desc = 0x0000000000000000;
3172
3173     // — bits [0, 14): start_address —
3174     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
3175     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
3176     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
3177     desc |= SMEM_DESC_ENCODE(addr);
3178
3179     // — bits [16, 30): leading_byte_offset —
3180     // 原始值 16 (字节), 编码后为 (16 & 0xFFFF) >> 4 = 1。
3181     // << 16 将编码值放到 bits [16, 30)。
3182     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
3183     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
3184     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
3185     // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
3186     // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
3187     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
3188
3189     // — bits [32, 46): stride_byte_offset —
3190     // 原始值 1024 (字节), 编码后为 (1024 & 0xFFFF) >> 4 = 64。
3191     // << 32 将编码值放到 bits [32, 46)。
3192     // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
3193     // 1024 的来源 (K-Major + 128B swizzle + half) :
3194     // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
3195     // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
3196     // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组) 。
3197     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
3198
3199     // — bits [62, 64): layout_type (swizzle mode) —
3200     // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
3201     // 1 = 128B swizzle mode。
3202     // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
3203     // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
3204     // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
3205     desc |= 1llu << 62;
3206
3207     return desc;
3208 }
3209
3210 // ---- WGMMMA PTX 指令宏 ----
3211 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
3212 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算 D[64×128] += A[64×16] × B[16×128]。

```

```

3213 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
3214 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
3215 // 每线程 64 half = 32 uint32。
3216 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
3217 // Scaled: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
3218 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
3219 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
3220 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
3221 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, Scaled, ScaleA, ScaleB, TransA, \
3222                                     TransB) \
3223 { \
3224     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
3225     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
3226     asm volatile( \
3227         "{\n" \
3228         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
3229         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
3230         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
3231         " %16, %17, %18, %19, %20, %21, %22, %23, " \
3232         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
3233         " %32, " \
3234         " %33, " \
3235         " %34, %35, %36, %37, %38;\n" \
3236         "}\n" \
3237         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
3238         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
3239         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
3240         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
3241         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
3242         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
3243         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
3244         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
3245         : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
3246         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
3247         "n"(int32_t(TransB))); \
3248 } \
3249
3250 // ---- TMA Shared Memory Layout ----
3251 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
3252 //
3253 // 每个 stage 包含两块 smem:
3254 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
3255 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
3256 //
3257 // ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
3258 //
3259 //   上层数据: 源矩阵 BT [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
3260 //   物理存储: TMA 将 BT 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
3261 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
3262 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
3263 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
3264 //   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
3265 //               通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
3266 //               实际 [BN, BK] → 逻辑 [BK, BN] 的“转置”是通过 descB 中的 128B
3267 //               swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
3268 //
3269 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
3270 //       128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
3271 //       stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
3272 //                           = 8 rows × (BK=64) halves × 2 bytes = 1024。
3273 //       这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
3274 //
3275 //   与 MMA(TN) 的对比:

```

```

3276 // MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
3277 // WGMMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
3278 // 简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3279 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3280     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3281     // B tile 标记:
3282     // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3283     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3284     //   以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3285     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
3286     //   但写 BN×BK 更能体现物理存储形态
3287     alignas(128) half B[BN * BK * QSIZE];
3288 };
3289
3290 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
3291 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
3292 //
3293 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
3294 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
3295 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3296 //
3297 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3298 //   将 A/B tile 从 HBM 搬到 shared memory。
3299 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
3300 //
3301 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3302 // - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用
3303 // - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3304 //
3305 // 本节重点理解:
3306 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3307 //   即可搬运整个 2D tile, 无需所有线程参与。
3308 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3309 // 3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3310 //   Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
3311 //
3312 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
3313 // WGMMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3314 // K Tile: BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMMA atom
3315 // M Tile: BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMMA atom
3316 // N Tile: BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
3317 // Block Tile: C[128,128] = A[128,64] × B[64,128]
3318 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
3319 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
3320 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
3321 //
3322 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
3323 // 的延迟被计算完全掩盖。
3324 //
3325 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
3326 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3327 // Block: (256, 1, 1), 2 warpgroups
3328 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3329 template <const int kWgmmaM = 64, // WGMMMA atom M dim (m64n128k16)
3330           const int kWgmmaN = 128, // WGMMMA atom N dim
3331           const int kWgmmaK = 16, // WGMMMA atom K dim
3332           const int BM = 128, // block tile M, 2 × kWgmmaM
3333           const int BN = 128, // block tile N, 1 × kWgmmaN
3334           const int BK = 64, // block tile K, 4 × kWgmmaK
3335           const int kNumThreads = 256, // 2 warpgroups × 128 threads
3336           const int kStages = 3, // TMA pipeline depth (full/empty barriers)
3337           const int kBlockSwizzle = 0> // 1 enables 3D grid swizzle for L2 locality
3338 __global__ void __launch_bounds__(kNumThreads, 1)

```

```

3339     hgemm_wgmma_stages_tn(
3340         int M, int N, int K, half *C,
3341         const CUtensorMap *__restrict__ tensorMapA,
3342         const CUtensorMap *__restrict__ tensorMapB) {
3343     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3344     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3345     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3346     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3347     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3348     // 不展开宿主侧 create_tensor_map 细节。
3349
3350     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3351     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3352     const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3353     const int by = blockIdx.y;
3354     constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3355     // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3356     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3357     constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3358     // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3359     // 当只有一个 consumer 时, 它负责全部 BM=128 行
3360     constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3361
3362     // 边界检查: 确保当前 block 不超出 M/N 范围
3363     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3364         return;
3365
3366     // ---- Shared Memory 分配 ----
3367     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3368     // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3369     // make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3370     // 完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3371     // 硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3372     // descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3373     // 1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3374     // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle<64>()
3375     // , 该函数无法感知 smem 绝对地址, 硬编码了 phase=0 的假设, 因此必须
3376     // __align__(1024) 来保证 phase 确实为 0。从健壮性角度看本 kernel 也
3377     // 应该用 __align__(1024); 这里使用 128 也可以正常运行。
3378     extern __shared__ __align__(1024) uint8_t smem_tma_wgmma_ws[];
3379     WgmmaSMem<BM, BN, BK, kStages> &s =
3380         *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages>*>(smem_tma_wgmma_ws);
3381     half *s_a = s.A;
3382     half *s_b = s.B;
3383
3384     // ---- cuda::barrier 初始化 ----
3385     //
3386     // ★ Barrier 机制速查 (arrive / wait 核心语义):
3387     //
3388     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3389     //
3390     // init(bar, N): 设置 arrive_count = N。
3391     // 含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3392     //
3393     // arrive(): 线程声明"我已到达此 barrier 点"。
3394     // - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3395     // - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3396     //
3397     // wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3398     // - 即: 等待 phase 翻转。
3399     // - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3400     // (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3401     //

```

```

3402 // 典型用法: b.wait(b.arrive())
3403 // - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3404 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3405 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3406 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3407 //
3408 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129) :
3409 //
3410 // Producer (1 thread)          Consumer (128 threads)
3411 // -----
3412 //                                C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3413 // ┌ for each k_tile: ─┐          ┌ for each k_tile: ─┐
3414 // │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
3415 // │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪   │
3416 // │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算       │
3417 // │   ↓ 异步拷贝             │ │ C3: wait WGMMMA 完成   │
3418 // │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q])  │
3419 // │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
3420 // └──────────────────┘          └──────────────────┘
3421 //
3422 // 关键不变式 (invariant) :
3423 // - Producer 不会覆盖 Consumer 正在读的 stage
3424 // - Consumer 不会读 Producer 还没写完的 stage
3425 // - 通过 full/empty 两个 barrier 的 phase 交替来保证
3426 //
3427 // 每个 stage 有两个 barrier:
3428 //   full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3429 //   empty[stage]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3430 //
3431 // arrive_count = 128 (consumer) + 1 (producer) = 129:
3432 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3433 #pragma nv_diag_suppress static_var_with_dynamic_init
3434 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3435 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3436 #pragma nv_diag_default static_var_with_dynamic_init
3437
3438 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3439 const int NUM_K_TILES = div_ceil(K, BK);
3440 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3441 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3442
3443 // 初始化 barriers (仅 thread 0 执行)
3444 if (threadIdx.x == 0) {
3445     for (int i = 0; i < kStages; ++i) {
3446         // arrive_count = 129: 128 consumer + 1 producer
3447         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3448         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3449         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3450         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer
3451         init(&empty[i], kConsumerThreads + 1); // same
3452     }
3453     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3454     // 对 async proxy (TMA/WGMMMA) 可见。
3455     tma_fence_proxy_async_shared_cta();
3456 }
3457 __syncthreads();
3458
3459 // =====
3460 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3461 // =====
3462 //
3463 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3464 // - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。

```



```

3465 // - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3466 //
3467 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3468 // Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3469 // Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3470 //
3471 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3472 // - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3473 // - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3474 //
3475 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3476 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3477 // =====
3478 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3479 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
3480 // 只有 wg_tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3481 // =====
3482 if (wg_idx == 0) {
3483     NOTES_V2_REG_DEALLOC(40);
3484     if (wg_tid == 0) {
3485         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3486         int stage = 0;
3487         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3488             if (stage == kStages)
3489                 stage = 0;
3490
3491             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
3492             //
3493             // 模式 empty[stage].wait(empty[stage].arrive()):
3494             // - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3495             //   获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3496             //   128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
3497             //   → phase 立即翻转。
3498             // - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3499             //   由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
3500             //   wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3501             //
3502             // 语义: Producer 说"我准备好写 stage stage 了, Consumer 用完没有?"
3503             //   如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3504             //   如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3505             empty[stage].wait(empty[stage].arrive());
3506
3507             // Step P2: 提交 TMA 2D 拷贝指令
3508             // cp_async_bulk_tensor_2d_global_to_shared:
3509             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3510             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3511             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3512             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3513             //
3514             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
3515             // 无需线程逐元素搬运, 零寄存器开销。
3516             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3517             tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3518                 full[stage]);
3519
3520             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3521             tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3522                 full[stage]);
3523
3524             // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
3525             //
3526             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3527             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)

```



```

3528 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3529 // - Phase 翻转条件:
3530 //     (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3531 //     (b) 所有声明的 async 字节已写入 smem
3532 // 两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3533 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3534 tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3535 }
3536 }
3537 }
3538 // =====
3539 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3540 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3541 // 所有 128 个线程 (4 warps) 全部参与。
3542 // =====
3543 else {
3544 // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3545 //
3546 // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3547 // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3548 // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3549 //
3550 // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3551 // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3552 NOTES_V2_REG_ALLOC(232);
3553
3554 for (int i = 0; i < kStages; ++i) {
3555     [[maybe_unused]] auto token = empty[i].arrive();
3556 }
3557
3558 // 累加器寄存器声明
3559 // d[kWarpgroupM / kWmmaM][kWmmaN / 16][4]:
3560 // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3561 // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3562 // - d[*][g][*]: N 方向第 g 组 16 列
3563 // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3564 // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3565 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3566 uint32_t d[kWarpgroupM / kWmmaM][kWmmaN / 16][4] = {};
3567
3568 int stage = 0;
3569 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3570 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3571     if (stage == kStages)
3572         stage = 0;
3573
3574     // Step C1: 等待 TMA 数据就绪 (full 信号)
3575     //
3576     // 模式 full[stage].wait(full[stage].arrive()):
3577     // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3578     // - 当前 phase 累积: 128/129, phase 尚未翻转。
3579     // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3580     //   贡献第 129 次到达 + TMA 字节全部写完 -> phase 翻转 -> wait 返回。
3581     //
3582     // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有?"
3583     full[stage].wait(full[stage].arrive());
3584
3585     // Step C2: 发射 WGMMMA 指令序列
3586     //
3587     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3588     // 标准流程: FENCE -> 发射 WGMMMA -> COMMIT -> WAIT。
3589     //
3590     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :

```

```

3591 // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3592 // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3593 // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3594 //
3595 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3596 // D[64×128] += A[64×16] × B[16×128]
3597 // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3598 // ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都用 ScaleD=1。
3599 WGMMMA_FENCE();
3600
3601 // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3602 // 每个 atom 处理 64 行 M 方向数据。
3603 #pragma unroll
3604 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3605     // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3606     // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3607     half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3608
3609     // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3610     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3611     #pragma unroll
3612     for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
3613         // 第 k_step 次 WGMMMA:
3614         // A: wgmma_sA + k_step * kWgmmaK (A 的 K 维起始位置)
3615         // B: s_b + stage * BK * BN + k_step * kWgmmaK (B 的 K 维起始位置)
3616         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3617         // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3618         // ScaleD=1: 累加 (K 维迭代需要累积结果)
3619         WGMMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
3620                                     s_b + stage * BK * BN + k_step * kWgmmaK,
3621                                     1, // ScaleD=1: accumulate (不清零)
3622                                     1, 1, 0, 0);
3623     }
3624 }
3625
3626 // Step C3: 提交并等待 WGMMMA 完成
3627 //
3628 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3629 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3630 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3631 //
3632 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3633 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3634 // ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3635 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3636 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3637 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3638 WGMMMA_COMMIT_GROUP();
3639 WGMMMA_WAIT_GROUP(0);
3640
3641 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3642 //
3643 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3644 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3645 //
3646 // 语义: Consumer 说"stage stage 我已经读完了, 你可以放心覆盖了。"
3647 [[maybe_unused]] auto token = empty[stage].arrive();
3648 }
3649
3650 // =====
3651 // Epilogue: 将寄存器中的累加结果写回 global memory C
3652 // =====
3653 //

```

```

3654 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3655 //
3656 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3657 // 这些 half 分布在 d[2][8][4] 数组中:
3658 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3659 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3660 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3661 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3662 //
3663 // 其中:
3664 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3665 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3666 //
3667 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3668 // +-----+-----+
3669 // | reg[0] | reg[2] | <- rows [row, row+8)
3670 // |(col,+2)|(col+8)|
3671 // +-----+-----+
3672 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3673 // |(col,+2)|(col+8)|
3674 // +-----+-----+
3675 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3676 //
3677 // 三维遍历:
3678 // m_it=0: rows 0~63, m_it=1: rows 64~127
3679 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3680 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3681 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3682
3683 const int lane = wg_tid % 32;
3684 const int warp = wg_tid / 32;
3685 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3686 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3687 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3688 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3689 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3690 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3691 const int row = warp * 16 + lane / 4;
3692 // block_C: 当前 C tile 在 global memory 中的起始地址
3693 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3694 half *block_C = C + by * BM * N + bx * BN;
3695
3696 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3697 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3698 #pragma unroll
3699 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3700     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3701     #pragma unroll
3702     for (int g = 0; g < kWgmmaN / 16; ++g) {
3703         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3704         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3705         // 左上象限: (row, col)
3706         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3707         // 左下象限: (row+8, col)
3708         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3709         // 右上象限: (row, col+8)
3710         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3711         // 右下象限: (row+8, col+8)
3712         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3713     }
3714 }
3715 }
3716 }

```

```

3717 #endif /* NOTES_V2_ENABLE_WGMMA */
3718
3719
3720 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3721 // SM120 不支持 WGMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3722 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3723 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3724     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3725     half A[BM * BK * QSIZE];
3726     half B[BN * BK * QSIZE];
3727 };
3728
3729 // 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
3730 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3731 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
3732           const int kMmaN = 8,           // MMA atom N dim
3733           const int kMmaK = 16,          // MMA atom K dim
3734           const int kMmaTileM = 2,       // consumer warps along M, warp tile M = 32
3735           const int kMmaTileN = 2,       // consumer warps along N, warp tile N = 16
3736           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
3737           const int kValTileN = 8,       // value-repeat along N, BN = 8*2*8 = 128
3738           const int kValTileK = 4,       // MMA_K slices per BK tile, BK = 16*4 = 64
3739           const int kStages = 2,         // TMA full/empty pipeline depth
3740           const int kNumThreads = 256,   // 128 producer + 128 consumer threads
3741           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
3742 __global__ void __launch_bounds__(kNumThreads)
3743     hgemm_tma_mma_ws_tn(
3744         int M, int N, int K, half *C,
3745         const CUtensorMap *__restrict__ tensorMapA,
3746         const CUtensorMap *__restrict__ tensorMapB) {
3747
3748     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3749     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3750     constexpr int BK = kMmaK * kValTileK;
3751     static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3752     static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3753     static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3754     static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3755     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3756
3757     const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3758     const int by = blockIdx.y;
3759     constexpr int kConsumerThreads = kNumThreads / 2;
3760     static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3761         "Consumer threads must cover the MMA warp grid");
3762
3763     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3764         return;
3765
3766     // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3767     // 以确保硬件 swizzle phase 从零开始。消费者 swizzle<64>()
3768     // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的
3769     // 物理地址, 破坏整个输出。
3770     //
3771     // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128), 而本 kernel
3772     // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3773     //
3774     // WGMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMA descriptor,
3775     // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3776     // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会
3777     // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMA 路径不
3778     // 需要 1024 字节对齐也能得到正确数据。
3779     //

```

```

3780 // 本 kernel 消费者：使用 ldmatrix + 手写 swizzle<64>() 来计算
3781 // smem 物理地址。这个函数是纯软件公式，没有任何 base_offset
3782 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3783 // 如果基址只满足 128B 对齐，TMA 硬件会以非零 phase 写入数据，
3784 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3785 //
3786 // 简言之：WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移；
3787 // ldmatrix 路径没有等价机制，必须靠 1024B 对齐来保证 phase=0。
3788 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3789 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3790     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3791 half *s_a = s.A;
3792 half *s_b = s.B;
3793
3794 #pragma nv_diag_suppress static_var_with_dynamic_init
3795 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3796 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3797 #pragma nv_diag_default static_var_with_dynamic_init
3798
3799 const int NUM_K_TILES = div_ceil(K, BK);
3800 const int wg_idx = threadIdx.x / kConsumerThreads;
3801 const int wg_tid = threadIdx.x % kConsumerThreads;
3802
3803 if (threadIdx.x == 0) {
3804     for (int stage = 0; stage < kStages; ++stage) {
3805         init(&full[stage], kConsumerThreads + 1);
3806         init(&empty[stage], kConsumerThreads + 1);
3807     }
3808     tma_fence_proxy_async_shared_cta();
3809 }
3810 __syncthreads();
3811
3812 if (wg_idx == 0) {
3813     // 生产者 Warpgroup (wg_idx==0)
3814     NOTES_V2_REG_DEALLOC(40);
3815     if (wg_tid == 0) {
3816         int stage = 0;
3817         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3818             if (stage == kStages)
3819                 stage = 0;
3820
3821             empty[stage].wait(empty[stage].arrive());
3822
3823             tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3824                 full[stage]);
3825             tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3826                 full[stage]);
3827             tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3828         }
3829     }
3830 } else {
3831     // 消费者 Warpgroup (wg_idx==1)
3832     // 初始化：标记所有 stage 为"空" (可被 Producer 写入)
3833     NOTES_V2_REG_ALLOC(232);
3834
3835     for (int stage = 0; stage < kStages; ++stage) {
3836         [[maybe_unused]] auto token = empty[stage].arrive();
3837     }
3838
3839     const int warp_id = wg_tid / kWarpSize;
3840     const int lane_id = wg_tid % kWarpSize;
3841     // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3842     const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}

```

```

3843 const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
3844 uint32_t RA[kValTileM][4];
3845 uint32_t RB[kValTileN][2];
3846 uint32_t RC[kValTileM][kValTileN][2] = {0};
3847
3848 const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3849 const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3850 int stage = 0;
3851 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3852     if (stage == kStages)
3853         stage = 0;
3854
3855     full[stage].wait(full[stage].arrive());
3856     tma_fence_proxy_async_shared_cta();
3857
3858     // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3859     // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3860     // 同步机制与 warp specialization 的线程分工:
3861     //
3862     // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3863     //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3864     //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3865     //    拷贝都完成后再进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3866     //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3867     //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3868     //
3869     // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3870     //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3871     //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3872     //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3873     //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3874     //
3875     // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3876     //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的
3877     //    行区间)。它们之间不需要读写共享数据, 因此不需要 __syncthreads 来
3878     //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令, 保证同一
3879     //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3880
3881     // 注意: 这里已经没有NUM_K_TILES循环了, 因为K loop load已经被WG0生产者处理了
3882 #pragma unroll
3883     for (int k_step = 0; k_step < kValTileK; ++k_step) {
3884
3885 #pragma unroll
3886         for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3887             // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3888             const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3889             const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3890             // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3891             // lane_smem_a_k 中传给 swizzle<64>(), 而非像 swizzle kernel
3892             // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3893             //   swizzle<64> 作用于完整 BK=64, swizzle 周期覆盖全部 64 列,
3894             //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
3895             //   偏移必须在 swizzle 内部参与 chunk 计算。
3896             //   swizzle<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3897             //   swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
3898             const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3899             const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3900                 (stage * BM * BK + lane_smem_a_m * BK +
3901                  swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
3902                 sizeof(half);
3903             LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3904         }
3905     }

```



```

3906 #pragma unroll
3907 for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3908     // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3909     const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3910     const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3911     const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3912     const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3913         (stage * BN * BK + lane_smem_b_n * BK +
3914         swizzle<64>(lane_smem_b_n, lane_smem_b_k)) *
3915         sizeof(half);
3916     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3917 }
3918
3919 #pragma unroll
3920 for (int i = 0; i < kValTileM; ++i) {
3921     #pragma unroll
3922     for (int j = 0; j < kValTileN; ++j) {
3923         HMMMA16816(RC[i][j][0], RC[i][j][1],
3924             RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3925             RB[j][0], RB[j][1],
3926             RC[i][j][0], RC[i][j][1]);
3927     }
3928 }
3929 }
3930 [[maybe_unused]] auto token = empty[stage].arrive();
3931 }
3932
3933 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3934 // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3935 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3936 // 由 lane 0 发出对齐的 128-bit store.
3937 #pragma unroll
3938 for (int i = 0; i < kValTileM; ++i) {
3939     const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3940     const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3941     #pragma unroll
3942     for (int j = 0; j < kValTileN; ++j) {
3943         const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3944         RA[0][0] = RC[i][j][0];
3945         RA[1][0] = RC[i][j][1];
3946         RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3947         RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3948         RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
3949         RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3950         RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3951         RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3952         if (lane_id % 4 == 0) {
3953             const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3954             const int store_gmem_c_addr_0 =
3955                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3956             const int store_gmem_c_addr_1 =
3957                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3958             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3959                 *reinterpret_cast<float4*>(&RA[0][0]);
3960             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
3961                 *reinterpret_cast<float4*>(&RA[1][0]);
3962         }
3963     }
3964 }
3965 }
3966 }
3967
3968 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */

```

```

3969
3970 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3971 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3972 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
3973 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3974 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3975 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3976 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3977 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3978 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3979 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3980 //
3981 // * gmem_prob_stride + 1 的含义:
3982 //   语法层面 — 数组名退化 + 指针算术:
3983 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3984 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3985 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3986 //   语义层面 — 跳过隐式的最内维 stride:
3987 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3988 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3989 //     固定等于 sizeof(element_type), 不需要显式传入。
3990 //     对于 tensorRank=2 的 FP16 矩阵:
3991 //       dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
3992 //       dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
3993 //     gmem_prob_stride 数组的布局是内维在前:
3994 //       [0] = sizeof(half)                        ← 内维, 不传
3995 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3996 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3997 //
3998 // * smem_box_stride 为什么不需要 +1?
3999 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
4000 //   最内维 stride 也必须显式传入 (不隐式)。
4001 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
4002 //   都是连续存放的, 维度间没有 padding/stride。
4003 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
4004 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
4005 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
4006 //   两个 stride 值, 不需要跳过任何一个。
4007 //
4008 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
4009
4010 template <int BlockMajorSize, int BlockMinorSize>
4011 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
4012                                               half *gmem_ptr,
4013                                               int blocks_height,
4014                                               int blocks_width) {
4015     void *gmem_address = (void *)gmem_ptr;
4016     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
4017                                     (uint64_t)BlockMajorSize * blocks_height,
4018                                     1, 1, 1};
4019     uint64_t gmem_prob_stride[5] = {
4020         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
4021     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
4022                                    uint32_t(BlockMajorSize), 1, 1, 1};
4023     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
4024     CUresult result = cuTensorMapEncodeTiled(
4025         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
4026         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
4027         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
4028         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
4029     if (result != CUDA_SUCCESS)
4030         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
4031 }

```

```

4032 // BlockMajorSize = major dim (outer), BlockMinorSize = minor dim (inner, contiguous).
4033 // For row-major [Major, Minor] matrices.
4034 // Default <128, 64> matches HGEMM A/B^T and FA Q; FA K/V use <64, 64>.
4035 template <int BlockMajorSize = 128, int BlockMinorSize = 64>
4036 __host__ __static inline CUTensorMap *allocate_and_create_tensor_map(
4037     half *src, int blocks_height, int blocks_width) {
4038     CUTensorMap *tma_map_d;
4039     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
4040     CUTensorMap tma_map_host;
4041     create_tensor_map<BlockMajorSize, BlockMinorSize>(&tma_map_host, src,
4042                                                         blocks_height,
4043                                                         blocks_width);
4044     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
4045                cudaMemcpyHostToDevice);
4046     return tma_map_d;
4047 }
4048 }
4049 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
4050
4051 // =====
4052 // Phase 8: FlashAttention-2 + MMA m16n8k16)
4053 // =====
4054 // 面试要点 (FlashAttention-2算法) :
4055 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
4056 //    但 HBM 带宽是瓶颈 → FlashAttention-2用 tiling + online softmax 避免写回
4057 // 2. FA 三板斧:
4058 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
4059 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
4060 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
4061 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
4062 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
4063 //    - 优点: 减少 warp 间通信和 shuffle
4064 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
4065 //    for each K,V tile:
4066 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
4067 //        m_new = max(m_old, row_max(S_cur * scale))
4068 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
4069 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
4070 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
4071 //    O_final = O_new / l_final
4072 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
4073 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
4074 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
4075 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
4076 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
4077 //      都天然同构”的通用结论
4078 //
4079 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
4080 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
4081 // Grid: ((N + 63) / 64, B * H, 1), Br=64
4082 // Block: (128, 1, 1), kNumThreads=kWarpSize*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
4083 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
4084
4085 // =====
4086 // FlashAttention-2 Split-Q Kernel (完整实现)
4087 // =====
4088 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
4089 //
4090 // Tile 设计 (以 kHeadDim=64 为例) :
4091 // Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*{4,8}*1 = {64,128}
4092 // Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
4093 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
4094 //

```

```

4095 // 执行流程:
4096 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
4097 // 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
4098 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
4099 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
4100 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
4101 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
4102 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
4103 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
4104 // 7) 3e: Online rescaling — 0_new = exp(m_old-m_new)*0_old + P@V
4105 // 8) 最终 rescale: 0_final = (1/l_final) * 0_final
4106 // 9) Epilogue: warp shuffle + 128-bit collective store
4107 //
4108 // ★ Pad 与手动 XOR swizzle 的 profile 结论 (SM120, B=1,H=32,N=4096,D=64) :
4109 // - XOR 能消除目标 ldmatrix lane pattern 的 bank conflict, 但收益是局部的;
4110 // XOR 本身及 `x2` 的计算不是主要瓶颈, nvcc 已将该式化简为 bit operations。
4111 // - compact Q/K/V XOR 改变了 cp.async 的 shared destination pattern: LDGSTS
4112 // wavefronts 从 pad 的 30.15M 增至 68.16M (2.26x), long-scoreboard、
4113 // LG-throttle、MIO-throttle 分别约为 pad 的 3.25x、2.74x、1.60x。
4114 // - compact XOR 虽节省约 5 KiB smem, 但没有提高此 kernel 的 occupancy; 最终
4115 // 约 120.0 TFLOPS, 显著低于 Q/K/V kPad=8 的 166.6 TFLOPS。
4116 // - 所以当前 kernel 默认对 Q/K/V 都用 kPad=8。保留 swizzle 路径是为了学习和消融: 评价
4117 // shared layout 必须同时观察 ldmatrix reads 与 cp.async/LDGSTS writes, 不能只看
4118 // bank-conflict counter, 也不能只优化 XOR 地址算术。
4119 // ---- 寄存器填充辅助函数 (FlashAttention 用, 前移以供 TMA_MMA_WS kernel 使用) ----
4120 template <typename T, int M, const int N, const int K = 2>
4121 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
4122 #pragma unroll
4123     for (int i = 0; i < M; ++i)
4124 #pragma unroll
4125         for (int j = 0; j < N; ++j)
4126 #pragma unroll
4127             for (int k = 0; k < K; ++k)
4128                 R[i][j][k] = val;
4129 }
4130
4131 template <typename T, int M, const int N = 2>
4132 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
4133 #pragma unroll
4134     for (int i = 0; i < M; ++i)
4135 #pragma unroll
4136         for (int j = 0; j < N; ++j)
4137             R[i][j] = val;
4138 }
4139
4140 template <
4141     const int kHeadDim,           // head dim: 32, 64, 128
4142     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
4143     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
4144     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
4145     const int kMmaAccF32,         // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
4146     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
4147     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
4148     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
4149     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
4150     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
4151     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
4152     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
4153     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
4154     const int kStagesK,           // pipeline stages for K: >= 1; NO stages required for Q/V
4155     const int kPadQ,              // Q row padding; 0 selects compact XOR swizzle
4156     const int kPadK,              // K row padding; 0 selects compact XOR swizzle
4157     const int kPadV>

```

```

4158 __global__ void __launch_bounds__(kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK)
4159 flash_attn_mma_stages_split_q(
4160     half *Q, half *K, half *V, half *O, int N, int H) {
4161     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
4162     static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
4163     static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
4164         "Q/K/V padding must be non-negative");
4165     constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
4166     constexpr bool kSwizzleQ = kPadQ == 0;
4167     constexpr bool kSwizzleK = kPadK == 0;
4168     constexpr bool kSwizzleV = kPadV == 0;
4169     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 16*8*1=128
4170     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 8*1*8=64
4171     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 32*8*1=256
4172     const int Tc = (N + Bc - 1) / Bc;
4173     // 原始实现默认 seqLen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
4174     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
4175     const float scale = 1.0f / sqrtf((float)kHeadDim);
4176
4177     // Block indexing
4178     const int Nb_id = blockIdx.y / H; // batch id
4179     const int Nh_id = blockIdx.y % H; // head id
4180     const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
4181     const int tid = threadIdx.x;
4182     const int warp_id = tid / kWarpSize;
4183     const int lane_id = tid % kWarpSize;
4184     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
4185     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
4186
4187     // Global memory base offsets for this (batch, head)
4188     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
4189     const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
4190     const int K_gmem_offset = Q_gmem_offset;
4191     const int V_gmem_offset = Q_gmem_offset;
4192     const int O_gmem_offset = Q_gmem_offset;
4193
4194     // Thread-to-smem mapping for cooperative load
4195     int load_smem_Q_Br = tid / (kNumThreads / Br);
4196     int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
4197     int load_smem_K_Bc = tid / (kNumThreads / Bc);
4198     int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
4199     int load_smem_V_Bc = tid / (kNumThreads / Bc);
4200     int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
4201
4202     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
4203     if (load_gmem_Q_Br >= N)
4204         return;
4205
4206     // Shared memory layout
4207     // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
4208     // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
4209     // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
4210     // the 0/8 phase inside the final 16-half tile.
4211     extern __shared__ half smem[];
4212     constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
4213     constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
4214     constexpr int kSmemStrideQ = kHeadDim + kPadQ;
4215     constexpr int kSmemStrideK = kHeadDim + kPadK;
4216     constexpr int kSmemStrideV = kHeadDim + kPadV;
4217     half *Q_tile_smem = smem;
4218     half *K_tile_smem = Q_tile_smem + Q_tile_size;
4219     half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;
4220

```

```

4221 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
4222 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
4223 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
4224
4225 // Online Softmax persistent state
4226 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
4227 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
4228 float lane_block_row_max_old[kValTileSeqLenQ][2];
4229 float lane_block_row_sum_old[kValTileSeqLenQ][2];
4230 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
4231 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
4232
4233 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
4234 uint32_t R_K[kValTileSeqLenK][2]; // K regs
4235 uint32_t R_V[kValTileHeadDimV][2]; // V regs
4236 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
4237 // f16 acc: 每 tile 2 个 uint32 (4 half); f32 acc: 每 tile 4 个 uint32 (4 f32)。
4238 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
4239 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs]; // S=Q@K^T / P=softmax(S)
4240 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O for current tile
4241 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O accumulator (final output)
4242
4243 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
4244 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
4245
4246 // =====
4247 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
4248 // =====
4249 {
4250     int load_gmem_Q_addr =
4251         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
4252     if constexpr (kSwizzleQ) {
4253 #pragma unroll
4254         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4255             int col = load_smem_Q_d + i;
4256             uint32_t ptr = smem_Q_base_ptr +
4257                 ((col / kMmaAtomK) * Br * kMmaAtomK +
4258                 load_smem_Q_Br * kMmaAtomK +
4259                 swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
4260                 sizeof(half);
4261             CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
4262         }
4263     } else {
4264         uint32_t load_smem_Q_ptr =
4265             smem_Q_base_ptr +
4266             (load_smem_Q_Br * kMemStrideQ + load_smem_Q_d) * sizeof(half);
4267 #pragma unroll
4268         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4269             CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4270         }
4271     }
4272     CP_ASYNC_COMMIT_GROUP();
4273 }
4274
4275 // =====
4276 // Step 2: 预加载前 (kStagesK-1) 个 K tile (多 stage pipeline 预热)
4277 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4278 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
4279 // =====
4280 if constexpr (kStagesK > 1) {
4281 #pragma unroll
4282     for (int stage = 0; stage < (kStagesK - 1); ++stage) {
4283         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;

```



```

4284     int load_gmem_K_addr =
4285         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4286     if constexpr (kSwizzleK) {
4287 #pragma unroll
4288         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4289             int col = load_smem_K_d + i;
4290             uint32_t ptr = smem_K_base_ptr +
4291                 (stage * K_tile_size +
4292                  (col / kMmaAtomK) * Bc * kMmaAtomK +
4293                  load_smem_K_Bc * kMmaAtomK +
4294                  swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4295                 sizeof(half);
4296             CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4297         }
4298     } else {
4299         uint32_t load_smem_K_ptr =
4300             smem_K_base_ptr +
4301             (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
4302              load_smem_K_d) *
4303             sizeof(half);
4304 #pragma unroll
4305         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4306             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4307         }
4308     }
4309     CP_ASYNC_COMMIT_GROUP();
4310 }
4311 CP_ASYNC_WAIT_GROUP(kStagesK - 2);
4312 __syncthreads();
4313 }
4314
4315 // =====
4316 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
4317 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4318 // =====
4319 #pragma unroll 1
4320 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
4321     int smem_sel = tile_K_seqlen % kStagesK;
4322     int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;
4323
4324     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4325     // kStagesK>1: V 加载当前 tile, K 预取下一 tile (pipeline)
4326     // kStagesK=1: V 和 K 都加载当前 tile (smem_sel==smem_sel_next==0),
4327     // 无 pipeline 预取, 每轮 QK 前必须 wait 当前 K 就绪。
4328     // Load current V tile (no pipeline for V — one stage is enough)
4329     {
4330         int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
4331         int load_gmem_V_addr =
4332             V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4333         if constexpr (kSwizzleV) {
4334 #pragma unroll
4335             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4336                 int col = load_smem_V_d + i;
4337                 uint32_t ptr = smem_V_base_ptr +
4338                     ((col / kMmaAtomK) * Bc * kMmaAtomK +
4339                     load_smem_V_Bc * kMmaAtomK +
4340                     swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
4341                     sizeof(half);
4342                 CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
4343             }
4344         } else {
4345             uint32_t load_smem_V_ptr =
4346                 smem_V_base_ptr +

```

```

4347         (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
4348 #pragma unroll
4349     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4350         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4351     }
4352 }
4353 CP_ASYNC_COMMIT_GROUP();
4354 }
4355
4356 if constexpr (kStagesK > 1) {
4357     // Prefetch next K tile (pipelined)
4358     if ((tile_K_seqlen + 1) < Tc) {
4359         int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
4360         int load_gmem_K_addr =
4361             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4362         if constexpr (kSwizzleK) {
4363             #pragma unroll
4364             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4365                 int col = load_smem_K_d + i;
4366                 uint32_t ptr = smem_K_base_ptr +
4367                     (smem_sel_next * K_tile_size +
4368                     (col / kMmaAtomK) * Bc * kMmaAtomK +
4369                     load_smem_K_Bc * kMmaAtomK +
4370                     swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4371                     sizeof(half);
4372                 CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4373             }
4374         } else {
4375             uint32_t load_smem_K_ptr =
4376                 smem_K_base_ptr +
4377                 (smem_sel_next * K_tile_size +
4378                 load_smem_K_Bc * kSmemStrideK +
4379                 load_smem_K_d) *
4380                 sizeof(half);
4381             #pragma unroll
4382             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4383                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4384             }
4385         }
4386         CP_ASYNC_COMMIT_GROUP();
4387     }
4388 } else {
4389     // kStagesK==1: 加载当前 K tile (smem_sel_next == smem_sel == 0) 。
4390     // Step 2 预加载循环 (kStagesK-1=0) 不执行, 所以每轮 3a 必须加载当前 K。
4391     {
4392         int load_gmem_K_Bc = tile_K_seqlen * Bc + load_smem_K_Bc;
4393         int load_gmem_K_addr =
4394             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4395         if constexpr (kSwizzleK) {
4396             #pragma unroll
4397             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4398                 int col = load_smem_K_d + i;
4399                 uint32_t ptr = smem_K_base_ptr +
4400                     (smem_sel * K_tile_size +
4401                     (col / kMmaAtomK) * Bc * kMmaAtomK +
4402                     load_smem_K_Bc * kMmaAtomK +
4403                     swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4404                     sizeof(half);
4405                 CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4406             }
4407         } else {
4408             uint32_t load_smem_K_ptr =
4409                 smem_K_base_ptr +

```

```

4410         (smem_sel * K_tile_size +
4411         load_smem_K_Bc * kSmemStrideK +
4412         load_smem_K_d) *
4413         sizeof(half);
4414 #pragma unroll
4415     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4416         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4417     }
4418 }
4419 CP_ASYNC_COMMIT_GROUP();
4420 }
4421 }
4422
4423 // 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
4424 // kStagesK==1: 3a 刚提交当前 K 的 cp.async, 必须 wait 后才能 ldmatrix.
4425 // kStagesK>1: K 已在上一轮预取并 wait (Step 2 或 3e 末尾的 wait), 无需再 wait.
4426 if constexpr (kStagesK == 1) {
4427     CP_ASYNC_WAIT_GROUP(0);
4428     __syncthreads();
4429 }
4430 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
4431 #pragma unroll
4432 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4433     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4434     #pragma unroll
4435     for (int i = 0; i < kValTileSeqLenQ; ++i) {
4436         int warp_smem_Q_Br =
4437             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4438         int lane_smem_Q_Br =
4439             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4440         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4441         uint32_t lane_smem_Q_ptr;
4442         if constexpr (kSwizzleQ) {
4443             lane_smem_Q_ptr = smem_Q_base_ptr +
4444                 ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
4445                 lane_smem_Q_Br * kMmaAtomK +
4446                 swizzle<kMmaAtomK>(lane_smem_Q_Br,
4447                 lane_smem_Q_d % kMmaAtomK)) *
4448                 sizeof(half);
4449         } else {
4450             lane_smem_Q_ptr = smem_Q_base_ptr +
4451                 (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
4452         }
4453         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4454             lane_smem_Q_ptr);
4455     }
4456
4457     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4458     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4459     #pragma unroll
4460     for (int j = 0; j < kValTileSeqLenK; ++j) {
4461         int warp_smem_K_Bc =
4462             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4463         int lane_smem_K_Bc =
4464             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4465         int lane_smem_K_d =
4466             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4467         uint32_t lane_smem_K_ptr;
4468         if constexpr (kSwizzleK) {
4469             lane_smem_K_ptr = smem_K_base_ptr +
4470                 (smem_sel * K_tile_size +
4471                 (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
4472                 lane_smem_K_Bc * kMmaAtomK +

```

```

4473         swizzle<kMmaAtomK>(lane_smem_K_Bc,
4474                             lane_smem_K_d % kMmaAtomK)) *
4475         sizeof(half);
4476     } else {
4477         lane_smem_K_ptr = smem_K_base_ptr +
4478             (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
4479             lane_smem_K_d) *
4480             sizeof(half);
4481     }
4482     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4483 }
4484
4485 // MMA: S[tile] += Q[tile] @ K^T[tile]
4486 #pragma unroll
4487 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4488     #pragma unroll
4489     for (int j = 0; j < kValTileSeqLenK; ++j) {
4490         if constexpr (kMmaAccF32) {
4491             HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
4492                         R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4493                         R_K[j][0], R_K[j][1],
4494                         R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
4495         } else {
4496             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4497                     R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4498                     R_S[i][j][1]);
4499         }
4500     }
4501 }
4502 } // end loop over d
4503 __syncthreads();
4504
4505 // =====
4506 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4507 // =====
4508 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4509 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4510 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4511 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4512 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4513 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4514 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4515 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4516 // kValTileSeqLenK tiles.
4517
4518 float lane_row_max_new[kValTileSeqLenQ][2];
4519 float lane_row_sum_new[kValTileSeqLenQ][2];
4520 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4521 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4522
4523 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4524 #pragma unroll
4525 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4526     #pragma unroll
4527     for (int j = 0; j < kValTileSeqLenK; ++j) {
4528         float tmp_max_0, tmp_max_1;
4529         if constexpr (kMmaAccF32) {
4530             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
4531             tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
4532             tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
4533         } else {
4534             float2 t_reg_S_0 =
4535                 __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}

```

```

4536     float2 t_reg_S_1 =
4537         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4538     tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
4539     tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4540 }
4541 lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4542 lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4543 }
4544 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4545 // {0,4,8,...,28} hold valid data)
4546 lane_row_max_new[i][0] =
4547     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4548 lane_row_max_new[i][1] =
4549     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4550 }
4551
4552 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4553 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4554 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4555 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4556 #pragma unroll
4557 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4558     // m_new = max(m_old, m_cur)
4559     float block_row_max_new_0 =
4560         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4561     float block_row_max_new_1 =
4562         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4563
4564     #pragma unroll
4565     for (int j = 0; j < kValTileSeqLenK; ++j) {
4566         if constexpr (kMmaAccF32) {
4567             float *t_fptr_S = reinterpret_cast<float*>(&R_S[i][j][0]);
4568             half *t_hptr_S = reinterpret_cast<half*>(&R_S[i][j][0]);
4569             t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
4570             t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
4571             t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
4572             t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
4573             lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
4574             lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
4575             t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
4576             t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
4577             t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
4578             t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
4579         } else {
4580             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4581             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4582             t_reg_S_0.x =
4583                 __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4584             t_reg_S_0.y =
4585                 __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4586             t_reg_S_1.x =
4587                 __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4588             t_reg_S_1.y =
4589                 __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4590             lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4591             lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4592             HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4593             HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
4594         }
4595     }
4596
4597     // Warp-level reduce sum (kWarpWidth = 4, same as max)
4598     lane_row_sum_new[i][0] =

```

```

4599     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4600     lane_row_sum_new[i][1] =
4601         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4602 }
4603 __syncthreads();
4604
4605 // =====
4606 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
4607 // =====
4608 // Wait for V to be ready before computing P@V
4609 if constexpr (kStagesK > 1) {
4610     if ((tile_K_seqlen + 1) < Tc) {
4611         CP_ASYNC_WAIT_GROUP(1);
4612     } else {
4613         CP_ASYNC_WAIT_GROUP(0);
4614     }
4615 } else {
4616     CP_ASYNC_WAIT_GROUP(0);
4617 }
4618 __syncthreads();
4619
4620 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_0, 0);
4621
4622 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4623 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4624 #pragma unroll
4625 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4626     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4627     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4628     // transposed ldmatrix
4629 #pragma unroll
4630 for (int j = 0; j < kValTileHeadDimV; ++j) {
4631     int warp_smem_V_d =
4632         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4633     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4634     int lane_smem_V_d = warp_smem_V_d;
4635     uint32_t lane_smem_V_ptr;
4636     if constexpr (kSwizzleV) {
4637         lane_smem_V_ptr = smem_V_base_ptr +
4638             ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
4639              lane_smem_V_Bc * kMmaAtomK +
4640              swizzle<kMmaAtomK>(lane_smem_V_Bc,
4641                               lane_smem_V_d % kMmaAtomK)) *
4642             sizeof(half);
4643     } else {
4644         lane_smem_V_ptr = smem_V_base_ptr +
4645             (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
4646     }
4647     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4648 }
4649
4650 // P matrix layout in R_S[i][j][2]:
4651 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4652 // | 64x64 | warp_KV 0 |
4653 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4654 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4655 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4656 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4657 // tile_V_Bc selects which 16-column slice of P to use:
4658 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
4659 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4660 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4661 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6

```



```

4662 // 对应的 MMA A fragment 布局可以这样记:
4663 //   - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
4664 //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
4665 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
4666 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4667 // fragment 都无条件成立的通用结论。layout转换逻辑:
4668 //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
4669 int w = tile_V_Bc * 2;
4670 #pragma unroll
4671 for (int i = 0; i < kValTileSeqLenP; ++i) {
4672 #pragma unroll
4673     for (int j = 0; j < kValTileHeadDimV; ++j) {
4674         if constexpr (kMmaAccF32) {
4675             HMMA16816F32(R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3],
4676                         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
4677                         R_V[j][0], R_V[j][1],
4678                         R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3]);
4679         } else {
4680             HMMA16816(R_0[i][j][0], R_0[i][j][1],
4681                     R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
4682                     R_V[j][0], R_V[j][1],
4683                     R_0[i][j][0], R_0[i][j][1]);
4684         }
4685     }
4686 }
4687 } // end for tile_V_Bc
4688 __syncthreads();
4689
4690 // =====
4691 // 3e: Online rescaling —  $0_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * 0_{\text{old}} + P@V$ 
4692 // =====
4693 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{\text{old}} - m_{\text{new}})$  做 0 与 l 的 rescale
4694 #pragma unroll
4695 for (int i = 0; i < kValTileSeqLenP; ++i) {
4696     float block_row_max_new_0 = lane_row_max_new[i][0];
4697     float block_row_max_new_1 = lane_row_max_new[i][1];
4698     float block_row_sum_new_0 = lane_row_sum_new[i][0];
4699     float block_row_sum_new_1 = lane_row_sum_new[i][1];
4700
4701     float block_row_max_old_0 = lane_block_row_max_old[i][0];
4702     float block_row_max_old_1 = lane_block_row_max_old[i][1];
4703
4704     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4705     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4706
4707     // Handle first iteration: m_old = -inf, need to use m_new directly
4708     block_row_max_old_0 =
4709         (tile_K_seqLen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4710     block_row_max_old_1 =
4711         (tile_K_seqLen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4712
4713     float rescale_o_factor_0 =
4714         __expf(block_row_max_old_0 - block_row_max_new_0);
4715     float rescale_o_factor_1 =
4716         __expf(block_row_max_old_1 - block_row_max_new_1);
4717
4718     // Rescale 0_old + Add P@V in one fused step
4719 #pragma unroll
4720     for (int j = 0; j < kValTileHeadDimV; ++j) {
4721         if constexpr (kMmaAccF32) {
4722             float *t_fptr_0 = reinterpret_cast<float*>(&R_0[i][j][0]);
4723             float *t_fptr_D = reinterpret_cast<float*>(&R_D[i][j][0]);
4724             t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_0[0]);

```

```

4725     t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_0[1]);
4726     t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_0[2]);
4727     t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_0[3]);
4728 } else {
4729     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4730     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4731     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4732     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4733     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4734     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4735     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4736     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4737     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4738     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4739 }
4740 }
4741
4742 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
4743 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4744 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4745 lane_block_row_sum_old[i][0] = __fmaf_rn(
4746     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4747 lane_block_row_sum_old[i][1] = __fmaf_rn(
4748     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4749
4750 // Update m
4751 lane_block_row_max_old[i][0] = block_row_max_new_0;
4752 lane_block_row_max_old[i][1] = block_row_max_new_1;
4753 }
4754
4755 // Wait for next K tile to be ready in smem before next iteration
4756 if constexpr (kStagesK > 1) {
4757     if ((tile_K_seqlen + 1) < Tc) {
4758         CP_ASYNC_WAIT_GROUP(0);
4759     }
4760     __syncthreads();
4761 }
4762 } // end outer loop over K seqlen
4763 __syncthreads();
4764
4765 // =====
4766 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4767 // =====
4768 #pragma unroll
4769 for (int i = 0; i < kValTileSeqLenP; ++i) {
4770     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4771     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4772 #pragma unroll
4773 for (int j = 0; j < kValTileHeadDimV; ++j) {
4774     if constexpr (kMmaAccF32) {
4775         float *t_fptr_D = reinterpret_cast<float*>(&R_D[i][j][0]);
4776         half *t_hptr_D = reinterpret_cast<half*>(&R_D[i][j][0]);
4777         t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
4778         t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
4779         t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
4780         t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
4781     } else {
4782         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4783         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4784         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4785         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4786         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4787         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;

```

```

4788     HALF2(R_D[i][j][0]) = __float2half2_rn(t_reg_D_0);
4789     HALF2(R_D[i][j][1]) = __float2half2_rn(t_reg_D_1);
4790 }
4791 }
4792 }
4793
4794 // =====
4795 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4796 // =====
4797 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4798 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4799 #pragma unroll
4800 for (int i = 0; i < kValTileSeqLenP; ++i) {
4801     #pragma unroll
4802     for (int j = 0; j < kValTileHeadDimV; ++j) {
4803         uint32_t R_Z[2][4];
4804         R_Z[0][0] = R_D[i][j][0];
4805         R_Z[1][0] = R_D[i][j][1];
4806         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4807         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4808         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4809         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4810         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4811         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4812
4813         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4814         if (lane_id % 4 == 0) {
4815             int store_warp_regs_0_Br =
4816                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4817             int store_lane_gmem_0_Br =
4818                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4819             int store_warp_regs_0_d =
4820                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4821             int store_lane_gmem_0_d = store_warp_regs_0_d;
4822             int store_gmem_0_addr_0 = 0_gmem_offset +
4823                 (store_lane_gmem_0_Br + 0) * kHeadDim +
4824                 store_lane_gmem_0_d;
4825             int store_gmem_0_addr_1 = 0_gmem_offset +
4826                 (store_lane_gmem_0_Br + 8) * kHeadDim +
4827                 store_lane_gmem_0_d;
4828             // LDST128BITS = reinterpret_cast<float4*>
4829             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4830                 *reinterpret_cast<float4*>(&R_Z[0][0]);
4831             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4832                 *reinterpret_cast<float4*>(&R_Z[1][0]);
4833         }
4834     }
4835 }
4836 }
4837
4838 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
4839 // =====
4840 // Phase 8b: FlashAttention-2 TMA + MMA Warp Specialization (SM120)
4841 // =====
4842 // 面试要点 (FA TMA MMA WS vs cp.async FA) :
4843 //   - Producer (WG0, 128T) 用 TMA 128B SWIZZLE 搬运 Q/K/V tile 到 smem
4844 //   - Consumer (WG1, 256T) 用 ldmatrix + mma.sync.m16n8k16 做 Q@K^T / P@V
4845 //   - 与 hgemm_tma_mma_ws_tn 的对比:
4846 //     * HGEMM 只做一次 GEMM, K 维迭代; FA 做 QK^T 和 PV 两次 GEMM, KV seqlen 迭代
4847 //     * HGEMM 的 A/B 都 staged; FA 中 Q 只 load 一次 (split-Q), K staged, V 单 buffer
4848 //
4849 // ★ split-Q 语义 (关键, 避免误解) :
4850 //   - grid = (seqlen/Br, B*H), 每个 block 处理一个 **block-level Q tile [Br, d]**

```

```

4851 //      (不是全局 Q)。对 Q 的 seqlen 做 block-level 并行。
4852 //      - 每个 block 遍历**完整的 KV seqlen** (Tc = seqlen/Bc 次外循环)。
4853 //      - 因此每个 block 的 Q tile 只需 **load 一次到 smem**，在整个 KV 遍历中复用
4854 //      → 只需 full_Q barrier (无 empty_Q)。
4855 //
4856 // ★ Pipeline 同步协议 (arrive_count = 257 = 256 consumer + 1 producer) :
4857 //      - Q: full_Q — Producer 发 arrive_tx, Consumer 在主循环前 wait 一次
4858 //      - K: full_K[kStagesK] / empty_K[kStagesK] — 多 stage pipeline
4859 //      - V: full_V / empty_V — 单 buffer 串行 (QK 前发 TMA → QK 重叠 → PV 前等 → PV 后释放)
4860 //
4861 // ★ V ldmatrix.x2.trans 正确性 (已论证无风险) :
4862 //      TMA 128B SWIZZLE 是 1-1 映射, ldmatrix 用 swizzle<64>(row,col) 计算的物理地址
4863 //      = TMA 写入的物理地址 (smem 1024B 对齐保证 phase=0)。ldmatrix.x2.trans 的转置
4864 //      语义在寄存器层面工作, 与 smem 物理布局无关 → 必然正确。
4865 //
4866 // ★ Shared Memory 占用分析 (D=64/128, kStagesK=1/2/3/4, kStagesV=1/2) :
4867 //      smem = (Q[Br,D] + K[kStagesK,Bc,D] + V[kStagesV,Bc,D]) * sizeof(half)
4868 //            = D * (Br + kStagesK*Bc + kStagesV*Bc) * 2
4869 //            = D * (128 + (kStagesK+kStagesV)*64) * 2      (Br=128, Bc=64)
4870 //
4871 //      | kStagesK | kStagesV | D=64 bytes | D=128 bytes |
4872 //      |-----|-----|-----|-----|
4873 //      | 1      | 1      | 32 KB      | 64 KB      |
4874 //      | 2      | 1      | 40 KB      | 80 KB      |
4875 //      | 2      | 2      | 48 KB      | 96 KB      |
4876 //      | 3      | 1      | 48 KB      | 96 KB      |
4877 //      | 3      | 2      | 56 KB      | 112 KB     |
4878 //      | 4      | 1      | 56 KB      | 112 KB     |
4879 //
4880 //      SM120 (RTX PRO 5000) optin smem 上限 ~100KB:
4881 //      - D=64: S=1/2/3/4 全部可行 (32-56 KB)
4882 //      - D=128: S=1/2/3 可行 (64-96 KB); S=4 (112 KB) 超出 optin 上限, 会被
4883 //        check_smem_feasible 兜底判为 SMEM SKIP。D=128 推荐 S=2 (甜点: 80 KB,
4884 //        2 blocks/SM, pipeline 深度足够隐藏 TMA 延迟)。
4885 //
4886 // v1 限制: kHeadDim=64 or 128; seqlen % Br == 0 且 seqlen % Bc == 0 (不处理尾 tile)
4887 //
4888 // =====
4889 // FA TMA WS smem offset helper (D=64/128 通用)
4890 // =====
4891 // 背景: CU_TENSOR_MAP_SWIZZLE_128B 硬件要求 box innermost dim ≤ 128B = 64 half
4892 // (见 /usr/local/cuda/include/cuda.h 的 cuTensorMapEncodeTiled 文档)。因此 D=128
4893 // 不能用单个 TMA box=(128, Br) 覆盖整行。
4894 //
4895 // 方案: D=128 时 box 固定为 (64, Br), 沿 head_dim 方向连续发 kTmaChunks 次 TMA
4896 // (minor_coord = c*64), 写入 chunk-major smem 布局 [kTmaChunks, Br, 64]:
4897 // - chunk c 的 smem 起始偏移 = c * Br * 64
4898 // - chunk c 内部按 [Br, 64] row-major + SWIZZLE_128B 存放 (64 half = 128B 周期)
4899 //
4900 // 本 helper 计算 (row, col) 在该 chunk-major 布局下的 swizzled smem 元素偏移:
4901 // chunk = col / 64      ∈ [0, kTmaChunks)
4902 // col_in = col % 64     ∈ [0, 64)
4903 // offset = chunk * Br * 64 + row * 64 + swizzle<64>(row, col_in)
4904 //
4905 // D=64 退化: kTmaChunks=1, chunk=0, offset = row*64 + swizzle<64>(row, col)
4906 // 与原公式 (row*64 + swizzle<64>(row, col)) 完全一致 → 无回归。
4907 template <int kHeadDim, int Br>
4908 __device__ __forceinline__ int swizzle_fa(int row, int col) {
4909     static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
4910     if constexpr (kHeadDim == 64) {
4911         return row * 64 + swizzle<64>(row, col);
4912     } else { // kHeadDim == 128
4913         const int chunk = col >> 6; // col / 64 ∈ {0, 1}

```

```

4914     const int col_in = col & 63;          // col % 64
4915     return chunk * (Br * 64) + row * 64 + swizzle<64>(row, col_in);
4916 }
4917 }
4918
4919 template <
4920     const int kHeadDim,          // head dim: v1 only 64
4921     const int kMmaAtomM,        // 16 (MMA instruction M dim)
4922     const int kMmaAtomN,        // 8 (MMA instruction N dim)
4923     const int kMmaAtomK,        // 16 (MMA instruction K dim)
4924     const int kMmaAccF32,       // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
4925     const int kMmaTileSeqLenQ,  // MMA tiles along Q's M dim, 8 → Br=16*8=128
4926     const int kMmaTileSeqLenK,  // MMA tiles along K's N dim, 1 → Bc basis=8
4927     const int kMmaTileSeqLenP,  // MMA tiles for P@V M dim, == kMmaTileSeqLenQ
4928     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim direction)
4929     const int kValTileSeqLenQ,  // value tiles along Q's M, 1 → Br per warp=16
4930     const int kValTileSeqLenK,  // value tiles along K's N, 8 → Bc_warp=8*8=64
4931     const int kValTileSeqLenP,  // value tiles for P@V M dim, 1
4932     const int kValTileHeadDimV, // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
4933     const int kStagesK,         // K pipeline depth (>=1)
4934     const int kStagesV,         // V pipeline depth (>=1; 1=single buffer, >=2=pipelined)
4935     const int kNumThreads>      // 384, 128 producer + 256 consumer
4936 __global__ void __launch_bounds__(kNumThreads, 1)
4937     flash_attn_tma_mma_ws_stages_split_q(
4938         half *Q, half *K, half *V, half *O, int N, int H,
4939         const UTensorMap *__restrict__ tensorMapQ,
4940         const UTensorMap *__restrict__ tensorMapK,
4941         const UTensorMap *__restrict__ tensorMapV) {
4942     static_assert(kHeadDim == 64 || kHeadDim == 128,
4943         "v1 supports kHeadDim == 64 or 128");
4944     static_assert(kNumThreads == 384, "128 producer + 256 consumer");
4945     static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
4946     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
4947     static_assert(kStagesV >= 1, "kStagesV must be >= 1");
4948     constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
4949
4950     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 128
4951     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
4952     // BK for QK: kHeadDim=64, split into kHeadDim/kMmaAtomK=4 MMA K slices
4953     // BK for PV: Bc=64, split into Bc/kMmaAtomK=4 MMA K slices (P rows)
4954     constexpr int kConsumerThreads = 256; // 8 warps
4955     constexpr int kProducerThreads = 128; // 4 warps, only thread 0 issues TMA
4956     constexpr int kQTileBytes = Br * kHeadDim * sizeof(half); // 16 KB
4957     constexpr int kKTileBytes = Bc * kHeadDim * sizeof(half); // 8 KB
4958     constexpr int kVTileBytes = Bc * kHeadDim * sizeof(half); // 8 KB
4959     // TMA box innermost 固定 64 half (128B), 满足 CU_TENSOR_MAP_SWIZZLE_128B 的
4960     // box innermost ≤ 128B 硬约束。D=128 时沿 head_dim 发 kTmaChunks 次 TMA,
4961     // 写入 chunk-major smem 布局 [kTmaChunks, Br, 64]。
4962     constexpr int kTmaBoxMinor = 64;
4963     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
4964
4965     // Block indexing — split-Q: blockIdx.x indexes Q tile along seqlen
4966     const int Nb_id = blockIdx.y / H;
4967     const int Nh_id = blockIdx.y % H;
4968     const int Q_tile_id = blockIdx.x;
4969     const int Tc = (N + Bc - 1) / Bc; // number of KV tiles along seqlen
4970     const float scale = 1.0f / sqrtf((float)kHeadDim);
4971
4972     // Per-head gmem base offset (self-attention: Q=K=V share layout)
4973     const int QKV_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
4974     const int O_gmem_offset = QKV_gmem_offset;
4975     // TMA descriptor covers the entire [B*H*N, D] gmem as one 2D matrix; the
4976     // producer must offset major_coord by this head/batch base so each block

```

```

4977 // loads its own (batch, head) Q/K/V tile instead of always reading head 0.
4978 const int qkv_major_base = (Nb_id * H + Nh_id) * N;
4979
4980 // ---- Shared memory layout ----
4981 // TMA CU_TENSOR_MAP_SWIZZLE_128B requires 1024B-aligned smem base so the
4982 // hardware swizzle phase starts at zero. Consumer swizzle<64>() assumes
4983 // zero phase (no base_offset compensation like WGMMMA descriptor).
4984 // Q: [Br, kHeadDim] = [128, 64] = 16 KB (1024B aligned ✓)
4985 // K: [kStagesK, Bc, kHeadDim] = [kStagesK, 64, 64] (8 KB per stage)
4986 // V: [kStagesV, Bc, kHeadDim] = [kStagesV, 64, 64] (8 KB per stage)
4987 extern __shared__ __align__(1024) uint8_t smem_fa_tma_ws[];
4988 half *Q_smem = reinterpret_cast<half*>(smem_fa_tma_ws);
4989 half *K_smem = Q_smem + Br * kHeadDim;
4990 half *V_smem = K_smem + kStagesK * Bc * kHeadDim;
4991
4992 const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
4993 const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem);
4994 const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem);
4995
4996 // ---- Barriers ----
4997 // arrive_count = kConsumerThreads + 1 = 257 (256 consumer arrives + 1 producer arrive_tx)
4998 #pragma nv_diag_suppress static_var_with_dynamic_init
4999 __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
5000 __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kStagesK];
5001 __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kStagesK];
5002 __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kStagesV];
5003 __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kStagesV];
5004 #pragma nv_diag_default static_var_with_dynamic_init
5005
5006 // Thread partition: first kProducerThreads (0~127) = Producer WG,
5007 // remaining kConsumerThreads (128~383) = Consumer WG.
5008 // WARN: Must use kProducerThreads (not kConsumerThreads) as the divisor, otherwise
5009 // the split is wrong: 384/256=1.5 would give Producer 256 threads and
5010 // Consumer only 128, but barriers expect 256 consumer arrives → deadlock.
5011 const int wg_idx = (threadIdx.x < kProducerThreads) ? 0 : 1;
5012 const int wg_tid = (threadIdx.x < kProducerThreads) ? threadIdx.x :
5013                 (threadIdx.x - kProducerThreads);
5014
5015 if (threadIdx.x == 0) {
5016     init(&full_Q, kConsumerThreads + 1);
5017     for (int s = 0; s < kStagesV; ++s) {
5018         init(&full_V[s], kConsumerThreads + 1);
5019         init(&empty_V[s], kConsumerThreads + 1);
5020     }
5021     for (int s = 0; s < kStagesK; ++s) {
5022         init(&full_K[s], kConsumerThreads + 1);
5023         init(&empty_K[s], kConsumerThreads + 1);
5024     }
5025     tma_fence_proxy_async_shared_cta();
5026 }
5027 __syncthreads();
5028
5029 // =====
5030 // Producer Warpgroup (WG0, threadIdx.x 0~127)
5031 // Only wg_tid == 0 issues TMA. Split-Q: Q tile loaded once, K staged, V single buffer.
5032 // D=128 时每个 tile 沿 head_dim 发 kTmaChunks 次 TMA (box innermost=64 half) ,
5033 // 共享同一 barrier: N 次 cp.async.bulk.tensor 各 reduce chunk_bytes 到 tx count,
5034 // 1 次 mbarrier_arrive_expect_tx 声明总字节 + producer thread arrive.
5035 // =====
5036 if (wg_idx == 0) {
5037     NOTES_V2_REG_DEALLOC(40);
5038     if (wg_tid == 0) {
5039         // Step P0: Load block-level Q tile [Br, d] once. D=128 时发 kTmaChunks 次。

```



```

5040 // minor_coord = c * 64, major_coord = qkv_major_base + Q_tile_id * Br
5041 // smem dst = Q_smem + c * Br * 64 (chunk-major 布局)
5042 for (int c = 0; c < kTmaChunks; ++c) {
5043     tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,
5044                 c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
5045 }
5046 tma_arrive_expect_tx(full_Q, kQTileBytes);
5047
5048 // Step P1: Prefetch first (kStagesK-1) K tiles.
5049 // K tile coords: minor = c*64, major = qkv_major_base + s*Bc
5050 for (int s = 0; s < kStagesK - 1; ++s) {
5051     empty_K[s].wait(empty_K[s].arrive());
5052     for (int c = 0; c < kTmaChunks; ++c) {
5053         tma_load_2d(K_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5054                     tensorMapK, c * kTmaBoxMinor,
5055                     qkv_major_base + s * Bc, full_K[s]);
5056     }
5057     tma_arrive_expect_tx(full_K[s], kKTileBytes);
5058 }
5059
5060 // Step P1b: Prefetch first (kStagesV-1) V tiles (only if kStagesV > 1).
5061 // kStagesV=1: no prefetch; V[k] loaded in P2 overlaps only QK gemm.
5062 // kStagesV>=2: V prefetched here; in P2 V[k+kStagesV-1] overlaps
5063 // QK+softmax+PV of current iter → true V pipeline.
5064 if constexpr (kStagesV > 1) {
5065     for (int s = 0; s < kStagesV - 1; ++s) {
5066         empty_V[s].wait(empty_V[s].arrive());
5067         for (int c = 0; c < kTmaChunks; ++c) {
5068             tma_load_2d(V_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5069                         tensorMapV, c * kTmaBoxMinor,
5070                         qkv_major_base + s * Bc, full_V[s]);
5071         }
5072         tma_arrive_expect_tx(full_V[s], kVTileBytes);
5073     }
5074 }
5075
5076 // Step P2: Main loop over KV seqlen tiles
5077 for (int k = 0; k < Tc; ++k) {
5078     [[maybe_unused]] const int stage = k % kStagesK;
5079
5080     // Issue V[k+kStagesV-1] TMA to stage_next_v (pipelined).
5081     // kStagesV=1: loads V[k] to stage 0, overlaps with QK of same iter.
5082     // kStagesV>=2: loads V[k+kStagesV-1], overlaps with QK+softmax+PV.
5083     {
5084         const int v_tile = k + kStagesV - 1;
5085         if (v_tile < Tc) {
5086             const int stage_next_v = v_tile % kStagesV;
5087             empty_V[stage_next_v].wait(empty_V[stage_next_v].arrive());
5088             for (int c = 0; c < kTmaChunks; ++c) {
5089                 tma_load_2d(V_smem + stage_next_v * Bc * kHeadDim +
5090                             c * Bc * kTmaBoxMinor,
5091                             tensorMapV, c * kTmaBoxMinor,
5092                             qkv_major_base + v_tile * Bc,
5093                             full_V[stage_next_v]);
5094             }
5095             tma_arrive_expect_tx(full_V[stage_next_v], kVTileBytes);
5096         }
5097     }
5098
5099     // Prefetch K[k+kStagesK-1] (pipelined).
5100     if (k + kStagesK - 1 < Tc) {
5101         const int stage_next = (k + kStagesK - 1) % kStagesK;
5102         empty_K[stage_next].wait(empty_K[stage_next].arrive());

```

```

5103     for (int c = 0; c < kTmaChunks; ++c) {
5104         tma_load_2d(K_smem + stage_next * Bc * kHeadDim +
5105             c * Bc * kTmaBoxMinor,
5106             tensorMapK, c * kTmaBoxMinor,
5107             qkv_major_base + (k + kStagesK - 1) * Bc,
5108             full_K[stage_next]);
5109     }
5110     tma_arrive_expect_tx(full_K[stage_next], kKTileBytes);
5111 }
5112 }
5113 }
5114 }
5115 // =====
5116 // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
5117 // All 256 threads participate in ldmatrix + mma.sync.
5118 // =====
5119 else {
5120     const int warp_id = wg_tid / kWarpSize; // 0~7
5121     const int lane_id = wg_tid % kWarpSize; // 0~31
5122     // Split-Q warp layout: 8 warps form 8x1 grid (warp_QP = 0~7, warp_KV = 0)
5123     // Br = 16 * 8 * 1 = 128 → 8 warps each handle 16 rows of Q
5124     const int warp_QP = warp_id;
5125     const int warp_KV = 0;
5126
5127     // Step C0: init — mark all K stages and V stages as "empty" (consumable by producer)
5128     // Consumer register budget per Triton flash_attn_v2 maxnreg strategy on
5129     // Blackwell warp_specialize: D=128 → 168, otherwise → 80.
5130     if constexpr (kHeadDim == 128) {
5131         NOTES_V2_REG_ALLOC(168);
5132     } else {
5133         NOTES_V2_REG_ALLOC(80);
5134     }
5135
5136     for (int s = 0; s < kStagesK; ++s) {
5137         [[maybe_unused]] auto token = empty_K[s].arrive();
5138     }
5139     for (int s = 0; s < kStagesV; ++s) {
5140         [[maybe_unused]] auto token = empty_V[s].arrive();
5141     }
5142
5143     // Wait for Q tile ready once (split-Q: Q smem is reused across all KV iters)
5144     full_Q.wait(full_Q.arrive());
5145
5146     // Online softmax persistent state (per-warp, per-row-pair)
5147     float lane_block_row_max_old[kValTileSeqLenQ][2];
5148     float lane_block_row_sum_old[kValTileSeqLenQ][2];
5149     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
5150     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
5151
5152     uint32_t R_Q[kValTileSeqLenQ][4];
5153     uint32_t R_K[kValTileSeqLenK][2];
5154     uint32_t R_V[kValTileHeadDimV][2];
5155     uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs];
5156     uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5157     uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5158
5159     fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
5160
5161     // ---- Main loop over KV seqlen ----
5162     for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
5163         const int stage = tile_K_seqlen % kStagesK;
5164         const int stage_v = tile_K_seqlen % kStagesV;

```

```

5166 // Wait for K[stage] ready
5167 full_K[stage].wait(full_K[stage].arrive());
5168 tma_fence_proxy_async_shared_cta();
5169
5170 // ---- 3b: Q @ K^T = S[Br, Bc] ----
5171 // Q smem reused (split-Q). K from stage `stage`.
5172 // smem addr (TMA 128B swizzle, swizzle<64>)
5173 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
5174 #pragma unroll
5175 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
5176 // ldmatrix.x4: load Q m16k16 fragment (Q row-major, non-transpose)
5177 #pragma unroll
5178 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5179     const int warp_smem_Q_Br =
5180         warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
5181     const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
5182     const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
5183     const uint32_t lane_smem_Q_ptr =
5184         smem_Q_base_ptr +
5185         swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
5186             sizeof(half);
5187     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5188                 lane_smem_Q_ptr);
5189 }
5190
5191 // ldmatrix.x2: load K k16n8 fragment (K row-major = K^T col-major)
5192 #pragma unroll
5193 for (int j = 0; j < kValTileSeqLenK; ++j) {
5194     const int warp_smem_K_Bc =
5195         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
5196     const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
5197     const int lane_smem_K_d =
5198         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
5199     const uint32_t lane_smem_K_ptr =
5200         smem_K_base_ptr +
5201         (stage * Bc * kHeadDim +
5202          swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d)) *
5203             sizeof(half);
5204     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
5205 }
5206
5207 #pragma unroll
5208 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5209 #pragma unroll
5210     for (int j = 0; j < kValTileSeqLenK; ++j) {
5211         if constexpr (kMmaAccF32) {
5212             HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
5213                           R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5214                           R_K[j][0], R_K[j][1],
5215                           R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
5216         } else {
5217             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
5218                       R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
5219                       R_S[i][j][0], R_S[i][j][1]);
5220         }
5221     }
5222 }
5223 }
5224
5225 // Release K[stage] for producer reuse as early as possible: QK^T GEMM has
5226 // consumed all K smem data; the subsequent softmax (3c) and PV GEMM (3d)
5227 // only touch registers (R_S, R_0) and V smem, so K[stage] is free to be
5228 // overwritten by the producer's next TMA prefetch.

```

```

5229 {
5230     [[maybe_unused]] auto token = empty_K[stage].arrive();
5231 }
5232
5233 // ---- 3c: Online Safe Softmax — row max + exp + sum, P back to R_S ----
5234 // Softmax only touches R_S (registers), so V TMA latency can be overlapped
5235 // with this compute. Defer full_V.wait() until just before P@V (3d).
5236 float lane_row_max_new[kValTileSeqLenQ][2];
5237 float lane_row_sum_new[kValTileSeqLenQ][2];
5238 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
5239 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
5240
5241 // Pass 1: row-wise max across kValTileSeqLenK tiles (warp_reduce_max<4>)
5242 #pragma unroll
5243 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5244     #pragma unroll
5245     for (int j = 0; j < kValTileSeqLenK; ++j) {
5246         float tmp_max_0, tmp_max_1;
5247         if constexpr (kMmaAccF32) {
5248             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5249             tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
5250             tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
5251         } else {
5252             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5253             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
5254             tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
5255             tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
5256         }
5257         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
5258         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
5259     }
5260     lane_row_max_new[i][0] =
5261         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
5262     lane_row_max_new[i][1] =
5263         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
5264 }
5265
5266 // Pass 2: P = exp(S*scale - m_new), store back to R_S, accumulate row sums
5267 #pragma unroll
5268 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5269     float block_row_max_new_0 =
5270         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
5271     float block_row_max_new_1 =
5272         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
5273
5274     #pragma unroll
5275     for (int j = 0; j < kValTileSeqLenK; ++j) {
5276         if constexpr (kMmaAccF32) {
5277             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5278             half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
5279             t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
5280             t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
5281             t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
5282             t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
5283             lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
5284             lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
5285             t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
5286             t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
5287             t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
5288             t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
5289         } else {
5290             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5291             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));

```

```

5292         t_reg_S_0.x = __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
5293         t_reg_S_0.y = __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
5294         t_reg_S_1.x = __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
5295         t_reg_S_1.y = __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
5296         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
5297         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
5298         HALF2(R_S[i][j][0]) = __float2half2_rn(t_reg_S_0);
5299         HALF2(R_S[i][j][1]) = __float2half2_rn(t_reg_S_1);
5300     }
5301 }
5302 lane_row_sum_new[i][0] =
5303     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
5304 lane_row_sum_new[i][1] =
5305     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
5306 }
5307
5308 // ---- 3d: P @ V = 0[Br, d] ----
5309 // Wait for V[stage_v] ready just before P@V.
5310 // kStagesV=1: V was loaded in same iter, TMA overlaps QK+softmax.
5311 // kStagesV>=2: V was loaded in previous iter, TMA overlaps QK+softmax+PV.
5312 full_V[stage_v].wait(full_V[stage_v].arrive());
5313 tma_fence_proxy_async_shared_cta();
5314
5315 // ldmatrix.x2.trans: V row-major → col-major B fragment for NN matmul
5316 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_0, 0);
5317 #pragma unroll
5318 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
5319     #pragma unroll
5320     for (int j = 0; j < kValTileHeadDimV; ++j) {
5321         const int warp_smem_V_d =
5322             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
5323         const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
5324         const int lane_smem_V_d = warp_smem_V_d;
5325         const uint32_t lane_smem_V_ptr =
5326             smem_V_base_ptr +
5327             (stage_v * Bc * kHeadDim +
5328              swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d)) *
5329             sizeof(half);
5330         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
5331     }
5332
5333     // P fragment (R_S) directly feeds A of P@V MMA (layout reuse trick)
5334     const int w = tile_V_Bc * 2;
5335     #pragma unroll
5336     for (int i = 0; i < kValTileSeqLenP; ++i) {
5337         #pragma unroll
5338         for (int j = 0; j < kValTileHeadDimV; ++j) {
5339             if constexpr (kMmaAccF32) {
5340                 HMMA16816F32(R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3],
5341                             R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
5342                             R_V[j][0], R_V[j][1],
5343                             R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3]);
5344             } else {
5345                 HMMA16816(R_0[i][j][0], R_0[i][j][1],
5346                           R_S[i][w][0], R_S[i][w][1],
5347                           R_S[i][w + 1][0], R_S[i][w + 1][1],
5348                           R_V[j][0], R_V[j][1],
5349                           R_0[i][j][0], R_0[i][j][1]);
5350             }
5351         }
5352     }
5353 }

```

```

5355 // Release V[stage_v] for producer reuse: PV GEMM has consumed all
5356 // V smem data; the subsequent online rescaling (3e) only touches registers
5357 // (R_0, R_D, R_S, lane_block_*), so V[stage_v] is free to be overwritten.
5358 {
5359     [[maybe_unused]] auto token = empty_V[stage_v].arrive();
5360 }
5361
5362 // ---- 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  ----
5363 #pragma unroll
5364 for (int i = 0; i < kValTileSeqLenP; ++i) {
5365     float block_row_max_new_0 = lane_row_max_new[i][0];
5366     float block_row_max_new_1 = lane_row_max_new[i][1];
5367     float block_row_sum_new_0 = lane_row_sum_new[i][0];
5368     float block_row_sum_new_1 = lane_row_sum_new[i][1];
5369     float block_row_max_old_0 = lane_block_row_max_old[i][0];
5370     float block_row_max_old_1 = lane_block_row_max_old[i][1];
5371
5372     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
5373     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
5374
5375     // First iteration: m_old = -inf, use m_new directly
5376     block_row_max_old_0 = (tile_K_seqlen > 0 ? block_row_max_old_0
5377                               : block_row_max_new_0);
5378     block_row_max_old_1 = (tile_K_seqlen > 0 ? block_row_max_old_1
5379                               : block_row_max_new_1);
5380
5381     float rescale_o_factor_0 =
5382         __expf(block_row_max_old_0 - block_row_max_new_0);
5383     float rescale_o_factor_1 =
5384         __expf(block_row_max_old_1 - block_row_max_new_1);
5385
5386 #pragma unroll
5387 for (int j = 0; j < kValTileHeadDimV; ++j) {
5388     if constexpr (kMmaAccF32) {
5389         float *t_fptr_0 = reinterpret_cast<float*>(&R_0[i][j][0]);
5390         float *t_fptr_D = reinterpret_cast<float*>(&R_D[i][j][0]);
5391         t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_0[0]);
5392         t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_0[1]);
5393         t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_0[2]);
5394         t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_0[3]);
5395     } else {
5396         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
5397         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
5398         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
5399         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
5400         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
5401         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
5402         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
5403         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
5404         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
5405         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
5406     }
5407 }
5408
5409 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
5410 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
5411 lane_block_row_sum_old[i][0] =
5412     __fmaf_rn(rescale_o_factor_0, block_row_sum_old_0,
5413               block_row_sum_new_0);
5414 lane_block_row_sum_old[i][1] =
5415     __fmaf_rn(rescale_o_factor_1, block_row_sum_old_1,
5416               block_row_sum_new_1);
5417 lane_block_row_max_old[i][0] = block_row_max_new_0;

```



```

5418     lane_block_row_max_old[i][1] = block_row_max_new_1;
5419 }
5420 }
5421
5422 // ---- Step 4: Final rescale — 0_final = (1/l_final) * 0_final ----
5423 #pragma unroll
5424 for (int i = 0; i < kValTileSeqLenP; ++i) {
5425     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
5426     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
5427 #pragma unroll
5428 for (int j = 0; j < kValTileHeadDimV; ++j) {
5429     if constexpr (kMmaAccF32) {
5430         float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
5431         half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
5432         t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
5433         t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
5434         t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
5435         t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
5436     } else {
5437         float2 t_reg_D_0 = __half2float2(HALF2(R_D[i][j][0]));
5438         float2 t_reg_D_1 = __half2float2(HALF2(R_D[i][j][1]));
5439         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
5440         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
5441         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
5442         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
5443         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
5444         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
5445     }
5446 }
5447 }
5448
5449 // ---- Step 5: Epilogue — warp shuffle + 128-bit collective store ----
5450 #pragma unroll
5451 for (int i = 0; i < kValTileSeqLenP; ++i) {
5452 #pragma unroll
5453 for (int j = 0; j < kValTileHeadDimV; ++j) {
5454     uint32_t R_Z[2][4];
5455     R_Z[0][0] = R_D[i][j][0];
5456     R_Z[1][0] = R_D[i][j][1];
5457     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
5458     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
5459     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
5460     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
5461     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
5462     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
5463
5464     if (lane_id % 4 == 0) {
5465         const int store_warp_regs_0_Br =
5466             warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
5467         const int store_lane_gmem_0_Br =
5468             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
5469         const int store_warp_regs_0_d =
5470             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
5471         const int store_lane_gmem_0_d = store_warp_regs_0_d;
5472         const int store_gmem_0_addr_0 =
5473             0_gmem_offset + store_lane_gmem_0_Br * kHeadDim +
5474             store_lane_gmem_0_d;
5475         const int store_gmem_0_addr_1 =
5476             0_gmem_offset + (store_lane_gmem_0_Br + 8) * kHeadDim +
5477             store_lane_gmem_0_d;
5478         *reinterpret_cast<float4 *>(&O[store_gmem_0_addr_0]) =
5479             *reinterpret_cast<float4 *>(&R_Z[0][0]);
5480         *reinterpret_cast<float4 *>(&O[store_gmem_0_addr_1]) =

```

```

5481         *reinterpret_cast<float4*>(&R_Z[1][0]);
5482     }
5483 }
5484 }
5485 }
5486 }
5487 #endif // END NOTES_V2_ENABLE_TMA_MMA_WS
5488
5489 // =====
5490 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
5491 // =====
5492
5493 static inline void check(cudaError_t err, const char *msg) {
5494     if (err != cudaSuccess) {
5495         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
5496         exit(EXIT_FAILURE);
5497     }
5498 }
5499
5500
5501 static void test_block_reduce(int N) {
5502
5503     srand(42);
5504     float *h_a = (float *)malloc((size_t)N * sizeof(float));
5505     for (int i = 0; i < N; i++)
5506         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5507
5508     // CPU reference: sum of all elements
5509     double ref = 0.0;
5510     for (int i = 0; i < N; i++) ref += (double)h_a[i];
5511
5512     float *d_a, *d_y;
5513     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
5514     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
5515
5516     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
5517     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
5518
5519     dim3 block(128);
5520     dim3 grid((N + 127) / 128);
5521     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
5522     check(cudaGetLastError(), "blockreduce launch");
5523     check(cudaDeviceSynchronize(), "blockreduce sync");
5524
5525     float result;
5526     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
5527
5528     float err = fabsf(result - (float)ref);
5529     printf("| %-56s | %.6e | %-4s | %-22s |\n", "BlockReduce", err,
5530         err < 1e-2f ? "PASS" : "FAIL", "None");
5531
5532     free(h_a);
5533     cudaFree(d_a); cudaFree(d_y);
5534 }
5535
5536
5537 static void test_dot(int N) {
5538
5539     srand(42);
5540     float *h_a = (float *)malloc((size_t)N * sizeof(float));
5541     float *h_b = (float *)malloc((size_t)N * sizeof(float));
5542     for (int i = 0; i < N; i++) {
5543         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

5544     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5545 }
5546
5547 // CPU reference
5548 double ref = 0.0;
5549 for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
5550
5551 float *d_a, *d_b, *d_y;
5552 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
5553 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
5554 check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
5555
5556 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
5557 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
5558 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
5559
5560 dim3 block(128);
5561 dim3 grid((N + 127) / 128);
5562 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
5563 check(cudaGetLastError(), "dot launch");
5564 check(cudaDeviceSynchronize(), "dot sync");
5565
5566 float result;
5567 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
5568
5569 float err = fabsf(result - (float)ref);
5570 printf("| %-56s | %.6e | %-4s | %-22s |\n", "Dot", err,
5571     err < 1e-2f ? "PASS" : "FAIL", "None");
5572
5573 // ---- Dot Vec4 ----
5574 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
5575 dim3 block_v4(32);
5576 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
5577 check(cudaGetLastError(), "dot_vec4 launch");
5578 check(cudaDeviceSynchronize(), "dot_vec4 sync");
5579
5580 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
5581 float err_v4 = fabsf(result - (float)ref);
5582 printf("| %-56s | %.6e | %-4s | %-22s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL", "None");
5583
5584 free(h_a); free(h_b);
5585 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
5586 }
5587
5588
5589 static void test_relu(int N) {
5590     srand(42);
5591     float *h_x = (float *)malloc((size_t)N * sizeof(float));
5592     float *h_y = (float *)malloc((size_t)N * sizeof(float));
5593     for (int i = 0; i < N; i++)
5594         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5595
5596     float *d_x, *d_y;
5597     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
5598     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
5599     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
5600
5601     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
5602     dim3 block256(256);
5603     dim3 grid256((N + 255) / 256);
5604     relu<<<grid256, block256>>>(d_x, d_y, N);
5605     check(cudaGetLastError(), "relu launch");
5606     check(cudaDeviceSynchronize(), "relu sync");

```

```

5607 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
5608 float max_err = 0.0f;
5609 for (int i = 0; i < N; i++) {
5610     float expected = fmaxf(0.0f, h_x[i]);
5611     float err = fabsf(h_y[i] - expected);
5612     if (err > max_err) max_err = err;
5613 }
5614 printf("| %-56s | %.6e | %-4s | %-22s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5615
5616 dim3 block64(64);
5617 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
5618 check(cudaGetLastError(), "relu_vec4 launch");
5619 check(cudaDeviceSynchronize(), "relu_vec4 sync");
5620 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
5621 max_err = 0.0f;
5622 for (int i = 0; i < N; i++) {
5623     float expected = fmaxf(0.0f, h_x[i]);
5624     float err = fabsf(h_y[i] - expected);
5625     if (err > max_err) max_err = err;
5626 }
5627 printf("| %-56s | %.6e | %-4s | %-22s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5628
5629 free(h_x); free(h_y);
5630 cudaFree(d_x); cudaFree(d_y);
5631 }
5632
5633
5634 static void test_elementwise(int N) {
5635     srand(42);
5636     float *h_a = (float *)malloc((size_t)N * sizeof(float));
5637     float *h_b = (float *)malloc((size_t)N * sizeof(float));
5638     for (int i = 0; i < N; i++) {
5639         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5640         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5641     }
5642
5643     float *d_a, *d_b, *d_c;
5644     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
5645     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
5646     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
5647     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
5648     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
5649
5650     dim3 block256(256);
5651     dim3 grid256((N + 255) / 256);
5652     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
5653     check(cudaGetLastError(), "eadd launch");
5654     check(cudaDeviceSynchronize(), "eadd sync");
5655     float *h_c = (float *)malloc((size_t)N * sizeof(float));
5656     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
5657     float max_err = 0.0f;
5658     for (int i = 0; i < N; i++) {
5659         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
5660         if (err > max_err) max_err = err;
5661     }
5662     printf("| %-56s | %.6e | %-4s | %-22s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5663
5664     dim3 block64(64);
5665     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
5666     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
5667     check(cudaGetLastError(), "eadd_vec4 launch");

```

```

5668 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
5669 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
5670 max_err = 0.0f;
5671 for (int i = 0; i < N; i++) {
5672     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
5673     if (err > max_err) max_err = err;
5674 }
5675 printf("| %-56s | %.6e | %-4s | %-22s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
5676     , "None");
5677 free(h_a); free(h_b); free(h_c);
5678 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5679 }
5680
5681
5682 static void test_histogram(int N) {
5683     srand(42);
5684     int BINS = 16;
5685     int *h_hist = (int *)calloc(BINS, sizeof(int));
5686     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
5687     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
5688     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
5689     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
5690
5691     int *d_idx, *d_hist;
5692     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
5693     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
5694     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
5695     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
5696
5697     dim3 block256(256);
5698     dim3 grid256((N + 255) / 256);
5699     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
5700     check(cudaGetLastError(), "histogram launch");
5701     check(cudaDeviceSynchronize(), "histogram sync");
5702     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
5703
5704     float max_err = 0.0f;
5705     for (int i = 0; i < BINS; i++) {
5706         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
5707         if (err > max_err) max_err = err;
5708     }
5709     printf("| %-56s | %.6e | %-4s | %-22s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5710
5711     free(h_hist); free(h_hist_ref); free(h_idx);
5712     cudaFree(d_idx); cudaFree(d_hist);
5713 }
5714
5715
5716 static void test_merge_attn_states(int num_tokens, int num_heads,
5717     int head_size) {
5718
5719     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
5720     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
5721
5722     float *h_prefix_out = (float *)malloc(out_size);
5723     float *h_suffix_out = (float *)malloc(out_size);
5724     float *h_prefix_lse = (float *)malloc(lse_size);
5725     float *h_suffix_lse = (float *)malloc(lse_size);
5726     float *h_output_ref = (float *)malloc(out_size);
5727
5728     srand(42);

```

```

5729 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
5730     h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5731     h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5732 }
5733 for (int i = 0; i < num_heads * num_tokens; i++) {
5734     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
5735     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
5736 }
5737
5738 // CPU reference: 与 kernel 完全相同的浮点计算
5739 for (int t = 0; t < num_tokens; t++) {
5740     for (int h = 0; h < num_heads; h++) {
5741         float p_lse = h_prefix_lse[h * num_tokens + t];
5742         float s_lse = h_suffix_lse[h * num_tokens + t];
5743         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
5744         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
5745
5746         float max_lse = fmaxf(p_lse, s_lse);
5747         p_lse -= max_lse;
5748         s_lse -= max_lse;
5749         float p_se = expf(p_lse);
5750         float s_se = expf(s_lse);
5751         float p_scale = p_se / (p_se + s_se);
5752         float s_scale = s_se / (p_se + s_se);
5753
5754         int head_off = t * num_heads * head_size + h * head_size;
5755         for (int d = 0; d < head_size; d++) {
5756             h_output_ref[head_off + d] =
5757                 h_prefix_out[head_off + d] * p_scale +
5758                 h_suffix_out[head_off + d] * s_scale;
5759         }
5760     }
5761 }
5762
5763 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
5764 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
5765 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
5766 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
5767 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
5768 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
5769
5770 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
5771                 cudaMemcpyHostToDevice),
5772        "merge_attn H2D p_out");
5773 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
5774                 cudaMemcpyHostToDevice),
5775        "merge_attn H2D s_out");
5776 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
5777                 cudaMemcpyHostToDevice),
5778        "merge_attn H2D p_lse");
5779 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
5780                 cudaMemcpyHostToDevice),
5781        "merge_attn H2D s_lse");
5782
5783 int threads_per_head = head_size / 4;
5784 int total_threads = num_tokens * num_heads * threads_per_head;
5785 dim3 block(128);
5786 dim3 grid((total_threads + 127) / 128);
5787 merge_attn_states<<<grid, block>>>(
5788     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
5789     num_tokens, num_heads, head_size);
5790 check(cudaGetLastError(), "merge_attn launch");
5791 check(cudaDeviceSynchronize(), "merge_attn sync");

```



```

5792 float *h_output = (float *)malloc(out_size);
5793 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
5794        "merge_attn D2H");
5795
5796 float max_err = 0.0f;
5797 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
5798     float err = fabsf(h_output[i] - h_output_ref[i]);
5799     if (err > max_err) max_err = err;
5800 }
5801 printf("| %-56s | %.6e | %-4s | %-22s |\n", "MergeAttnStates", max_err,
5802        max_err < 1e-4f ? "PASS" : "FAIL", "None");
5803
5804 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
5805 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
5806 h_suffix_lse[0] = 0.0f;
5807 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
5808                 cudaMemcpyHostToDevice),
5809        "merge_attn H2D inf lse");
5810 merge_attn_states<<<grid, block>>>(
5811     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
5812     num_tokens, num_heads, head_size);
5813 check(cudaGetLastError(), "merge_attn inf launch");
5814 check(cudaDeviceSynchronize(), "merge_attn inf sync");
5815 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
5816        "merge_attn inf D2H");
5817
5818 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重 α=0)
5819 float inf_err = 0.0f;
5820 for (int d = 0; d < head_size; d++) {
5821     float err = fabsf(h_output[d] - h_suffix_out[d]);
5822     if (err > inf_err) inf_err = err;
5823 }
5824 printf("| %-56s | %.6e | %-4s | %-22s |\n", "MergeAttnStates-inf", inf_err,
5825        inf_err < 1e-4f ? "PASS" : "FAIL", "None");
5826
5827 free(h_prefix_out);
5828 free(h_suffix_out);
5829 free(h_prefix_lse);
5830 free(h_suffix_lse);
5831 free(h_output_ref);
5832 free(h_output);
5833 cudaFree(d_prefix_out);
5834 cudaFree(d_suffix_out);
5835 cudaFree(d_prefix_lse);
5836 cudaFree(d_suffix_lse);
5837 cudaFree(d_output);
5838 }
5839
5840
5841
5842 static void test_softmax(int N) {
5843     // Use N = blockDim.x so one block processes all elements as one token.
5844     constexpr int kNumThreads = 256;
5845     if (N != kNumThreads) {
5846         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
5847         return;
5848     }
5849
5850     srand(42);
5851     float *h_x = (float *)malloc((size_t)N * sizeof(float));
5852     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
5853     for (int i = 0; i < N; i++)
5854         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]

```

```

5855 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
5856 float max_val = -FLT_MAX;
5857 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
5858 double sum_exp = 0.0;
5859 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
5860 for (int i = 0; i < N; i++)
5861     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
5862
5863
5864 float *d_x, *d_y;
5865 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
5866 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
5867
5868 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
5869
5870 dim3 block(256);
5871 dim3 grid(1); // one token covering all N elements
5872 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
5873 check(cudaGetLastError(), "softmax launch");
5874 check(cudaDeviceSynchronize(), "softmax sync");
5875
5876 float *h_y = (float *)malloc((size_t)N * sizeof(float));
5877 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
5878
5879 float max_err = 0.0f;
5880 for (int i = 0; i < N; i++) {
5881     float err = fabsf(h_y[i] - h_y_ref[i]);
5882     if (err > max_err) max_err = err;
5883 }
5884 printf("| %-56s | %.6e | %-4s | %-22s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
5885        ", "None");
5886
5887 // ---- Safe Softmax ----
5888 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
5889 check(cudaGetLastError(), "safe_softmax launch");
5890 check(cudaDeviceSynchronize(), "safe_softmax sync");
5891 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
5892 max_err = 0.0f;
5893 for (int i = 0; i < N; i++) {
5894     float err = fabsf(h_y[i] - h_y_ref[i]);
5895     if (err > max_err) max_err = err;
5896 }
5897 printf("| %-56s | %.6e | %-4s | %-22s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
5898        None");
5899
5900 // ---- Naive Softmax ----
5901 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
5902 check(cudaGetLastError(), "naive_softmax launch");
5903 check(cudaDeviceSynchronize(), "naive_softmax sync");
5904 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
5905 max_err = 0.0f;
5906 for (int i = 0; i < N; i++) {
5907     float err = fabsf(h_y[i] - h_y_ref[i]);
5908     if (err > max_err) max_err = err;
5909 }
5910 printf("| %-56s | %.6e | %-4s | %-22s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
5911        None");
5912 free(h_x); free(h_y); free(h_y_ref);
5913 cudaFree(d_x); cudaFree(d_y);
5914 }

```

```

5915 static void test_rms_norm(int N, int K) {
5916
5917     srand(42);
5918     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
5919     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
5920     for (int i = 0; i < N * K; i++)
5921         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5922     float g = 1.5f; // gain
5923
5924     // CPU reference: y = (x / rms(x)) * g
5925     float epsilon = 1e-5f;
5926     for (int n = 0; n < N; n++) {
5927         double sum_sq = 0.0;
5928         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
5929         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
5930         for (int k = 0; k < K; k++)
5931             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
5932     }
5933
5934     float *d_x, *d_y;
5935     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
5936     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
5937     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
5938
5939     dim3 block(128);
5940     dim3 grid(N);
5941     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
5942     check(cudaGetLastError(), "rmsnorm launch");
5943     check(cudaDeviceSynchronize(), "rmsnorm sync");
5944
5945     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
5946     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
5947
5948     float max_err = 0.0f;
5949     for (int i = 0; i < N * K; i++) {
5950         float err = fabsf(h_y[i] - h_y_ref[i]);
5951         if (err > max_err) max_err = err;
5952     }
5953     printf("| %-56s | %.6e | %-4s | %-22s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None")
5954         ;
5955
5956     // ---- RMS Norm Vec4 ----
5957     dim3 block_rv4(32);
5958     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
5959     check(cudaGetLastError(), "rmsnorm_vec4 launch");
5960     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
5961     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
5962     max_err = 0.0f;
5963     for (int i = 0; i < N * K; i++) {
5964         float err = fabsf(h_y[i] - h_y_ref[i]);
5965         if (err > max_err) max_err = err;
5966     }
5967     printf("| %-56s | %.6e | %-4s | %-22s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
5968         None");
5969
5970     free(h_x); free(h_y); free(h_y_ref);
5971     cudaFree(d_x); cudaFree(d_y);
5972 }
5973
5974 static void test_layer_norm(int N, int K) {
5975     srand(42);

```

```

5976 float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
5977 float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
5978 for (int i = 0; i < N * K; i++)
5979     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5980 float g = 1.5f, b = 0.3f; // gain and bias
5981
5982 // CPU reference: y = ((x - mean) / std) * g + b
5983 float epsilon = 1e-5f;
5984 for (int n = 0; n < N; n++) {
5985     double sum = 0.0;
5986     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
5987     float mean = (float)sum / (float)K;
5988     double sum_sq = 0.0;
5989     for (int k = 0; k < K; k++) {
5990         float diff = h_x[n * K + k] - mean;
5991         sum_sq += (double)diff * (double)diff;
5992     }
5993     float std = sqrtf((float)sum_sq / (float)K + epsilon);
5994     for (int k = 0; k < K; k++)
5995         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
5996 }
5997
5998 float *d_x, *d_y;
5999 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
6000 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
6001 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
6002
6003 dim3 block(128);
6004 dim3 grid(N);
6005 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
6006 check(cudaGetLastError(), "layernorm launch");
6007 check(cudaDeviceSynchronize(), "layernorm sync");
6008
6009 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
6010 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
6011
6012 float max_err = 0.0f;
6013 for (int i = 0; i < N * K; i++) {
6014     float err = fabsf(h_y[i] - h_y_ref[i]);
6015     if (err > max_err) max_err = err;
6016 }
6017 printf("| %-56s | %.6e | %-4s | %-22s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
6018
6019 // ---- Layer Norm Vec4 ----
6020 dim3 block_lv4(32);
6021 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
6022 check(cudaGetLastError(), "layernorm_vec4 launch");
6023 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
6024 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
6025 max_err = 0.0f;
6026 for (int i = 0; i < N * K; i++) {
6027     float err = fabsf(h_y[i] - h_y_ref[i]);
6028     if (err > max_err) max_err = err;
6029 }
6030 printf("| %-56s | %.6e | %-4s | %-22s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
6031
6032 free(h_x); free(h_y); free(h_y_ref);
6033 cudaFree(d_x); cudaFree(d_y);
6034 }
6035
6036

```

```

6037 static void test_rope(int seq_len, int N) {
6038
6039     int total_pairs = seq_len * N;
6040     int total_elems = total_pairs * 2;
6041     size_t size = (size_t)total_elems * sizeof(float);
6042
6043     float *h_x = (float *)malloc(size);
6044     float *h_y_ref = (float *)malloc(size);
6045
6046     srand(42);
6047     for (int i = 0; i < total_elems; i++)
6048         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6049
6050     // CPU reference: 2D rotation for each pair
6051     for (int idx = 0; idx < total_pairs; idx++) {
6052         int token_pos = idx / N;
6053         int token_idx = idx % N;
6054         float x1 = h_x[idx * 2];
6055         float x2 = h_x[idx * 2 + 1];
6056         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
6057         float angle = (float)token_pos * theta;
6058         float cos_v = cosf(angle);
6059         float sin_v = sinf(angle);
6060         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
6061         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
6062     }
6063
6064     float *d_x, *d_y;
6065     check(cudaMalloc(&d_x, size), "rope alloc X");
6066     check(cudaMalloc(&d_y, size), "rope alloc Y");
6067     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
6068
6069     dim3 block(256);
6070     dim3 grid((total_pairs + 255) / 256);
6071     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
6072     check(cudaGetLastError(), "rope launch");
6073     check(cudaDeviceSynchronize(), "rope sync");
6074
6075     float *h_y = (float *)malloc(size);
6076     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
6077
6078     float max_err = 0.0f;
6079     for (int i = 0; i < total_elems; i++) {
6080         float err = fabsf(h_y[i] - h_y_ref[i]);
6081         if (err > max_err) max_err = err;
6082     }
6083     printf("| %-56s | %.6e | %-4s | %-22s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
6084
6085     free(h_x);
6086     free(h_y);
6087     free(h_y_ref);
6088     cudaFree(d_x);
6089     cudaFree(d_y);
6090 }
6091
6092
6093 static void test_mat_transpose(int row, int col) {
6094
6095     size_t size_in = (size_t)row * col * sizeof(float);
6096     size_t size_out = (size_t)col * row * sizeof(float);
6097
6098     float *h_x = (float *)malloc(size_in);
6099     float *h_y_ref = (float *)malloc(size_out);

```

```

6100
6101 srand(42);
6102 for (int i = 0; i < row * col; i++)
6103     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6104
6105 // CPU reference: y[j][i] = x[i][j] (row-major)
6106 for (int i = 0; i < row; i++)
6107     for (int j = 0; j < col; j++)
6108         h_y_ref[j * row + i] = h_x[i * col + j];
6109
6110 float *d_x, *d_y;
6111 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
6112 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
6113 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
6114
6115 dim3 block(16, 16);
6116 dim3 grid((col + 15) / 16, (row + 15) / 16);
6117 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
6118 check(cudaGetLastError(), "mattrans launch");
6119 check(cudaDeviceSynchronize(), "mattrans sync");
6120
6121 float *h_y = (float *)malloc(size_out);
6122 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
6123
6124 float max_err = 0.0f;
6125 for (int i = 0; i < col * row; i++) {
6126     float err = fabsf(h_y[i] - h_y_ref[i]);
6127     if (err > max_err) max_err = err;
6128 }
6129 printf("| %-56s | %.6e | %-4s | %-22s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "
    None");
6130
6131 free(h_x);
6132 free(h_y);
6133 free(h_y_ref);
6134 cudaFree(d_x);
6135 cudaFree(d_y);
6136 }
6137
6138
6139 static void test_mat_transpose_padded(int row, int col) {
6140
6141     size_t size_in = (size_t)row * col * sizeof(float);
6142     size_t size_out = (size_t)col * row * sizeof(float);
6143
6144     float *h_x = (float *)malloc(size_in);
6145     float *h_y_ref = (float *)malloc(size_out);
6146
6147     srand(42);
6148     for (int i = 0; i < row * col; i++)
6149         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6150
6151     // CPU reference: y[j][i] = x[i][j] (row-major)
6152     for (int i = 0; i < row; i++)
6153         for (int j = 0; j < col; j++)
6154             h_y_ref[j * row + i] = h_x[i * col + j];
6155
6156     float *d_x, *d_y;
6157     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
6158     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
6159     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
6160
6161     dim3 block(16, 16);

```



```

6162 dim3 grid((col + 15) / 16, (row + 63) / 64);
6163 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
6164 check(cudaGetLastError(), "mattrans_padded launch");
6165 check(cudaDeviceSynchronize(), "mattrans_padded sync");
6166
6167 float *h_y = (float *)malloc(size_out);
6168 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
6169
6170 float max_err = 0.0f;
6171 for (int i = 0; i < col * row; i++) {
6172     float err = fabsf(h_y[i] - h_y_ref[i]);
6173     if (err > max_err) max_err = err;
6174 }
6175 printf("| %-56s | %.6e | %-4s | %-22s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "
    FAIL", "None");
6176
6177 free(h_x);
6178 free(h_y);
6179 free(h_y_ref);
6180 cudaFree(d_x);
6181 cudaFree(d_y);
6182 }
6183
6184
6185 static void test_sgemv(int M, int K) {
6186
6187     srand(42);
6188     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
6189     float *h_x = (float *)malloc((size_t)K * sizeof(float));
6190     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
6191     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6192     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6193
6194     // CPU reference: y[m] = sum_k A[m,k] * x[k]
6195     for (int m = 0; m < M; m++) {
6196         double sum = 0.0;
6197         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
6198         h_y_ref[m] = (float)sum;
6199     }
6200
6201     float *d_a, *d_x, *d_y;
6202     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
6203     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
6204     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
6205
6206     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
6207     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
6208
6209     dim3 block(32, 4);
6210     dim3 grid((M + 3) / 4);
6211     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
6212     check(cudaGetLastError(), "sgemv launch");
6213     check(cudaDeviceSynchronize(), "sgemv sync");
6214
6215     float *h_y = (float *)malloc((size_t)M * sizeof(float));
6216     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
6217
6218     float max_err = 0.0f;
6219     for (int m = 0; m < M; m++) {
6220         float err = fabsf(h_y[m] - h_y_ref[m]);
6221         if (err > max_err) max_err = err;
6222     }
6223     printf("| %-56s | %.6e | %-4s | %-22s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "

```

```

        None");
6224
6225 // ---- SGEMV K32 ----
6226 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
6227 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
6228 check(cudaGetLastError(), "sgemv_k32 launch");
6229 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
6230
6231 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
6232
6233 max_err = 0.0f;
6234 for (int m = 0; m < M; m++) {
6235     float err = fabsf(h_y[m] - h_y_ref[m]);
6236     if (err > max_err) max_err = err;
6237 }
6238 printf("| %-56s | %.6e | %-4s | %-22s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
        ");
6239
6240 // ---- SGEMV K16 ----
6241 free(h_a); free(h_x); free(h_y); free(h_y_ref);
6242 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
6243
6244 int K16 = 16;
6245 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
6246 h_x = (float *)malloc((size_t)K16 * sizeof(float));
6247 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
6248 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6249 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6250
6251 for (int m = 0; m < M; m++) {
6252     double sum = 0.0;
6253     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
6254     h_y_ref[m] = (float)sum;
6255 }
6256
6257 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
6258 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
6259 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
6260
6261 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
6262 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
6263
6264 dim3 grid_k16((M + 7) / 8);
6265 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
6266 check(cudaGetLastError(), "sgemv_k16 launch");
6267 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
6268
6269 h_y = (float *)malloc((size_t)M * sizeof(float));
6270 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
6271
6272 max_err = 0.0f;
6273 for (int m = 0; m < M; m++) {
6274     float err = fabsf(h_y[m] - h_y_ref[m]);
6275     if (err > max_err) max_err = err;
6276 }
6277 printf("| %-56s | %.6e | %-4s | %-22s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
        ");
6278
6279 free(h_a); free(h_x); free(h_y); free(h_y_ref);
6280 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
6281 }
6282
6283

```

```

6284 static void test_sgemm(int M, int N, int K) {
6285
6286     size_t size_a = (size_t)M * K * sizeof(float);
6287     size_t size_b = (size_t)K * N * sizeof(float);
6288     size_t size_c = (size_t)M * N * sizeof(float);
6289
6290     float *h_a = (float *)malloc(size_a);
6291     float *h_b = (float *)malloc(size_b);
6292     float *h_c_ref = (float *)malloc(size_c);
6293
6294     srand(42);
6295     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6296     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
6297
6298     float *d_a, *d_b, *d_c;
6299     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
6300     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
6301     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
6302
6303     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
6304     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
6305
6306     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
6307     cublasHandle_t handle;
6308     cublasCreate(&handle);
6309     float alpha = 1.0f, beta = 0.0f;
6310     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
6311                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
6312     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
6313
6314     // Kernel
6315     cudaEvent_t start, stop;
6316     cudaEventCreate(&start);
6317     cudaEventCreate(&stop);
6318
6319     dim3 block(32, 32);
6320     dim3 grid((N + 31) / 32, (M + 31) / 32);
6321     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
6322     check(cudaGetLastError(), "sgemm launch");
6323     check(cudaDeviceSynchronize(), "sgemm sync");
6324
6325     float *h_c = (float *)malloc(size_c);
6326     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
6327
6328     // Verify
6329     float max_err = 0.0f;
6330     for (int i = 0; i < M * N; i++) {
6331         float err = fabsf(h_c[i] - h_c_ref[i]);
6332         if (err > max_err) max_err = err;
6333     }
6334     printf("| %-56s | %.6e | %-4s | %-22s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
6335
6336     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
6337
6338     dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
6339     check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
6340     sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
6341     check(cudaGetLastError(), "sgemm_vec4 launch");
6342     check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
6343
6344     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
6345
6346     max_err = 0.0f;

```

```

6347     for (int i = 0; i < M * N; i++) {
6348         float err = fabsf(h_c[i] - h_c_ref[i]);
6349         if (err > max_err) max_err = err;
6350     }
6351     printf("| %-56s | %.6e | %-4s | %-22s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "
        None");
6352
6353     free(h_a); free(h_b); free(h_c); free(h_c_ref);
6354     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
6355     cublasDestroy(handle);
6356     cudaEventDestroy(start);
6357     cudaEventDestroy(stop);
6358 }
6359
6360
6361 static void test_hgemmm_mma(int M, int N, int K) {
6362
6363     size_t size_a = (size_t)M * K * sizeof(half);
6364     size_t size_b = (size_t)K * N * sizeof(half);
6365     size_t size_c = (size_t)M * N * sizeof(half);
6366
6367     half *h_a = (half *)malloc(size_a);
6368     half *h_b = (half *)malloc(size_b);
6369     half *h_c_ref = (half *)malloc(size_c);
6370
6371     srand(42);
6372     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6373     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6374
6375     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
6376     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
6377     size_t size_b_t = (size_t)N * K * sizeof(half);
6378     half *h_b_t = (half *)malloc(size_b_t);
6379     for (int n = 0; n < N; n++)
6380         for (int k = 0; k < K; k++)
6381             h_b_t[n * K + k] = h_b[k * N + n];
6382
6383     half *d_a, *d_b, *d_b_t, *d_c;
6384     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
6385     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
6386     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
6387     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
6388
6389     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
6390     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
6391     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
6392
6393     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
6394     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16 accumulation.
6395     cublasHandle_t handle;
6396     cublasCreate(&handle);
6397     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6398     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
6399                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
6400                 &beta_h, d_c, CUDA_R_16F, N,
6401                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
6402     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
6403
6404     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
6405     cudaEvent_t start, stop;
6406     cudaEventCreate(&start);
6407     cudaEventCreate(&stop);
6408

```

```

6409 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
6410 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
6411 dim3 block(256);
6412 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
6413 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
6414 check(cudaGetLastError(), "hgemm launch");
6415 check(cudaDeviceSynchronize(), "hgemm sync");
6416
6417 half *h_c = (half *)malloc(size_c);
6418 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
6419
6420 // Verify
6421 float max_err = 0.0f;
6422 for (int i = 0; i < M * N; i++) {
6423     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6424     if (err > max_err) max_err = err;
6425 }
6426 printf("| %-56s | %.6e | %-4s | %-22s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL", "None"
        );
6427
6428 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
6429 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
6430 cublasDestroy(handle);
6431 cudaEventDestroy(start);
6432 cudaEventDestroy(stop);
6433 }
6434
6435
6436 static void test_hgemm_swizzle(int M, int N, int K) {
6437     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
6438     // + Register Double Buffering (kValTileK=4, BK=64)
6439     // TN layout: C[M×N] = A[M×K] × BT[N×K]
6440     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
6441     // (kValTileK=4, kStages=2, BK=64)
6442     // smem: kStages × (BM×BK + BN×BK) halves
6443
6444     size_t size_a = (size_t)M * K * sizeof(half);
6445     size_t size_b = (size_t)K * N * sizeof(half);
6446     size_t size_c = (size_t)M * N * sizeof(half);
6447
6448     half *h_a = (half *)malloc(size_a);
6449     half *h_b = (half *)malloc(size_b);
6450     half *h_c_ref = (half *)malloc(size_c);
6451
6452     srand(42);
6453     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6454     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6455
6456     // Kernel expects BT [N×K] row-major layout (TN layout convention).
6457     size_t size_b_t = (size_t)N * K * sizeof(half);
6458     half *h_b_t = (half *)malloc(size_b_t);
6459     for (int n = 0; n < N; n++)
6460         for (int k = 0; k < K; k++)
6461             h_b_t[n * K + k] = h_b[k * N + n];
6462
6463     half *d_a, *d_b, *d_b_t, *d_c;
6464     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
6465     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
6466     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
6467     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
6468
6469     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
6470     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");

```

```

6471 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
6472
6473 // cuBLAS FP16 reference
6474 cublasHandle_t handle;
6475 cublasCreate(&handle);
6476 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6477 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
6478             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
6479             &beta_h, d_c, CUDA_R_16F, N,
6480             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
6481 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
6482
6483 // MMA swizzle kernel (default params: kStages=2, kValTileK=4, BK=64)
6484 constexpr int BM = 128, BN = 128, BK = 64, K_STAGE_S = 2;
6485 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
6486 cudaFuncSetAttribute(
6487     (const void *)hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, K_STAGE_S, 0>,
6488     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6489 dim3 block(256);
6490 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
6491 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
6492 check(cudaGetLastError(), "hgemm_swizzle launch");
6493 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
6494
6495 half *h_c = (half *)malloc(size_c);
6496 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
6497
6498 // Verify
6499 float max_err = 0.0f;
6500 for (int i = 0; i < M * N; i++) {
6501     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6502     if (err > max_err) max_err = err;
6503 }
6504 printf("| %-56s | %.6e | %-4s | %-22s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "
6505     FAIL", "None");
6506
6507 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
6508 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
6509 cublasDestroy(handle);
6510 }
6511
6512 #if defined(NOTES_V2_ENABLE_CUTE)
6513 static void test_hgemm_cute(int M, int N, int K) {
6514     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3> + kStage=2
6515     // TN layout: C[MxN] = A[MxK] x B^T[NxK]
6516     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
6517     // Tile: BM=128, BN=256, BK=32, 128 threads/block
6518
6519     size_t size_a = (size_t)M * K * sizeof(half);
6520     size_t size_b = (size_t)K * N * sizeof(half);
6521     size_t size_c = (size_t)M * N * sizeof(half);
6522
6523     half *h_a = (half *)malloc(size_a);
6524     half *h_b = (half *)malloc(size_b);
6525     half *h_c_ref = (half *)malloc(size_c);
6526
6527     srand(42);
6528     for (int i = 0; i < M * K; i++)
6529         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6530     for (int i = 0; i < K * N; i++)
6531         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6532

```



```

6533 // CuTe kernel expects B^T [N×K] row-major (TN layout).
6534 size_t size_b_t = (size_t)N * K * sizeof(half);
6535 half *h_b_t = (half *)malloc(size_b_t);
6536 for (int n = 0; n < N; n++)
6537     for (int k = 0; k < K; k++)
6538         h_b_t[n * K + k] = h_b[k * N + n];
6539
6540 half *d_a, *d_b, *d_b_t, *d_c;
6541 check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
6542 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
6543 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
6544 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
6545
6546 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
6547 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
6548 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
6549
6550 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
6551 cublasHandle_t handle;
6552 cublasCreate(&handle);
6553 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6554 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
6555             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
6556             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
6557 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
6558
6559 // CuTe kernel (Stages=2, TN layout)
6560 launch_hgemm_mmma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
6561 check(cudaGetLastError(), "hgemm_cute launch");
6562 check(cudaDeviceSynchronize(), "hgemm_cute sync");
6563
6564 half *h_c = (half *)malloc(size_c);
6565 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
6566
6567 // Verify
6568 float max_err = 0.0f;
6569 for (int i = 0; i < M * N; i++) {
6570     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6571     if (err > max_err) max_err = err;
6572 }
6573 printf("| %-56s | %.6e | %-4s | %-22s |\n", "HGEMM CuTe Swizzle + Reg2x",
6574        max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
6575
6576 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
6577 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
6578 cublasDestroy(handle);
6579 }
6580 #endif /* NOTES_V2_ENABLE_CUTE */
6581
6582
6583 #if defined(NOTES_V2_ENABLE_WGMMMA)
6584 static void test_hgemm_wgmma(int M, int N, int K) {
6585     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
6586     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
6587     // Kernel: hgemm_wgmma_stages_tn with default template params
6588
6589     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
6590
6591     // M, K must be divisible by tile dims
6592     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6593         printf("| %-56s | %-12s | %-4s | %-22s |\n",
6594                "HGEMM WGMMMA", "SKIP", "SKIP", "None");
6595         return;

```

```

6596 }
6597
6598 size_t size_a = (size_t)M * K * sizeof(half);
6599 size_t size_b = (size_t)K * N * sizeof(half);
6600 size_t size_c = (size_t)M * N * sizeof(half);
6601
6602 half *h_a = (half *)malloc(size_a);
6603 half *h_b = (half *)malloc(size_b);
6604 half *h_c_ref = (half *)malloc(size_c);
6605
6606 srand(42);
6607 for (int i = 0; i < M * K; i++)
6608     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6609 for (int i = 0; i < K * N; i++)
6610     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6611
6612 // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
6613 size_t size_b_t = (size_t)N * K * sizeof(half);
6614 half *h_b_t = (half *)malloc(size_b_t);
6615 for (int n = 0; n < N; n++)
6616     for (int k = 0; k < K; k++)
6617         h_b_t[n * K + k] = h_b[k * N + n];
6618
6619 half *d_a, *d_b, *d_b_t, *d_c;
6620 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
6621 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
6622 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
6623 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
6624
6625 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
6626 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
6627         "wgmma H2D B (cuBLAS)");
6628 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
6629         "wgmma H2D B_t (kernel)");
6630
6631 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
6632 cublasHandle_t handle;
6633 cublasCreate(&handle);
6634 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6635 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
6636             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
6637             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
6638 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
6639         "wgmma D2H ref");
6640
6641 // Create TMA tensor maps for A and B^T
6642 // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
6643 // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
6644 CUsensorMap *tma_a =
6645     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6646 CUsensorMap *tma_b =
6647     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6648
6649 // Launch WGMMMA kernel
6650 // kBlockSwizzle=false → 2D grid, no swizzle
6651 size_t smem_bytes =
6652     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
6653 cudaFuncSetAttribute(
6654     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
6655     false>,
6656     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6657
6658 dim3 block(kNumThreads);

```

```

6659 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
6660 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
6661 <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6662 check(cudaGetLastError(), "wgmma launch");
6663 check(cudaDeviceSynchronize(), "wgmma sync");
6664
6665 half *h_c = (half *)malloc(size_c);
6666 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
6667
6668 // Verify
6669 float max_err = 0.0f;
6670 for (int i = 0; i < M * N; i++) {
6671     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6672     if (err > max_err)
6673         max_err = err;
6674 }
6675 printf("| %-5s | %.6e | %-4s | %-22s |\n", "HGEMM TMA WGMMMA WS (3-stage)", max_err,
6676        max_err < 1.0f ? "PASS" : "FAIL", "None");
6677
6678 free(h_a);
6679 free(h_b);
6680 free(h_b_t);
6681 free(h_c);
6682 free(h_c_ref);
6683 cudaFree(d_a);
6684 cudaFree(d_b);
6685 cudaFree(d_b_t);
6686 cudaFree(d_c);
6687 cudaFree(tma_a);
6688 cudaFree(tma_b);
6689 cublasDestroy(handle);
6690 }
6691 #endif /* NOTES_V2_ENABLE_WGMMMA */
6692
6693 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6694 template <int kStages, int kBlockSwizzle = 0>
6695 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
6696                                     half *d_b_t, half *d_c,
6697                                     CUTensorMap *tma_a, CUTensorMap *tma_b) {
6698     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
6699     constexpr int kMmaTileM = 2, kMmaTileN = 2;
6700     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
6701     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
6702     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
6703     constexpr int BK = kMmaK * kValTileK;
6704     constexpr int kNumThreads = 256;
6705     constexpr size_t payload_bytes =
6706         kStages * (BM * BK + BN * BK) * sizeof(half);
6707     constexpr size_t smem_bytes = payload_bytes;
6708     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6709                             const CUTensorMap *);
6710     Kernel kernel = hgemm_tma_mma_ws_tn<
6711         kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
6712         kValTileK, kStages, kNumThreads, kBlockSwizzle>;
6713
6714     int device = 0;
6715     int max_smem = 0;
6716     cudaFuncAttributes attributes{};
6717     check(cudaGetDevice(&device), "tma_mma_ws get device");
6718     check(cudaDeviceGetAttribute(&max_smem,
6719                                 cudaDevAttrMaxSharedMemoryPerBlockOptin,
6720                                 device),
6721           "tma_mma_ws max shared memory");

```

```

6722 check(cudaFuncGetAttributes(&attributes, kernel),
6723         "tma_mma_ws function attributes");
6724 if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
6725     printf("| %-56s | %-12s | %-4s | %-22s |\n",
6726           kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
6727                       : "HGEMM TMA MMA WS (S=3, SW=0)",
6728           "SMEM SKIP", "SKIP", "None");
6729     return false;
6730 }
6731
6732 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
6733                             smem_bytes),
6734         "tma_mma_ws set dynamic shared memory");
6735 dim3 block(kNumThreads);
6736 constexpr int kSwizzleN = 16;
6737 const int n_tiles = N / BN;
6738 const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
6739 const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
6740 dim3 grid(grid_x, M / BM, grid_z);
6741 kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6742 check(cudaGetLastError(), "tma_mma_ws launch");
6743 check(cudaDeviceSynchronize(), "tma_mma_ws sync");
6744 return true;
6745 }
6746
6747 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
6748     constexpr int BM = 128, BN = 128, BK = 64;
6749     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6750         printf("| %-56s | %-12s | %-4s | %-22s |\n", "HGEMM TMA MMA WS", "SKIP",
6751             "SKIP", "None");
6752         return;
6753     }
6754
6755     const size_t size_a = size_t(M) * K * sizeof(half);
6756     const size_t size_b = size_t(K) * N * sizeof(half);
6757     const size_t size_b_t = size_t(N) * K * sizeof(half);
6758     const size_t size_c = size_t(M) * N * sizeof(half);
6759     half *h_a = static_cast<half *>(malloc(size_a));
6760     half *h_b = static_cast<half *>(malloc(size_b));
6761     half *h_b_t = static_cast<half *>(malloc(size_b_t));
6762     half *h_c = static_cast<half *>(malloc(size_c));
6763     half *h_c_ref = static_cast<half *>(malloc(size_c));
6764     srand(42);
6765     for (int i = 0; i < M * K; ++i)
6766         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6767     for (int i = 0; i < K * N; ++i)
6768         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6769     for (int n = 0; n < N; ++n)
6770         for (int k = 0; k < K; ++k)
6771             h_b_t[n * K + k] = h_b[k * N + n];
6772
6773     half *d_a, *d_b, *d_b_t, *d_c;
6774     check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
6775     check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
6776     check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
6777     check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
6778     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
6779     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
6780     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
6781           "tma_mma_ws H2D B_t");
6782
6783     cublasHandle_t handle;
6784     cublasCreate(&handle);

```

```

6785 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
6786 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
6787             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
6788             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
6789 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
6790       "tma_mma_ws D2H reference");
6791
6792 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6793 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6794 for (int block_swizzle : {0, 1}) {
6795     for (int stages : {2, 3}) {
6796         const bool launched = stages == 2
6797             ? (block_swizzle
6798                ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
6799                                                  tma_a, tma_b)
6800                : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
6801                                              tma_a, tma_b))
6802             : (block_swizzle
6803                ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
6804                                                  tma_a, tma_b)
6805                : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
6806                                              tma_a, tma_b));
6807         if (!launched)
6808             continue;
6809         check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
6810               "tma_mma_ws D2H");
6811         float max_err = 0.0f;
6812         for (int i = 0; i < M * N; ++i)
6813             max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
6814                                                  __half2float(h_c_ref[i])));
6815         printf("| %-56s | %.6e | %-4s | %-22s |\n",
6816               block_swizzle
6817                 ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
6818                               : "HGEMM TMA MMA WS (S=3, SW=1)")
6819                 : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
6820                               : "HGEMM TMA MMA WS (S=3, SW=0)"),
6821               max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
6822     }
6823 }
6824
6825 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
6826 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
6827 cudaFree(tma_a); cudaFree(tma_b);
6828 cublasDestroy(handle);
6829 }
6830 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
6831
6832
6833 static void test_flash_attn(int seqlen, int head_dim) {
6834     // FlashAttention-2 with split-Q, MMA m16n8k16
6835     int B = 1, H = 8;
6836
6837     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6838
6839     srand(42);
6840     half *h_q = (half *)malloc(sz);
6841     half *h_k = (half *)malloc(sz);
6842     half *h_v = (half *)malloc(sz);
6843     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
6844         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6845         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6846         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6847     }

```

```

6848 // CPU reference in FP32:  $O = \text{softmax}(Q @ K^T / \sqrt{d}) @ V$ 
6849 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
6850 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
6851 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
6852 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
6853 int count = B * H * seqlen * head_dim;
6854 for (int i = 0; i < count; i++) {
6855     ref_q[i] = __half2float(h_q[i]);
6856     ref_k[i] = __half2float(h_k[i]);
6857     ref_v[i] = __half2float(h_v[i]);
6858 }
6859
6860
6861 float scale = 1.0f / sqrtf((float)head_dim);
6862 for (int bi = 0; bi < B * H; bi++) {
6863     for (int qi = 0; qi < seqlen; qi++) {
6864         //  $S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale$ 
6865         float smax = -INFINITY;
6866         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
6867         for (int kj = 0; kj < seqlen; kj++) {
6868             float s = 0.0f;
6869             for (int d = 0; d < head_dim; d++)
6870                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
6871                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
6872             S[kj] = s * scale;
6873             if (S[kj] > smax) smax = S[kj];
6874         }
6875         // softmax
6876         double sum_exp = 0.0;
6877         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
6878         float inv_sum = 1.0f / (float)sum_exp;
6879         //  $O[qi, :] = \sum_{kj} P[qi, kj] * V[kj, :]$ 
6880         for (int d = 0; d < head_dim; d++) {
6881             double o_acc = 0.0;
6882             for (int kj = 0; kj < seqlen; kj++)
6883                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
6884                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
6885             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
6886         }
6887         free(S);
6888     }
6889 }
6890
6891 half *d_q, *d_k, *d_v, *d_o;
6892 check(cudaMalloc(&d_q, sz), "fa alloc Q");
6893 check(cudaMalloc(&d_k, sz), "fa alloc K");
6894 check(cudaMalloc(&d_v, sz), "fa alloc V");
6895 check(cudaMalloc(&d_o, sz), "fa alloc O");
6896 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
6897 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
6898 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
6899
6900 // Template params for kHeadDim=64, kStagesK=2
6901 constexpr int kHeadDim = 64;
6902 constexpr int kStagesK = 2;
6903 constexpr int kPadQ = 8;
6904 constexpr int kPadK = 8;
6905 constexpr int kPadV = 8;
6906 constexpr int kMmaAtomM = 16;
6907 constexpr int kMmaAtomN = 8;
6908 constexpr int kMmaAtomK = 16;
6909 constexpr int kMmaTileSeqLenQ = 8;
6910 constexpr int kMmaTileSeqLenK = 1;

```



```

6911 constexpr int kMmaTileSeqLenP = 8;
6912 constexpr int kMmaTileHeadDimV = 1;
6913 constexpr int kValTileSeqLenQ = 1;
6914 constexpr int kValTileSeqLenK = 8;
6915 constexpr int kValTileSeqLenP = 1;
6916 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
6917
6918 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
6919 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
6920 if (seqlen < Br) return; // kernel requires seqlen >= tile size
6921 size_t smem_bytes =
6922     (Br * (kHeadDim + kPadQ) +
6923      kStagesK * Bc * (kHeadDim + kPadK) +
6924      Bc * (kHeadDim + kPadV)) * sizeof(half);
6925
6926 dim3 block(256);
6927 dim3 grid((seqlen + Br - 1) / Br, B * H);
6928
6929 // Test both accumulator variants: f16 (kMmaAccF32=0) and f32 (kMmaAccF32=1)
6930 for (int acc = 0; acc <= 1; ++acc) {
6931     const int kMmaAcc = acc;
6932     half *h_o = (half *)malloc(sz);
6933
6934     if (kMmaAcc == 0) {
6935         using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6936         FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
6937             kMmaAtomK, 0, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
6938             kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
6939             kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
6940         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6941         fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
6942     } else {
6943         using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6944         FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
6945             kMmaAtomK, 1, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
6946             kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
6947             kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
6948         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6949         fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
6950     }
6951     check(cudaGetLastError(), "fa launch");
6952     check(cudaDeviceSynchronize(), "fa sync");
6953
6954     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
6955
6956     float max_err = 0.0f;
6957     for (int i = 0; i < count; i++) {
6958         float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
6959         if (err > max_err) max_err = err;
6960     }
6961     const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
6962     char label[64];
6963     snprintf(label, sizeof(label), "FlashAttention-2 (kStagesK=2, Pad, %s)", acc_label);
6964     printf("| %-56s | %.6e | %-4s | %-22s |\n", label,
6965         max_err, max_err < 5e-1f ? "PASS" : "FAIL", "None");
6966     free(h_o);
6967 }
6968
6969 free(h_q); free(h_k); free(h_v);
6970 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
6971 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
6972 }
6973

```

```

6974
6975 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6976 // Test for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
6977 // Reference: cuDNN SDPA (half) if available, else CPU FP32 (float) fallback.
6978 template <int kHeadDim>
6979 static void test_flash_attn_tma_mma_ws_impl(int seqlen, int head_dim) {
6980     int B = 1, H = 8;
6981     constexpr int kStagesK = 2;
6982     constexpr int kStagesV = 1;
6983     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
6984     constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;
6985     constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
6986     constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
6987     constexpr int kValTileSeqLenP = 1;
6988     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
6989     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 128
6990     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
6991     constexpr int kNumThreads = 384;
6992
6993     if (seqlen % Br != 0 || seqlen % Bc != 0 || seqlen < Br) {
6994         printf("| %-56s | %-12s | %-4s | %-22s |\n",
6995             "FlashAttn-2 TMA MMA WS (unaligned)", "SKIP", "SKIP", "None");
6996         return;
6997     }
6998
6999     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
7000     srand(42);
7001     half *h_q = (half *)malloc(sz);
7002     half *h_k = (half *)malloc(sz);
7003     half *h_v = (half *)malloc(sz);
7004     for (int i = 0; i < B * H * seqlen * head_dim; ++i) {
7005         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7006         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7007         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7008     }
7009
7010     half *d_q, *d_k, *d_v, *d_o;
7011     check(cudaMalloc(&d_q, sz), "fa_tma_ws alloc Q");
7012     check(cudaMalloc(&d_k, sz), "fa_tma_ws alloc K");
7013     check(cudaMalloc(&d_v, sz), "fa_tma_ws alloc V");
7014     check(cudaMalloc(&d_o, sz), "fa_tma_ws alloc O");
7015     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D Q");
7016     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D K");
7017     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D V");
7018
7019     // Reference output: either cuDNN (half) or CPU (float)
7020     half *h_o_ref = nullptr;
7021     float *ref_o = nullptr;
7022     int count = B * H * seqlen * head_dim;
7023
7024     #if defined(NOTES_V2_ENABLE_CUDNN)
7025     {
7026         // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
7027         bool cudnn_ok = false;
7028         half *d_o_ref;
7029         check(cudaMalloc(&d_o_ref, sz), "fa_tma_ws alloc O_ref");
7030         {
7031             cudnnHandle_t cudnn_handle;
7032             cudnnCreate(&cudnn_handle);
7033             auto graph = std::make_shared<fe::graph::Graph>();
7034             graph->set_io_data_type(fe::DataType_t::HALF)
7035                 .set_intermediate_data_type(fe::DataType_t::FLOAT)
7036                 .set_compute_data_type(fe::DataType_t::FLOAT);

```

```

7037
7038     auto Q = graph->tensor(fe::graph::Tensor_attributes()
7039         .set_uid(1).set_dim({B, H, seqlen, head_dim})
7040         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
7041     auto K = graph->tensor(fe::graph::Tensor_attributes()
7042         .set_uid(2).set_dim({B, H, seqlen, head_dim})
7043         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
7044     auto V = graph->tensor(fe::graph::Tensor_attributes()
7045         .set_uid(3).set_dim({B, H, seqlen, head_dim})
7046         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
7047
7048     auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
7049         fe::graph::SDPA_attributes()
7050             .set_name("sdpa_ref")
7051             .set_attn_scale(1.0f / sqrtf((float)head_dim)));
7052
7053     O_sdpa->set_output(true).set_uid(4)
7054         .set_dim({B, H, seqlen, head_dim})
7055         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
7056
7057     auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
7058     if (build_status.is_good()) {
7059         std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
7060             {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
7061         int64_t ws_size = 0;
7062         if (graph->get_workspace_size(ws_size).is_good()) {
7063             int8_t *d_ws = nullptr;
7064             if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "fa_tma_ws workspace");
7065             if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
7066                 check(cudaDeviceSynchronize(), "fa_tma_ws cudnn sync");
7067                 cudnn_ok = true;
7068             }
7069             if (d_ws) cudaFree(d_ws);
7070         }
7071     }
7072     cudnnDestroy(cudnn_handle);
7073 }
7074
7075 if (cudnn_ok) {
7076     h_o_ref = (half *)malloc(sz);
7077     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H ref");
7078 } else {
7079     fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
7080 }
7081 cudaFree(d_o_ref);
7082 }
7083 #endif
7084
7085 // CPU reference fallback
7086 if (!h_o_ref) {
7087     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
7088     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
7089     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
7090     ref_o = (float *)malloc(sz * 4 / sizeof(half));
7091     for (int i = 0; i < count; ++i) {
7092         ref_q[i] = __half2float(h_q[i]);
7093         ref_k[i] = __half2float(h_k[i]);
7094         ref_v[i] = __half2float(h_v[i]);
7095     }
7096     float scale = 1.0f / sqrtf((float)head_dim);
7097     for (int bi = 0; bi < B * H; ++bi) {
7098         for (int qi = 0; qi < seqlen; ++qi) {
7099             float smax = -INFINITY;

```

```

7100     float *S = (float *)malloc((size_t)seqlen * sizeof(float));
7101     for (int kj = 0; kj < seqlen; ++kj) {
7102         float s = 0.0f;
7103         for (int d = 0; d < head_dim; ++d)
7104             s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
7105                 ref_k[bi * seqlen * head_dim + kj * head_dim + d];
7106         S[kj] = s * scale;
7107         if (S[kj] > smax) smax = S[kj];
7108     }
7109     double sum_exp = 0.0;
7110     for (int kj = 0; kj < seqlen; ++kj)
7111         sum_exp += (double)expf(S[kj] - smax);
7112     float inv_sum = 1.0f / (float)sum_exp;
7113     for (int d = 0; d < head_dim; ++d) {
7114         double o_acc = 0.0;
7115         for (int kj = 0; kj < seqlen; ++kj)
7116             o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
7117                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
7118         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
7119     }
7120     free(S);
7121 }
7122 }
7123 free(ref_q);
7124 free(ref_k);
7125 free(ref_v);
7126 }
7127
7128 // TMA descriptors: box innermost 固定 64 half (128B), 满足
7129 // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
7130 // D=64: blocks_width=1, 单次 TMA 覆盖整行。
7131 // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA,
7132 //         minor_coord = 0/64, 写入 chunk-major smem 布局 [2, Br, 64]。
7133 // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major, 作为 2D
7134 // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
7135 // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
7136 constexpr int kTmaBoxMinor = 64; // box innermost = 64 half = 128B
7137 constexpr int kTmaChunksQ = kHeadDim / kTmaBoxMinor;
7138 constexpr int kTmaChunksKV = kHeadDim / kTmaBoxMinor;
7139 CUTensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
7140     d_q, B * H * seqlen / Br, kTmaChunksQ);
7141 CUTensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
7142     d_k, B * H * seqlen / Bc, kTmaChunksKV);
7143 CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
7144     d_v, B * H * seqlen / Bc, kTmaChunksKV);
7145
7146 using FAKernel = void (*)(half *, half *, half *, half *, int, int,
7147     const CUTensorMap *, const CUTensorMap *,
7148     const CUTensorMap *);
7149 FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
7150     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
7151     kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
7152     kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
7153     kNumThreads>;
7154
7155 // smem = Q[Br*d] + K[kStagesK*Bc*d] + V[kStagesV*Bc*d]
7156 size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
7157     kStagesV * Bc * kHeadDim) * sizeof(half);
7158 int device = 0, max_smem = 0;
7159 cudaFuncAttributes attributes{};
7160 cudaGetDevice(&device);
7161 cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin,
7162     device);

```

```

7163 cudaFuncGetAttributes(&attributes, fa_k);
7164 bool smem_ok =
7165     (smem_bytes + attributes.sharedSizeBytes <= (size_t)max_smem);
7166 if (!smem_ok) {
7167     printf("| %-56s | %-12s | %-4s | %-22s |\n",
7168         "FlashAttn-2 TMA MMA WS (D=64)", "SMEM SKIP", "SKIP", "None");
7169 } else {
7170     dim3 block(kNumThreads);
7171     dim3 grid((seqlen + Br - 1) / Br, B * H);
7172     // Test both accumulator variants
7173     for (int acc = 0; acc <= 1; ++acc) {
7174         const int kMmaAcc = acc;
7175         half *h_o = (half *)malloc(sz);
7176
7177         if (kMmaAcc == 0) {
7178             using FAK = void (*)(half *, half *, half *, half *, int, int,
7179                 const CUTensorMap *, const CUTensorMap *,
7180                 const CUTensorMap *);
7181             FAK fk = flash_attn_tma_mma_ws_stages_split_q<
7182                 kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
7183                 kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
7184                 kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
7185                 kStagesV, kNumThreads>;
7186             cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
7187                 smem_bytes);
7188             fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
7189                 tma_k, tma_v);
7190         } else {
7191             using FAK = void (*)(half *, half *, half *, half *, int, int,
7192                 const CUTensorMap *, const CUTensorMap *,
7193                 const CUTensorMap *);
7194             FAK fk = flash_attn_tma_mma_ws_stages_split_q<
7195                 kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 1, kMmaTileSeqLenQ,
7196                 kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
7197                 kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
7198                 kStagesV, kNumThreads>;
7199             cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
7200                 smem_bytes);
7201             fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
7202                 tma_k, tma_v);
7203         }
7204         check(cudaGetLastError(), "fa_tma_ws launch");
7205         check(cudaDeviceSynchronize(), "fa_tma_ws sync");
7206
7207         check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
7208         float max_err = 0.0f;
7209         bool checked = h_o_ref || ref_o;
7210         if (checked) {
7211             for (int i = 0; i < count; ++i) {
7212                 float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
7213                 float err = fabsf(__half2float(h_o[i]) - ref_val);
7214                 if (err > max_err) max_err = err;
7215             }
7216         }
7217         const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
7218         char label[64];
7219         snprintf(label, sizeof(label),
7220             "FlashAttention-2 TMA MMA WS (D=%d, %s)", (int)kHeadDim,
7221             acc_label);
7222         printf("| %-56s | %.6e | %-4s | %-22s |\n",
7223             label, max_err,
7224             (checked && max_err < 5e-1f) ? "PASS" : (checked ? "FAIL" : "SKIP"),
7225             "None");

```

```

7226     free(h_o);
7227 }
7228 }
7229
7230 free(h_q); free(h_k); free(h_v);
7231 free(h_o_ref); free(ref_o);
7232 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
7233 cudaFree(tma_q); cudaFree(tma_k); cudaFree(tma_v);
7234 }
7235
7236 // D=64/128 dispatch wrapper
7237 static void test_flash_attn_tma_mma_ws(int seqlen, int head_dim) {
7238     if (head_dim == 64) {
7239         test_flash_attn_tma_mma_ws_impl<64>(seqlen, head_dim);
7240     } else if (head_dim == 128) {
7241         test_flash_attn_tma_mma_ws_impl<128>(seqlen, head_dim);
7242     } else {
7243         printf("| %-56s | %-12s | %-4s | %-22s |\n",
7244             "FlashAttn-2 TMA MMA WS (D!=64/128)", "SKIP", "SKIP", "None");
7245     }
7246 }
7247 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
7248
7249
7250 // Bench mode globals and helpers
7251 static bool g_bench_hgemm = false;
7252 static bool g_bench_hgemm_all = false;
7253 static bool g_bench_fa = false;
7254 static bool g_bench_all = false;
7255 static bool g_fa_skip_check = false;
7256 static bool g_swizzle_eq_check = false;
7257 enum class FALayout {All, Pad, SwizzleQ, SwizzleK, SwizzleV, SwizzleQK, SwizzleQV, SwizzleKV, Swizzle};
7258 static FALayout g_fa_layout = FALayout::Pad;
7259 static int g_bench_M = 8192, g_bench_N = 8192, g_bench_K = 8192;
7260 static int g_bench_B = 1, g_bench_H = 48, g_bench_Nfa = 8192, g_bench_D = 64;
7261 static int g_warmup = 3, g_repeat = 5;
7262
7263 static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
7264     double flops = 2.0 * M * N * K;
7265     return (float)(flops / (double)time_ms / 1e9);
7266 }
7267
7268 static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
7269     double flops = 4.0 * B * H * N * N * D;
7270     return (float)(flops / (double)time_ms / 1e9);
7271 }
7272
7273 static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
7274     int device = 0;
7275     int max_smem = 0;
7276     cudaFuncAttributes attrs{};
7277     cudaGetDevice(&device);
7278     cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
7279     cudaFuncGetAttributes(&attrs, kernel_func);
7280     return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
7281 }
7282
7283 static float bench_cublas_hgemm_tflops(cublasHandle_t handle, int M, int N, int K,
7284     half *d_a, half *d_b, half *d_c) {
7285     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
7286     cudaStream_t stream;
7287     cudaStreamCreate(&stream);
7288     cublasSetStream(handle, stream);

```



```

7289 // warmup
7290 for (int w = 0; w < g_warmup; ++w)
7291     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
7292                 d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
7293                 d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
7294 cudaStreamSynchronize(stream);
7295 // timed repeat
7296 cudaEvent_t start, stop;
7297 cudaEventCreate(&start);
7298 cudaEventCreate(&stop);
7299 cudaEventRecord(start, stream);
7300 for (int r = 0; r < g_repeat; ++r)
7301     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
7302                 d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
7303                 d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
7304 cudaEventRecord(stop, stream);
7305 cudaEventSynchronize(stop);
7306 float time_ms = 0;
7307 cudaEventElapsedTime(&time_ms, start, stop);
7308 time_ms /= g_repeat;
7309 cudaEventDestroy(start);
7310 cudaEventDestroy(stop);
7311 cublasSetStream(handle, nullptr);
7312 cudaStreamDestroy(stream);
7313 return bench_hgemm_tflops(M, N, K, time_ms);
7314 }
7315
7316 // =====
7317 // Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
7318 // =====
7319 template <int kStages, int kBlockSwizzle>
7320 static bool launch_timed_hgemm_mma(
7321     half *d_a, half *d_b_t, half *d_c, half *h_c,
7322     half *h_c_ref, int M, int N, int K, size_t size_c,
7323     cudaEvent_t start, cudaEvent_t stop, float &max_err,
7324     float &time_ms
7325 ) {
7326     constexpr int BM = 128, BN = 128, BK = 16;
7327     using Kernel = void (*)(half *, half *, half *, int, int, int);
7328     Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
7329     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
7330     if (!check_smem_feasible((const void *)k, smem)) return false;
7331     dim3 block(256);
7332     const int tiles_n = (N + BN - 1) / BN;
7333     constexpr int kSwizzleN = 16;
7334     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
7335     int gz = kBlockSwizzle ? kSwizzleN : 1;
7336     dim3 grid(gx, (M + BM - 1) / BM, gz);
7337     for (int w = 0; w < g_warmup; w++)
7338         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
7339     cudaDeviceSynchronize();
7340     cudaEventRecord(start);
7341     for (int r = 0; r < g_repeat; r++)
7342         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
7343     cudaEventRecord(stop);
7344     cudaEventSynchronize(stop);
7345     cudaEventElapsedTime(&time_ms, start, stop);
7346     time_ms /= g_repeat;
7347     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
7348     max_err = 0.0f;
7349     for (int i = 0; i < M * N; i++) {
7350         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
7351         if (err > max_err) max_err = err;

```

```

7352 }
7353 return true;
7354 }
7355
7356 static void bench_hgemm_mma(int M, int N, int K) {
7357     size_t size_a = (size_t)M * K * sizeof(half);
7358     size_t size_b = (size_t)K * N * sizeof(half);
7359     size_t size_c = (size_t)M * N * sizeof(half);
7360     half *h_a = (half *)malloc(size_a);
7361     half *h_b = (half *)malloc(size_b);
7362     half *h_c_ref = (half *)malloc(size_c);
7363     srand(42);
7364     for (int i = 0; i < M * K; i++)
7365         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7366     for (int i = 0; i < K * N; i++)
7367         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7368     size_t size_b_t = (size_t)N * K * sizeof(half);
7369     half *h_b_t = (half *)malloc(size_b_t);
7370     for (int n = 0; n < N; n++)
7371         for (int k = 0; k < K; k++)
7372             h_b_t[n * K + k] = h_b[k * N + n];
7373     half *d_a, *d_b, *d_b_t, *d_c;
7374     check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
7375     check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
7376     check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
7377     check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
7378     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
7379     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
7380     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
7381     cublasHandle_t handle;
7382     cublasCreate(&handle);
7383     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
7384     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
7385                 d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
7386                 CUBLAS_GEMM_DEFAULT);
7387     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
7388     half *h_c = (half *)malloc(size_c);
7389     float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
7390     cudaEvent_t start, stop;
7391     cudaEventCreate(&start);
7392     cudaEventCreate(&stop);
7393     for (int stages : {2, 3}) {
7394         for (int swizzle : {0, 1}) {
7395             float max_err = 0, time_ms = 0;
7396             bool ok = false;
7397             if (stages == 2)
7398                 ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
7399                     K, size_c, start, stop, max_err, time_ms)
7400                     : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
7401                     K, size_c, start, stop, max_err, time_ms);
7402             else
7403                 ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
7404                     K, size_c, start, stop, max_err, time_ms)
7405                     : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
7406                     K, size_c, start, stop, max_err, time_ms);
7407             char label[64];
7408             snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
7409             if (!ok) {
7410                 printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM SKIP", "SKIP", "None");
7411             } else {
7412                 float tflops = bench_hgemm_tflops(M, N, K, time_ms);
7413                 char tflops_str[32];
7414                 snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /

```

```

    cublas_tflops);
7415     printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
7416           max_err < 1.0f ? "PASS" : "FAIL", tflops_str);
7417 }
7418 }
7419 }
7420 cudaEventDestroy(start);
7421 cudaEventDestroy(stop);
7422 free(h_a);
7423 free(h_b);
7424 free(h_b_t);
7425 free(h_c);
7426 free(h_c_ref);
7427 cudaFree(d_a);
7428 cudaFree(d_b);
7429 cudaFree(d_b_t);
7430 cudaFree(d_c);
7431 cublasDestroy(handle);
7432 }
7433
7434 // =====
7435 // Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=4, BK=64)
7436 // =====
7437 template <int kStages, int kBlockSwizzle>
7438 static bool launch_timed_hgemm_swizzle(
7439     half *d_a, half *d_b_t, half *d_c, half *h_c,
7440     half *h_c_ref, int M, int N, int K, size_t size_c,
7441     cudaEvent_t start, cudaEvent_t stop, float &max_err,
7442     float &time_ms
7443 ) {
7444     constexpr int BM = 128, BN = 128, BK = 64;
7445     using Kernel = void (*)(half *, half *, half *, int, int, int);
7446     Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, kStages, kBlockSwizzle>;
7447     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
7448     if (!check_smem_feasible((const void *)k, smem)) return false;
7449     cudaFuncSetAttribute((const void *)k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
7450     dim3 block(256);
7451     const int tiles_n = (N + BN - 1) / BN;
7452     constexpr int kSwizzleN = 16;
7453     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
7454     int gz = kBlockSwizzle ? kSwizzleN : 1;
7455     dim3 grid(gx, (M + BM - 1) / BM, gz);
7456     for (int w = 0; w < g_warmup; w++)
7457         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
7458     cudaDeviceSynchronize();
7459     cudaEventRecord(start);
7460     for (int r = 0; r < g_repeat; r++)
7461         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
7462     cudaEventRecord(stop);
7463     cudaEventSynchronize(stop);
7464     cudaEventElapsedTime(&time_ms, start, stop);
7465     time_ms /= g_repeat;
7466     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
7467     max_err = 0.0f;
7468     for (int i = 0; i < M * N; i++) {
7469         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
7470         if (err > max_err) max_err = err;
7471     }
7472     return true;
7473 }
7474
7475 static void bench_hgemm_swizzle(int M, int N, int K) {
7476     size_t size_a = (size_t)M * K * sizeof(half);

```

```

7477 size_t size_b = (size_t)K * N * sizeof(half);
7478 size_t size_c = (size_t)M * N * sizeof(half);
7479 half *h_a = (half *)malloc(size_a);
7480 half *h_b = (half *)malloc(size_b);
7481 half *h_c_ref = (half *)malloc(size_c);
7482 srand(42);
7483 for (int i = 0; i < M * K; i++)
7484     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7485 for (int i = 0; i < K * N; i++)
7486     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7487 size_t size_b_t = (size_t)N * K * sizeof(half);
7488 half *h_b_t = (half *)malloc(size_b_t);
7489 for (int n = 0; n < N; n++)
7490     for (int k = 0; k < K; k++)
7491         h_b_t[n * K + k] = h_b[k * N + n];
7492 half *d_a, *d_b, *d_b_t, *d_c;
7493 check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
7494 check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
7495 check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
7496 check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
7497 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
7498 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
7499 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
7500 cublasHandle_t handle;
7501 cublasCreate(&handle);
7502 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
7503 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
7504     d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
7505     CUBLAS_GEMM_DEFAULT);
7506 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
7507 half *h_c = (half *)malloc(size_c);
7508 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
7509 cudaEvent_t start, stop;
7510 cudaEventCreate(&start);
7511 cudaEventCreate(&stop);
7512 for (int stages : {2, 3}) {
7513     for (int swizzle : {0, 1}) {
7514         float max_err = 0, time_ms = 0;
7515         bool ok = false;
7516         if (stages == 2)
7517             ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
7518                 N, K, size_c, start, stop, max_err, time_ms)
7519                 : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
7520                 N, K, size_c, start, stop, max_err, time_ms);
7521         else
7522             ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
7523                 N, K, size_c, start, stop, max_err, time_ms)
7524                 : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
7525                 N, K, size_c, start, stop, max_err, time_ms);
7526         char label[64];
7527         snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
7528         if (!ok) {
7529             printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM SKIP", "SKIP", "None");
7530         } else {
7531             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
7532             char tflops_str[32];
7533             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
7534                 cublas_tflops);
7535             printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
7536                 max_err < 1.0f ? "PASS" : "FAIL", tflops_str);
7537         }
7538     }

```

```

7539 cudaEventDestroy(start);
7540 cudaEventDestroy(stop);
7541 free(h_a);
7542 free(h_b);
7543 free(h_b_t);
7544 free(h_c);
7545 free(h_c_ref);
7546 cudaFree(d_a);
7547 cudaFree(d_b);
7548 cudaFree(d_b_t);
7549 cudaFree(d_c);
7550 cublasDestroy(handle);
7551 }
7552
7553 #if defined(NOTES_V2_ENABLE_CUTE)
7554 // =====
7555 // Bench: HGEMM CuTe
7556 // =====
7557 static void bench_hgemm_cute(int M, int N, int K) {
7558     size_t size_a = (size_t)M * K * sizeof(half);
7559     size_t size_b = (size_t)K * N * sizeof(half);
7560     size_t size_c = (size_t)M * N * sizeof(half);
7561     half *h_a = (half *)malloc(size_a);
7562     half *h_b = (half *)malloc(size_b);
7563     half *h_c_ref = (half *)malloc(size_c);
7564     srand(42);
7565     for (int i = 0; i < M * K; i++)
7566         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7567     for (int i = 0; i < K * N; i++)
7568         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7569     size_t size_b_t = (size_t)N * K * sizeof(half);
7570     half *h_b_t = (half *)malloc(size_b_t);
7571     for (int n = 0; n < N; n++)
7572         for (int k = 0; k < K; k++)
7573             h_b_t[n * K + k] = h_b[k * N + n];
7574     half *d_a, *d_b, *d_b_t, *d_c;
7575     check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
7576     check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
7577     check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
7578     check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
7579     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
7580     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
7581     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
7582     cublasHandle_t handle;
7583     cublasCreate(&handle);
7584     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
7585     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
7586         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
7587         CUBLAS_GEMM_DEFAULT);
7588     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
7589     half *h_c = (half *)malloc(size_c);
7590     float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
7591     cudaEvent_t start, stop;
7592     cudaEventCreate(&start);
7593     cudaEventCreate(&stop);
7594     for (int stages : {2, 3}) {
7595         for (int swizzle : {0, 1}) {
7596             char label[64];
7597             snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
7598             bool ok = false;
7599             float time_ms = 0, max_err = 0;
7600             if (stages == 2) {
7601                 auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 2, 1>

```

```

7602         : launch_hgemm_mma_stages_tn_cute<half, 2, 0>;
7603     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
7604     cudaDeviceSynchronize();
7605     cudaEventRecord(start);
7606     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
7607     cudaEventRecord(stop);
7608     cudaEventSynchronize(stop);
7609     cudaEventElapsedTime(&time_ms, start, stop);
7610     time_ms /= g_repeat;
7611     ok = true;
7612 } else {
7613     auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 3, 1>
7614         : launch_hgemm_mma_stages_tn_cute<half, 3, 0>;
7615     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
7616     cudaDeviceSynchronize();
7617     cudaEventRecord(start);
7618     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
7619     cudaEventRecord(stop);
7620     cudaEventSynchronize(stop);
7621     cudaEventElapsedTime(&time_ms, start, stop);
7622     time_ms /= g_repeat;
7623     ok = true;
7624 }
7625 if (ok) {
7626     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
7627     max_err = 0.0f;
7628     for (int i = 0; i < M * N; i++) {
7629         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
7630         if (err > max_err) max_err = err;
7631     }
7632     float tflops = bench_hgemm_tflops(M, N, K, time_ms);
7633     char tflops_str[32];
7634     snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
7635 cublas_tflops);
7636     printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
7637 max_err < 1.0f ? "PASS" : "FAIL", tflops_str);
7638 } else {
7639     printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "LAUNCH ERR", "FAIL", "None");
7640 }
7641 }
7642 cudaEventDestroy(start);
7643 cudaEventDestroy(stop);
7644 free(h_a);
7645 free(h_b);
7646 free(h_b_t);
7647 free(h_c);
7648 free(h_c_ref);
7649 cudaFree(d_a);
7650 cudaFree(d_b);
7651 cudaFree(d_b_t);
7652 cudaFree(d_c);
7653 cublasDestroy(handle);
7654 }
7655 #endif
7656
7657 #if defined(NOTES_V2_ENABLE_WGMMA)
7658 // =====
7659 // Bench: HGEMM WGMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
7660 // =====
7661 template <int kStages, int kBlockSwizzle>
7662 static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
7663 CUtensorMap *tma_a, CUtensorMap *tma_b,

```



```

7664         int M, int N, int K, size_t size_c,
7665         cudaEvent_t start, cudaEvent_t stop,
7666         float &max_err, float &time_ms) {
7667     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
7668     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
7669                             const CUTensorMap *);
7670     Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
7671               kBlockSwizzle>;
7672     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
7673     if (!check_smem_feasible((const void *)k, smem)) return false;
7674     cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
7675     dim3 block(kNumThreads);
7676     const int tiles_n = (N + BN - 1) / BN;
7677     constexpr int kSwizzleN = 16;
7678     int gx = kBlockSwizzle ? div_cceil(tiles_n, kSwizzleN) : tiles_n;
7679     int gz = kBlockSwizzle ? kSwizzleN : 1;
7680     dim3 grid(gx, (M + BM - 1) / BM, gz);
7681     for (int w = 0; w < g_warmup; w++)
7682         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
7683     cudaDeviceSynchronize();
7684     cudaEventRecord(start);
7685     for (int r = 0; r < g_repeat; r++)
7686         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
7687     cudaEventRecord(stop);
7688     cudaEventSynchronize(stop);
7689     cudaEventElapsedTime(&time_ms, start, stop);
7690     time_ms /= g_repeat;
7691     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
7692     max_err = 0.0f;
7693     for (int i = 0; i < M * N; i++) {
7694         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
7695         if (err > max_err) max_err = err;
7696     }
7697     return true;
7698 }
7699
7700 static void bench_hgemm_wgmma(int M, int N, int K) {
7701     constexpr int BM = 128, BN = 128, BK = 64;
7702     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
7703         printf("| %-56s | %-12s | %-4s | %-22s |\n", "HGEMM WGMMMA (unaligned)", "SKIP", "SKIP",
7704             "None");
7705         return;
7706     }
7707     size_t size_a = (size_t)M * K * sizeof(half);
7708     size_t size_b = (size_t)K * N * sizeof(half);
7709     size_t size_c = (size_t)M * N * sizeof(half);
7710     half *h_a = (half *)malloc(size_a);
7711     half *h_b = (half *)malloc(size_b);
7712     half *h_c_ref = (half *)malloc(size_c);
7713     srand(42);
7714     for (int i = 0; i < M * K; i++)
7715         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7716     for (int i = 0; i < K * N; i++)
7717         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7718     size_t size_b_t = (size_t)N * K * sizeof(half);
7719     half *h_b_t = (half *)malloc(size_b_t);
7720     for (int n = 0; n < N; n++)
7721         for (int k = 0; k < K; k++)
7722             h_b_t[n * K + k] = h_b[k * N + n];
7723     half *d_a, *d_b, *d_b_t, *d_c;
7724     check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
7725     check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
7726     check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");

```

```

7727 check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
7728 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
7729 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
7730 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
7731 cublasHandle_t handle;
7732 cublasCreate(&handle);
7733 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
7734 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
7735             d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
7736             CUBLAS_GEMM_DEFAULT);
7737 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
7738 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
7739 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
7740 half *h_c = (half *)malloc(size_c);
7741 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
7742 cudaEvent_t start, stop;
7743 cudaEventCreate(&start);
7744 cudaEventCreate(&stop);
7745 for (int stages : {1, 2, 3}) {
7746     for (int swizzle : {0, 1}) {
7747         float max_err = 0, time_ms = 0;
7748         bool ok = false;
7749         if (stages == 2)
7750             ok = swizzle
7751                 ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
7752                 K, size_c, start, stop, max_err, time_ms)
7753                 : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
7754                 size_c, start, stop, max_err, time_ms);
7755         else
7756             ok = swizzle
7757                 ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
7758                 K, size_c, start, stop, max_err, time_ms)
7759                 : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
7760                 size_c, start, stop, max_err, time_ms);
7761         char label[64];
7762         snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, SW=%d)", stages, swizzle);
7763         if (!ok) {
7764             printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM SKIP", "SKIP", "None");
7765         } else {
7766             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
7767             char tflops_str[32];
7768             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
7769             cublas_tflops);
7770             printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
7771             max_err < 1.0f ? "PASS" : "FAIL", tflops_str);
7772         }
7773     }
7774 }
7775 cudaEventDestroy(start);
7776 cudaEventDestroy(stop);
7777 free(h_a);
7778 free(h_b);
7779 free(h_b_t);
7780 free(h_c);
7781 free(h_c_ref);
7782 cudaFree(d_a);
7783 cudaFree(d_b);
7784 cudaFree(d_b_t);
7785 cudaFree(d_c);
7786 cudaFree(tma_a);
7787 cudaFree(tma_b);
7788 cublasDestroy(handle);

```

```

7789 #endif
7790
7791 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7792 // =====
7793 // Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
7794 // =====
7795 template <int kStages, int kBlockSwizzle>
7796 static bool bench_launch_tma_mma_ws(
7797     int M, int N, int K, half *d_a, half *d_b_t,
7798     half *d_c, half *h_c, half *h_c_ref, size_t size_c,
7799     CUTensorMap *tma_a, CUTensorMap *tma_b,
7800     cudaEvent_t start, cudaEvent_t stop,
7801     float &max_err, float &time_ms) {
7802     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
7803     constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
7804     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
7805                             const CUTensorMap *);
7806     Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
7807                             kNumThreads, kBlockSwizzle>;
7808     if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
7809     cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
7810                         payload_bytes);
7811     dim3 block(kNumThreads);
7812     constexpr int kSwizzleN = 16;
7813     const int n_tiles = N / BN;
7814     const int grid_x = kBlockSwizzle ? div_cceil(n_tiles, kSwizzleN) : n_tiles;
7815     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
7816     dim3 grid(grid_x, M / BM, grid_z);
7817     for (int w = 0; w < g_warmup; w++)
7818         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
7819     cudaDeviceSynchronize();
7820     cudaEventRecord(start);
7821     for (int r = 0; r < g_repeat; r++)
7822         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
7823     cudaEventRecord(stop);
7824     cudaEventSynchronize(stop);
7825     cudaEventElapsedTime(&time_ms, start, stop);
7826     time_ms /= g_repeat;
7827     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
7828     max_err = 0.0f;
7829     for (int i = 0; i < M * N; ++i)
7830         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
7831     return true;
7832 }
7833
7834 static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
7835     constexpr int BM = 128, BN = 128, BK = 64;
7836     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
7837         printf("| %-56s | %-12s | %-4s | %-22s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP",
7838             "SKIP", "None");
7839         return;
7840     }
7841     const size_t size_a = size_t(M) * K * sizeof(half);
7842     const size_t size_b = size_t(K) * N * sizeof(half);
7843     const size_t size_b_t = size_t(N) * K * sizeof(half);
7844     const size_t size_c = size_t(M) * N * sizeof(half);
7845     half *h_a = static_cast<half *>(malloc(size_a));
7846     half *h_b = static_cast<half *>(malloc(size_b));
7847     half *h_b_t = static_cast<half *>(malloc(size_b_t));
7848     half *h_c = static_cast<half *>(malloc(size_c));
7849     half *h_c_ref = static_cast<half *>(malloc(size_c));
7850     srand(42);
7851     for (int i = 0; i < M * K; ++i)

```

```

7852     h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
7853 for (int i = 0; i < K * N; ++i)
7854     h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
7855 for (int n = 0; n < N; ++n)
7856     for (int k = 0; k < K; ++k)
7857         h_b_t[n * K + k] = h_b[k * N + n];
7858 half *d_a, *d_b, *d_b_t, *d_c;
7859 check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
7860 check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
7861 check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
7862 check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
7863 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
7864 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
7865 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
7866         "bench tma mma ws H2D B_t");
7867 cublasHandle_t handle;
7868 cublasCreate(&handle);
7869 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
7870 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
7871             d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
7872             CUBLAS_GEMM_DEFAULT);
7873 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
7874         "bench tma mma ws D2H reference");
7875 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
7876 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
7877 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
7878 cudaEvent_t start, stop;
7879 cudaEventCreate(&start);
7880 cudaEventCreate(&stop);
7881 for (int stages : {1, 2, 3}) {
7882     for (int swizzle : {0, 1}) {
7883         float max_err = 0, time_ms = 0;
7884         bool ok = false;
7885         if (stages == 2)
7886             ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
7887                 h_c_ref, size_c, tma_a, tma_b,
7888                 start, stop, max_err, time_ms)
7889             : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
7890                 h_c_ref, size_c, tma_a, tma_b,
7891                 start, stop, max_err, time_ms);
7892         else
7893             ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
7894                 h_c_ref, size_c, tma_a, tma_b,
7895                 start, stop, max_err, time_ms)
7896             : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
7897                 h_c_ref, size_c, tma_a, tma_b,
7898                 start, stop, max_err, time_ms);
7899         char label[64];
7900         snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
7901         if (!ok) {
7902             printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM SKIP", "SKIP", "None");
7903         } else {
7904             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
7905             char tflops_str[32];
7906             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
7907                 cublas_tflops);
7908             printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
7909                 max_err < 1.0f ? "PASS" : "FAIL", tflops_str);
7909         }
7910     }
7911 }
7912 cudaEventDestroy(start);
7913 cudaEventDestroy(stop);

```

```

7914 free(h_a);
7915 free(h_b);
7916 free(h_b_t);
7917 free(h_c);
7918 free(h_c_ref);
7919 cudaFree(d_a);
7920 cudaFree(d_b);
7921 cudaFree(d_b_t);
7922 cudaFree(d_c);
7923 cudaFree(tma_a);
7924 cudaFree(tma_b);
7925 cublasDestroy(handle);
7926 }
7927 #endif
7928
7929 // =====
7930 // Bench: FlashAttention-2 Split-Q (template on kHeadDim for dispatch)
7931 // =====
7932 template <int kHeadDim, int kStagesK = 2, int kPadQ = 8, int kPadK = 8,
7933         int kPadV = 8, int kMmaAccF32 = 0>
7934 static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
7935         half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o,
7936         float cudnn_tflops_f16) {
7937     static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
7938     constexpr bool kSwizzleQ = kPadQ == 0;
7939     constexpr bool kSwizzleK = kPadK == 0;
7940     constexpr bool kSwizzleV = kPadV == 0;
7941     constexpr int kMmaAtomM = 16;
7942     constexpr int kMmaAtomN = 8;
7943     constexpr int kMmaAtomK = 16;
7944     constexpr int kMmaTileSeqLenQ = 8;
7945     constexpr int kMmaTileSeqLenK = 1;
7946     constexpr int kMmaTileSeqLenP = 8;
7947     constexpr int kMmaTileHeadDimV = 1;
7948     constexpr int kValTileSeqLenQ = 1;
7949     constexpr int kValTileSeqLenK = 8;
7950     constexpr int kValTileSeqLenP = 1;
7951     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
7952     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
7953     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
7954     constexpr const char *layout_name =
7955         kSwizzleQ && kSwizzleK && kSwizzleV ? "Swizzle" :
7956         kSwizzleQ && kSwizzleK ? "SwizzleQK" :
7957         kSwizzleQ && kSwizzleV ? "SwizzleQV" :
7958         kSwizzleK && kSwizzleV ? "SwizzleKV" :
7959         kSwizzleQ ? "SwizzleQ" :
7960         kSwizzleK ? "SwizzleK" :
7961         kSwizzleV ? "SwizzleV" : "Pad";
7962
7963     // Kernel requires seqlen >= Br (tile size); skip gracefully for short seqlen
7964     if (seqlen < Br) {
7965         char label[64];
7966         snprintf(label, sizeof(label), "FlashAttention-2 (D=%d, %s)", kHeadDim,
7967                 layout_name);
7968         printf("| %-56s | %-12s | %-4s | %-22s |\n", label,
7969                 "seqlen<Br", "SKIP", "None");
7970         return;
7971     }
7972     size_t smem_bytes =
7973         (Br * (kHeadDim + kPadQ) +
7974          kStagesK * Bc * (kHeadDim + kPadK) +
7975          Bc * (kHeadDim + kPadV)) * sizeof(half);
7976

```

```

7977 dim3 block(256);
7978 dim3 grid((seqlen + Br - 1) / Br, B * H);
7979
7980 cudaStream_t timing_stream;
7981 cudaStreamCreate(&timing_stream);
7982 cudaEvent_t start, stop;
7983 cudaEventCreate(&start);
7984 cudaEventCreate(&stop);
7985
7986 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
7987 FAKernel fa_k = flash_attn_mma_stages_split_q<
7988     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32,
7989     kMmaTileSeqLenQ, kMmaTileSeqLenK,
7990     kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
7991     kValTileSeqLenP, kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
7992 bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
7993 if (smem_ok) {
7994     cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
7995 }
7996
7997 // Warmup
7998 if (smem_ok) {
7999     for (int w = 0; w < g_warmup; w++)
8000         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
8001     check(cudaStreamSynchronize(timing_stream), "fa warmup sync");
8002 }
8003
8004 // Timed repeat
8005 cudaEventRecord(start, timing_stream);
8006 if (smem_ok) {
8007     for (int r = 0; r < g_repeat; r++)
8008         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
8009 }
8010 cudaEventRecord(stop, timing_stream);
8011 cudaEventSynchronize(stop);
8012
8013 float time_ms = 0;
8014 cudaEventElapsedTime(&time_ms, start, stop);
8015 time_ms /= g_repeat;
8016
8017 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
8018 half *h_o = (half *)malloc(sz);
8019 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");
8020
8021 int count = B * H * seqlen * head_dim;
8022 char label[64];
8023 snprintf(label, sizeof(label), "FlashAttention-2 (S=%d, D=%d, %s, %s)", kStagesK,
8024     kHeadDim, layout_name, kMmaAccF32 ? "F32Acc" : "F16Acc");
8025 float max_err = 0.0f;
8026 bool checked = h_o_ref || ref_o;
8027 if (smem_ok && checked) {
8028     for (int i = 0; i < count; i++) {
8029         float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
8030         float err = fabsf(__half2float(h_o[i]) - ref_val);
8031         if (err > max_err) max_err = err;
8032     }
8033 }
8034 if (smem_ok && checked) {
8035     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
8036     char tflops_str[32];
8037     if (cudnn_tflops_f16 > 0)
8038         snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cudnn_tflops_f16, tflops /
cudnn_tflops_f16);

```



```

8039     else
8040         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
8041         printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
8042             max_err < 5e-1f ? "PASS" : "FAIL", tflops_str);
8043     } else if (smem_ok) {
8044         float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
8045         char tflops_str[32];
8046         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
8047         printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "unchecked", "SKIP", tflops_str);
8048     } else {
8049         printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM too large", "SKIP", "None");
8050     }
8051
8052     cudaEventDestroy(start);
8053     cudaEventDestroy(stop);
8054     cudaStreamDestroy(timing_stream);
8055     free(h_o);
8056 }
8057
8058 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8059 // Bench for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
8060 template <int kHeadDim, int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
8061 static void bench_fa_tma_mma_ws_launch(int B, int H, int seqlen, int head_dim,
8062                                         half *h_o_ref, float *ref_o,
8063                                         half *d_q, half *d_k, half *d_v,
8064                                         half *d_o, float cudnn_tflops_f16) {
8065     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
8066     constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;
8067     constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
8068     constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
8069     constexpr int kValTileSeqLenP = 1;
8070     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
8071     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
8072     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
8073     constexpr int kNumThreads = 384;
8074
8075     if (seqlen < Br || seqlen % Br != 0 || seqlen % Bc != 0) {
8076         char label[64];
8077         snprintf(label, sizeof(label),
8078             "FlashAttn-2 TMA MMA WS (Sk=%d, Sv=%d, D=%d, unaligned)", kStagesK, kStagesV,
8079             (int)kHeadDim);
8080         printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SKIP", "SKIP", "None");
8081         return;
8082     }
8083
8084     // TMA descriptors: box innermost 固定 64 half (128B), 满足
8085     // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
8086     // D=64: blocks_width=1, 单次 TMA 覆盖整行。
8087     // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA,
8088     //         minor_coord = 0/64, 写入 chunk-major smem 布局 [2, Br, 64]。
8089     // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major, 作为 2D
8090     // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
8091     // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
8092     constexpr int kTmaBoxMinor = 64; // box innermost = 64 half = 128B
8093     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
8094     CUTensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
8095         d_q, B * H * seqlen / Br, kTmaChunks);
8096     CUTensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
8097         d_k, B * H * seqlen / Bc, kTmaChunks);
8098     CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
8099         d_v, B * H * seqlen / Bc, kTmaChunks);
8100
8101     using FAKernel = void (*)(half *, half *, half *, half *, int, int,

```

```

8102         const CUTensorMap *, const CUTensorMap *,
8103         const CUTensorMap *);
8104     FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
8105         kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32, kMmaTileSeqLenQ,
8106         kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
8107         kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
8108         kNumThreads>;
8109
8110     size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
8111         kStagesV * Bc * kHeadDim) * sizeof(half);
8112     bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
8113     if (smem_ok) {
8114         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize,
8115             smem_bytes);
8116     }
8117
8118     dim3 block(kNumThreads);
8119     dim3 grid((seqlen + Br - 1) / Br, B * H);
8120
8121     cudaStream_t timing_stream;
8122     cudaStreamCreate(&timing_stream);
8123     cudaEvent_t start, stop;
8124     cudaEventCreate(&start);
8125     cudaEventCreate(&stop);
8126
8127     if (smem_ok) {
8128         for (int w = 0; w < g_warmup; ++w)
8129             fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
8130                 seqlen, H, tma_q, tma_k,
8131                 tma_v);
8132         check(cudaStreamSynchronize(timing_stream), "fa_tma_ws warmup sync");
8133     }
8134     cudaEventRecord(start, timing_stream);
8135     if (smem_ok) {
8136         for (int r = 0; r < g_repeat; ++r)
8137             fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
8138                 seqlen, H, tma_q, tma_k,
8139                 tma_v);
8140     }
8141     cudaEventRecord(stop, timing_stream);
8142     cudaEventSynchronize(stop);
8143
8144     float time_ms = 0;
8145     cudaEventElapsedTime(&time_ms, start, stop);
8146     time_ms /= g_repeat;
8147
8148     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
8149     half *h_o = (half *)malloc(sz);
8150     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
8151     int count = B * H * seqlen * head_dim;
8152
8153     char label[64];
8154     snprintf(label, sizeof(label), "FlashAttention-2 TMA MMA WS (Sk=%d, Sv=%d, D=%d, %s)",
8155         kStagesK, kStagesV, (int)kHeadDim, kMmaAccF32 ? "F32Acc" : "F16Acc");
8156     float max_err = 0.0f;
8157     bool checked = h_o_ref || ref_o;
8158     if (smem_ok && checked) {
8159         for (int i = 0; i < count; ++i) {
8160             float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
8161             float err = fabsf(__half2float(h_o[i]) - ref_val);
8162             if (err > max_err) max_err = err;
8163         }
8164     }

```

```

8165 if (smem_ok && checked) {
8166     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
8167     char tflops_str[32];
8168     if (cudnn_tflops_f16 > 0)
8169         snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cudnn_tflops_f16, tflops /
cudnn_tflops_f16);
8170     else
8171         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
8172     printf("| %-56s | %.6e | %-4s | %-22s |\n", label, max_err,
8173         max_err < 5e-1f ? "PASS" : "FAIL", tflops_str);
8174 } else if (smem_ok) {
8175     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
8176     char tflops_str[32];
8177     snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
8178     printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "unchecked", "SKIP",
8179         tflops_str);
8180 } else {
8181     printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SMEM too large", "SKIP",
8182         "None");
8183 }
8184
8185 cudaEventDestroy(start);
8186 cudaEventDestroy(stop);
8187 cudaStreamDestroy(timing_stream);
8188 free(h_o);
8189 cudaFree(tma_q);
8190 cudaFree(tma_k);
8191 cudaFree(tma_v);
8192 }
8193
8194 // D=64/128 dispatch wrapper for bench
8195 template <int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
8196 static void bench_fa_tma_mma_ws_dispatch(int B, int H, int seqlen, int head_dim,
8197     half *h_o_ref, float *ref_o,
8198     half *d_q, half *d_k, half *d_v,
8199     half *d_o, float cudnn_tflops_f16) {
8200     if (head_dim == 64) {
8201         bench_fa_tma_mma_ws_launch<64, kStagesK, kStagesV, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
8202             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8203     } else if (head_dim == 128) {
8204         bench_fa_tma_mma_ws_launch<128, kStagesK, kStagesV, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
8205             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8206     } else {
8207         char label[64];
8208         snprintf(label, sizeof(label),
8209             "FlashAttn-2 TMA MMA WS (Sk=%d, Sv=%d, D!=64/128)", kStagesK, kStagesV);
8210         printf("| %-56s | %-12s | %-4s | %-22s |\n", label, "SKIP", "SKIP", "None");
8211     }
8212 }
8213 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
8214
8215 #if defined(NOTES_V2_ENABLE_CUDNN)
8216 static float bench_cudnn_sdpa_tflops(half *d_q, half *d_k, half *d_v,
8217     half *d_o_ref, int B, int H, int seqlen,
8218     int head_dim, fe::DataType_t compute_type) {
8219     cudnnHandle_t handle;
8220     cudnnCreate(&handle);
8221     auto graph = std::make_shared<fe::graph::Graph>();
8222     graph->set_io_data_type(fe::DataType_t::HALF)
8223         .set_intermediate_data_type(fe::DataType_t::FLOAT)
8224         .set_compute_data_type(compute_type);
8225
8226     auto Q = graph->tensor(fe::graph::Tensor_attributes()

```

```

8227 .set_uid(1).set_dim({B, H, seqlen, head_dim})
8228 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
8229 auto K = graph->tensor(fe::graph::Tensor_attributes()
8230 .set_uid(2).set_dim({B, H, seqlen, head_dim})
8231 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
8232 auto V = graph->tensor(fe::graph::Tensor_attributes()
8233 .set_uid(3).set_dim({B, H, seqlen, head_dim})
8234 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
8235
8236 auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
8237 fe::graph::SDPA_attributes()
8238 .set_name("sdpa_ref")
8239 .set_attn_scale(1.0f / sqrtf((float)head_dim)));
8240
8241 O_sdpa->set_output(true).set_uid(4)
8242 .set_dim({B, H, seqlen, head_dim})
8243 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
8244
8245 auto build_status = graph->build(handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
8246 float tflops = -1.0f;
8247 if (build_status.is_good()) {
8248     std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
8249         {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
8250     int64_t ws_size = 0;
8251     if (graph->get_workspace_size(ws_size).is_good()) {
8252         int8_t *d_ws = nullptr;
8253         if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench cudnn ws");
8254         if (graph->execute(handle, vp, d_ws).is_good()) {
8255             check(cudaDeviceSynchronize(), "bench cudnn sync");
8256             for (int w = 1; w < g_warmup; ++w)
8257                 (void)graph->execute(handle, vp, d_ws);
8258             cudaDeviceSynchronize();
8259             cudaEvent_t ev_s, ev_e;
8260             cudaEventCreate(&ev_s); cudaEventCreate(&ev_e);
8261             cudaEventRecord(ev_s);
8262             for (int r = 0; r < g_repeat; ++r)
8263                 (void)graph->execute(handle, vp, d_ws);
8264             cudaEventRecord(ev_e);
8265             cudaEventSynchronize(ev_e);
8266             float time_ms = 0;
8267             cudaEventElapsedTime(&time_ms, ev_s, ev_e);
8268             time_ms /= g_repeat;
8269             tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
8270             cudaEventDestroy(ev_s); cudaEventDestroy(ev_e);
8271         }
8272         if (d_ws) cudaFree(d_ws);
8273     }
8274 }
8275 cudnnDestroy(handle);
8276 return tflops;
8277 }
8278 #endif
8279
8280 static void bench_flash_attn(int B, int H, int N, int D) {
8281     int seqlen = N, head_dim = D;
8282
8283     if (head_dim != 64 && head_dim != 128) {
8284         printf("| %-56s | %-12s | %-4s | %-22s |\n", "FlashAttention-2", "unsupported D", "SKIP", "None");
8285         return;
8286     }
8287
8288     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
8289

```

```

8290 srand(42);
8291 half *h_q = (half *)malloc(sz);
8292 half *h_k = (half *)malloc(sz);
8293 half *h_v = (half *)malloc(sz);
8294 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
8295     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8296     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8297     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8298 }
8299
8300 // Device allocations — shared by kernel and reference
8301 half *d_q, *d_k, *d_v, *d_o;
8302 check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
8303 check(cudaMalloc(&d_k, sz), "bench fa alloc K");
8304 check(cudaMalloc(&d_v, sz), "bench fa alloc V");
8305 check(cudaMalloc(&d_o, sz), "bench fa alloc O");
8306 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
8307 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
8308 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
8309
8310 // Reference output: either cuDNN (half) or CPU (float)
8311 half *h_o_ref = nullptr;
8312 float *ref_o = nullptr;
8313 float cudnn_tflops_f16 = -1.0f, cudnn_tflops_f32 = -1.0f;
8314 int ref_count = B * H * seqlen * head_dim;
8315
8316 #if defined(NOTES_V2_ENABLE_CUDNN)
8317 if (!g_fa_skip_check) {
8318     half *d_o_ref;
8319     check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
8320     cudnn_tflops_f16 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
8321                                                B, H, seqlen, head_dim,
8322                                                fe::DataType_t::HALF);
8323     bool cudnn_ok = (cudnn_tflops_f16 > 0);
8324     if (cudnn_ok) {
8325         cudnn_tflops_f32 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
8326                                                    B, H, seqlen, head_dim,
8327                                                    fe::DataType_t::FLOAT);
8328         h_o_ref = (half *)malloc(sz);
8329         check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
8330     } else {
8331         fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
8332     }
8333     cudaFree(d_o_ref);
8334 }
8335 #endif
8336
8337 // CPU reference fallback
8338 if (!g_fa_skip_check && !h_o_ref) {
8339     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
8340     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
8341     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
8342     ref_o = (float *)malloc(sz * 4 / sizeof(half));
8343     for (int i = 0; i < ref_count; i++) {
8344         ref_q[i] = __half2float(h_q[i]);
8345         ref_k[i] = __half2float(h_k[i]);
8346         ref_v[i] = __half2float(h_v[i]);
8347     }
8348
8349     float scale = 1.0f / sqrtf((float)head_dim);
8350     for (int bi = 0; bi < B * H; bi++) {
8351         for (int qi = 0; qi < seqlen; qi++) {
8352             float smax = -INFINITY;

```

```

8353     float *S = (float *)malloc((size_t)seqlen * sizeof(float));
8354     for (int kj = 0; kj < seqlen; kj++) {
8355         float s = 0.0f;
8356         for (int d = 0; d < head_dim; d++)
8357             s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
8358                 ref_k[bi * seqlen * head_dim + kj * head_dim + d];
8359         S[kj] = s * scale;
8360         if (S[kj] > smax) smax = S[kj];
8361     }
8362     double sum_exp = 0.0;
8363     for (int kj = 0; kj < seqlen; kj++)
8364         sum_exp += (double)expf(S[kj] - smax);
8365     float inv_sum = 1.0f / (float)sum_exp;
8366     for (int d = 0; d < head_dim; d++) {
8367         double o_acc = 0.0;
8368         for (int kj = 0; kj < seqlen; kj++)
8369             o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
8370                 ref_v[bi * seqlen * head_dim + kj * head_dim + d];
8371         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
8372     }
8373     free(S);
8374 }
8375 }
8376 free(ref_q);
8377 free(ref_k);
8378 free(ref_v);
8379 }
8380
8381 if (head_dim == 64) {
8382     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
8383         cudaDeviceSynchronize();
8384         bench_fa_launch<64, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8385             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8386         cudaDeviceSynchronize();
8387         bench_fa_launch<64, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8388             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8389         cudaDeviceSynchronize();
8390         bench_fa_launch<64, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8391             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8392         cudaDeviceSynchronize();
8393         bench_fa_launch<64, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8394             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8395     }
8396     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
8397         cudaDeviceSynchronize();
8398         bench_fa_launch<64, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8399             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8400         cudaDeviceSynchronize();
8401         bench_fa_launch<64, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8402             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8403     }
8404     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
8405         cudaDeviceSynchronize();
8406         bench_fa_launch<64, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8407             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8408         cudaDeviceSynchronize();
8409         bench_fa_launch<64, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8410             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8411     }
8412     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
8413         cudaDeviceSynchronize();
8414         bench_fa_launch<64, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8415             d_q, d_k, d_v, d_o, cudnn_tflops_f16);

```



```

8416     cudaDeviceSynchronize();
8417     bench_fa_launch<64, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8418                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8419 }
8420 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
8421     cudaDeviceSynchronize();
8422     bench_fa_launch<64, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8423                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8424     cudaDeviceSynchronize();
8425     bench_fa_launch<64, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8426                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8427 }
8428 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
8429     cudaDeviceSynchronize();
8430     bench_fa_launch<64, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8431                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8432     cudaDeviceSynchronize();
8433     bench_fa_launch<64, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8434                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8435 }
8436 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
8437     cudaDeviceSynchronize();
8438     bench_fa_launch<64, 2, 8, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8439                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8440     cudaDeviceSynchronize();
8441     bench_fa_launch<64, 2, 8, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8442                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8443 }
8444 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
8445     cudaDeviceSynchronize();
8446     bench_fa_launch<64, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8447                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8448     cudaDeviceSynchronize();
8449     bench_fa_launch<64, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8450                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8451 }
8452 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8453 {
8454     cudaDeviceSynchronize();
8455     bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8456                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8457     cudaDeviceSynchronize();
8458     bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8459                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8460     cudaDeviceSynchronize();
8461     bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8462                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8463     cudaDeviceSynchronize();
8464     bench_fa_tma_mma_ws_dispatch<4, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8465                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8466     cudaDeviceSynchronize();
8467     bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8468                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8469     // F32Acc variants
8470     cudaDeviceSynchronize();
8471     bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8472                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8473     cudaDeviceSynchronize();
8474     bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8475                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8476     cudaDeviceSynchronize();
8477     bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8478                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);

```

```

8479     cudaDeviceSynchronize();
8480     bench_fa_tma_mma_ws_dispatch<4, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8481         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8482     cudaDeviceSynchronize();
8483     bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
8484         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8485 }
8486 #endif
8487 } else {
8488     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
8489         cudaDeviceSynchronize();
8490         bench_fa_launch<128, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8491             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8492         cudaDeviceSynchronize();
8493         bench_fa_launch<128, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8494             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8495         cudaDeviceSynchronize();
8496         bench_fa_launch<128, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8497             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8498         cudaDeviceSynchronize();
8499         bench_fa_launch<128, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8500             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8501     }
8502     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
8503         cudaDeviceSynchronize();
8504         bench_fa_launch<128, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8505             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8506         cudaDeviceSynchronize();
8507         bench_fa_launch<128, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8508             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8509     }
8510     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
8511         cudaDeviceSynchronize();
8512         bench_fa_launch<128, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8513             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8514         cudaDeviceSynchronize();
8515         bench_fa_launch<128, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8516             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8517     }
8518     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
8519         cudaDeviceSynchronize();
8520         bench_fa_launch<128, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8521             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8522         cudaDeviceSynchronize();
8523         bench_fa_launch<128, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8524             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8525     }
8526     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
8527         cudaDeviceSynchronize();
8528         bench_fa_launch<128, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8529             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8530         cudaDeviceSynchronize();
8531         bench_fa_launch<128, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8532             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8533     }
8534     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
8535         cudaDeviceSynchronize();
8536         bench_fa_launch<128, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8537             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8538         cudaDeviceSynchronize();
8539         bench_fa_launch<128, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8540             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8541     }

```

```

8542 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
8543     cudaDeviceSynchronize();
8544     bench_fa_launch<128, 2, 8, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8545                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8546     cudaDeviceSynchronize();
8547     bench_fa_launch<128, 2, 8, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8548                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8549 }
8550 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
8551     cudaDeviceSynchronize();
8552     bench_fa_launch<128, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8553                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8554     cudaDeviceSynchronize();
8555     bench_fa_launch<128, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8556                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8557 }
8558 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8559 {
8560     cudaDeviceSynchronize();
8561     bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8562                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8563     cudaDeviceSynchronize();
8564     bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8565                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8566     cudaDeviceSynchronize();
8567     bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8568                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8569     cudaDeviceSynchronize();
8570     bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
8571                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
8572     // F32Acc variants
8573     cudaDeviceSynchronize();
8574     bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8575                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8576     cudaDeviceSynchronize();
8577     bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8578                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8579     cudaDeviceSynchronize();
8580     bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
8581                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8582     cudaDeviceSynchronize();
8583     bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
8584                                         ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
8585 }
8586 #endif
8587 }
8588 cudaDeviceSynchronize();
8589
8590 free(h_q);
8591 free(h_k);
8592 free(h_v);
8593 free(h_o_ref);
8594 free(ref_o);
8595 cudaFree(d_q);
8596 cudaFree(d_k);
8597 cudaFree(d_v);
8598 cudaFree(d_o);
8599 }
8600
8601 // Host 端 v1/v2 等价性测试: 遍历 kColStride/i/j, 断言 swizzle_v1_impl == swizzle_v2_impl.
8602 // 用于在启用 NOTES_V2_ENABLE_SWIZZLE_V2 前后验证 v2 与 v1 bit-exact 等价.
8603 // kColStride=8 在 v2 内回退 v1, 故 8 也应恒等.
8604 template <int kColStride>

```

```

8605 static void swizzle_equiv_check_one(long &total, long &fail) {
8606     for (int i = 0; i < 256; ++i) {
8607         for (int j = 0; j < kColStride; ++j) {
8608             int v1 = swizzle_v1_impl<kColStride>(i, j);
8609             int v2 = swizzle_v2_impl<kColStride>(i, j);
8610             ++total;
8611             if (v1 != v2) {
8612                 ++fail;
8613                 if (fail <= 10)
8614                     printf(" MISMATCH cs=%d i=%d j=%d: v1=%d v2=%d\n", kColStride, i, j, v1, v2);
8615             }
8616         }
8617     }
8618 }
8619
8620 static void test_swizzle_equiv() {
8621     long total = 0, fail = 0;
8622     swizzle_equiv_check_one<8>(total, fail);
8623     swizzle_equiv_check_one<16>(total, fail);
8624     swizzle_equiv_check_one<32>(total, fail);
8625     swizzle_equiv_check_one<64>(total, fail);
8626     printf("| %-56s | %-12s | %-4s | %-22s |\n",
8627           "Swizzle v1/v2 equiv (host)", "N/A",
8628           fail == 0 ? "PASS" : "FAIL",
8629           fail == 0 ? "ALL PASS" : "FAIL");
8630     printf(" total=%ld fail=%ld\n", total, fail);
8631 }
8632
8633 int main(int argc, char *argv[]) {
8634     #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8635     cuInit(0);
8636     #endif
8637
8638     for (int i = 1; i < argc; i++) {
8639         if (strcmp(argv[i], "--bench-hgemm") == 0) {
8640             g_bench_hgemm = true;
8641         } else if (strcmp(argv[i], "--bench") == 0) {
8642             g_bench_hgemm = true;
8643             g_bench_fa = true;
8644         } else if (strcmp(argv[i], "--bench-fa") == 0) {
8645             g_bench_fa = true;
8646         } else if (strcmp(argv[i], "--bench-hgemm-all") == 0) {
8647             g_bench_hgemm_all = true;
8648         } else if (strcmp(argv[i], "--bench-fa-all") == 0) {
8649             g_bench_fa = true;
8650             g_fa_layout = FALayout::All;
8651         } else if (strcmp(argv[i], "--bench-all") == 0) {
8652             g_bench_all = true;
8653             g_fa_layout = FALayout::All;
8654         } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
8655             sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
8656         } else if (strcmp(argv[i], "--bhnd") == 0 && i + 1 < argc) {
8657             sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_H, &g_bench_Nfa, &g_bench_D);
8658         } else if (strcmp(argv[i], "--fa-layout") == 0 && i + 1 < argc) {
8659             const char *layout = argv[++i];
8660             if (strcmp(layout, "all") == 0)
8661                 g_fa_layout = FALayout::All;
8662             else if (strcmp(layout, "pad") == 0)
8663                 g_fa_layout = FALayout::Pad;
8664             else if (strcmp(layout, "swizzle-q") == 0)
8665                 g_fa_layout = FALayout::SwizzleQ;
8666             else if (strcmp(layout, "swizzle-k") == 0)
8667                 g_fa_layout = FALayout::SwizzleK;

```

```

8668     else if (strcmp(layout, "swizzle-v") == 0)
8669         g_fa_layout = FALayout::SwizzleV;
8670     else if (strcmp(layout, "swizzle-qk") == 0)
8671         g_fa_layout = FALayout::SwizzleQK;
8672     else if (strcmp(layout, "swizzle-qv") == 0)
8673         g_fa_layout = FALayout::SwizzleQV;
8674     else if (strcmp(layout, "swizzle-kv") == 0)
8675         g_fa_layout = FALayout::SwizzleKV;
8676     else if (strcmp(layout, "swizzle") == 0)
8677         g_fa_layout = FALayout::Swizzle;
8678     else {
8679         fprintf(stderr, "Unsupported FA layout: %s\n", layout);
8680         return EXIT_FAILURE;
8681     }
8682 } else if (strcmp(argv[i], "--fa-skip-check") == 0) {
8683     g_fa_skip_check = true;
8684 } else if (strcmp(argv[i], "--swizzle-eq-check") == 0) {
8685     g_swizzle_eq_check = true;
8686 } else if (strcmp(argv[i], "--warmup") == 0 && i + 1 < argc) {
8687     g_warmup = atoi(argv[++i]);
8688 } else if (strcmp(argv[i], "--repeat") == 0 && i + 1 < argc) {
8689     g_repeat = atoi(argv[++i]);
8690 }
8691 }
8692
8693 if (g_swizzle_eq_check) {
8694     printf("=== notes-v2.cu swizzle v1/v2 equivalence check ===\n");
8695     printf("| %-56s | %-12s | %-4s | %-22s |\n", "Kernel", "Max Err", "Pass", "TFLOPS vs cu{BLAS,DNN}");
8696     printf("
8697 |-----|-----|-----|-----|\n");
8698     test_swizzle_equiv();
8699     printf("=== Done ===\n");
8700     return 0;
8701 }
8702
8703 if (g_bench_hgemm || g_bench_fa || g_bench_all) {
8704     printf("=== notes-v2.cu bench mode ===\n");
8705     printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
8706           g_bench_M, g_bench_N, g_bench_K,
8707           g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
8708     printf("| %-56s | %-12s | %-4s | %-22s |\n", "Kernel", "Max Err", "Pass", "TFLOPS vs cu{BLAS,DNN}");
8709     printf("
8710 |-----|-----|-----|-----|\n");
8711
8712     if (g_bench_hgemm || g_bench_hgemm_all || g_bench_all) {
8713         #if defined(NOTES_V2_ENABLE_CUTE)
8714         bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
8715         #endif
8716         if (g_bench_hgemm_all || g_bench_all) {
8717             bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
8718             bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
8719         }
8720         #if defined(NOTES_V2_ENABLE_WGMMMA)
8721         bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
8722         #endif
8723         #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8724         bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
8725         #endif
8726     }
8727     if (g_bench_fa || g_bench_all)
8728         bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);

```

```

8727     printf("=== Bench done ===\n");
8728     return 0;
8729 }
8730
8731 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8732 if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
8733     int M = 128, N = 128, K = 64;
8734     if (argc > 4) {
8735         M = atoi(argv[2]);
8736         N = atoi(argv[3]);
8737         K = atoi(argv[4]);
8738     }
8739     printf("=== SM120 TMA MMA WS validation ===\n");
8740     printf("| %-56s | %-12s | %-4s | %-22s |\n", "Kernel", "Max Err", "Pass", "TFLOPS vs cu{BLAS,DNN}");
8741     printf("
8742 |-----|-----|-----|-----|\n");
8743     test_hgemm_tma_mma_ws(M, N, K);
8744     return 0;
8745 }
8746 #endif
8747 int M = 1024, N = 1024, K = 1024;
8748 if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
8749
8750 printf("=== notes-v2.cu verification harness ===\n");
8751 printf("| %-56s | %-12s | %-4s | %-22s |\n", "Kernel", "Max Err", "Pass", "TFLOPS vs cu{BLAS,DNN}");
8752 printf("
8753 |-----|-----|-----|-----|\n");
8754 test_block_reduce(N);
8755 test_dot(N);
8756 test_relu(1024);
8757 test_elementwise(1024);
8758 test_histogram(1024);
8759 test_merge_attn_states(512, 16, 128);
8760 test_softmax(256);
8761 test_rms_norm(8, 128);
8762 test_layer_norm(8, 128);
8763 test_rope(8, 128);
8764 test_mat_transpose(256, 256);
8765 test_mat_transpose_padded(256, 256);
8766 test_sgemv(256, 128);
8767 test_sgemm(M, N, K);
8768 test_hgemm_mma(M, N, K);
8769 test_hgemm_swizzle(M, N, K);
8770 #if defined(NOTES_V2_ENABLE_CUTE)
8771     test_hgemm_cute(M, N, K);
8772 #endif
8773 #if defined(NOTES_V2_ENABLE_WGMMA)
8774     test_hgemm_wgmma(M, N, K);
8775 #endif
8776 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8777     test_hgemm_tma_mma_ws(M, N, K);
8778 #endif
8779     test_flash_attn(1024, 64);
8780 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8781     test_flash_attn_tma_mma_ws(1024, 64);
8782     test_flash_attn_tma_mma_ws(1024, 128);
8783 #endif
8784
8785     printf("=== All tests done ===\n");

```

```

8786     return 0;
8787 }
8788
8789 // =====
8790 // Quick build & run reference
8791 // =====
8792 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
8793 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
8794 //   -I ../../third-party/cutlass/include \
8795 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
8796 //
8797 // # sm_89 单独编译 + 运行:
8798 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
8799 //
8800 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
8801 //   项目内置 third-party/cutlass) :
8802 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
8803 //   -I ../../third-party/cutlass/include \
8804 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
8805 //
8806 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
8807 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
8808 //   -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
8809 //
8810 // # sm_90a + CuTe + WGMMMA:
8811 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
8812 //   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMMA \
8813 //   -I ../../third-party/cutlass/include \
8814 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
8815 //
8816 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
8817 //   RTX PRO 5000 / RTX 5090):
8818 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
8819 //   -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
8820 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
8821 //
8822 // # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
8823 //   requires cudnn-frontend submodule: git submodule update --init):
8824 // nvcc -std=c++20 -O2 -arch=sm_120a \
8825 //   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
8826 //   -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
8827 //   -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
8828 //   -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
8829 // # Default FA bench (kPadQ=kPadK=kPadV=8 only):
8830 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa --bhnd 1,48,4096,64
8831 // # All FA layout variants:
8832 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa-all --bhnd 1,48,4096,64
8833 // # FA TMA MMA WS bench (SM120, D=64 only, kStagesK=2/3):
8834 // ./notes_v2_cute_ws_sm120a.bin --bench-fa-tma-ws --bhnd 1,32,4096,64

```